US 20250278190A1

(19) **United States**

(12) **Patent Application Publication** (10) **Pub. No.: US 2025/0278190 A1**

Viitanen et al. (43) **Pub. Date: Sep. 4, 2025**

(54) **IN-PLACE DATA MANAGEMENT WITHIN MEMORY BUFFERS**

(71) Applicant: **Nvidia Corporation**, Santa Clara, CA (US)

(72) Inventors: **Timo Tapani Viitanen**, Helsinki (FI); **Anders Jakob Lindqvist**, Limhamn (SE)

(21) Appl. No.: **18/591,882**

(22) Filed: **Feb. 29, 2024**

Publication Classification

(51) **Int. Cl.**

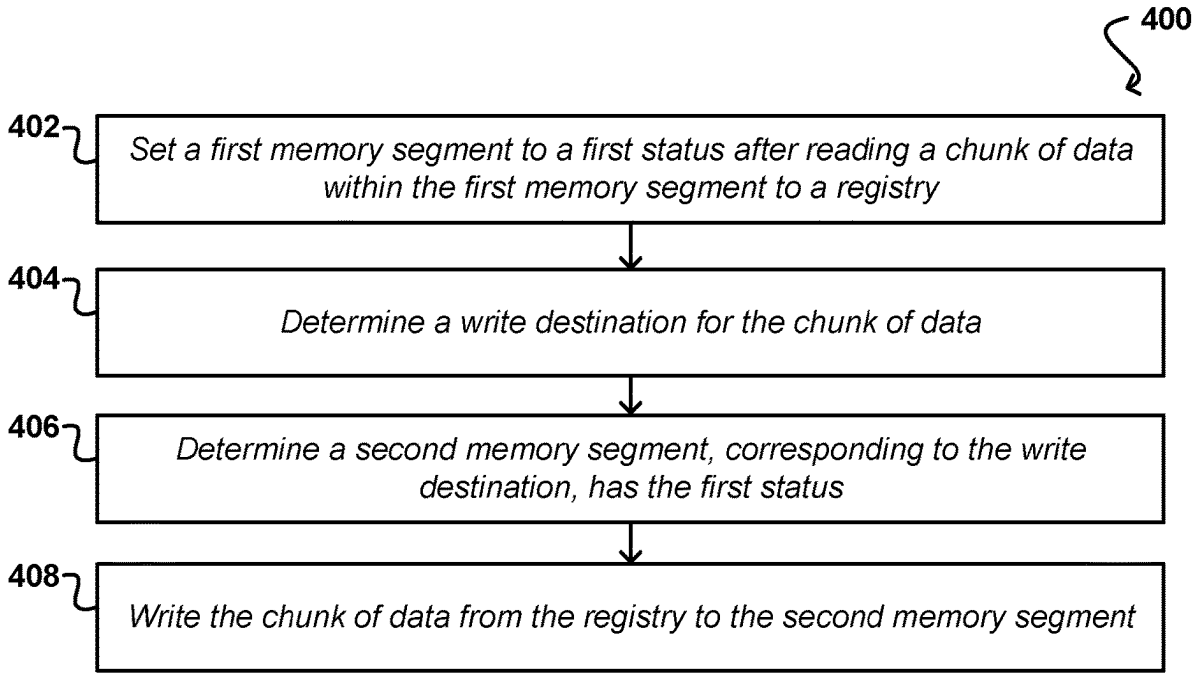
G06F 3/06	(2006.01)
G06T 1/20	(2006.01)
G06T 1/60	(2006.01)

(52) **U.S. Cl.**

CPC ***G06F 3/0611*** (2013.01); ***G06F 3/0659*** (2013.01); ***G06F 3/0673*** (2013.01); ***G06T 1/20*** (2013.01); ***G06T 1/60*** (2013.01)

(57) **ABSTRACT**

Approaches presented herein provide systems and methods for reading a portion of data from an unprocessed memory segment to a registry and changing a status identifier for the unprocessed memory segment indicative of the data being read to the registry. The data may then be written to a destination memory segment from the registry. If a corresponding status identifier for the destination memory segments meets a value indicative that the destination memory segment has not yet been read into the registry, it may be read into the registry before overwriting it, and recursively written into its own destination memory segment.



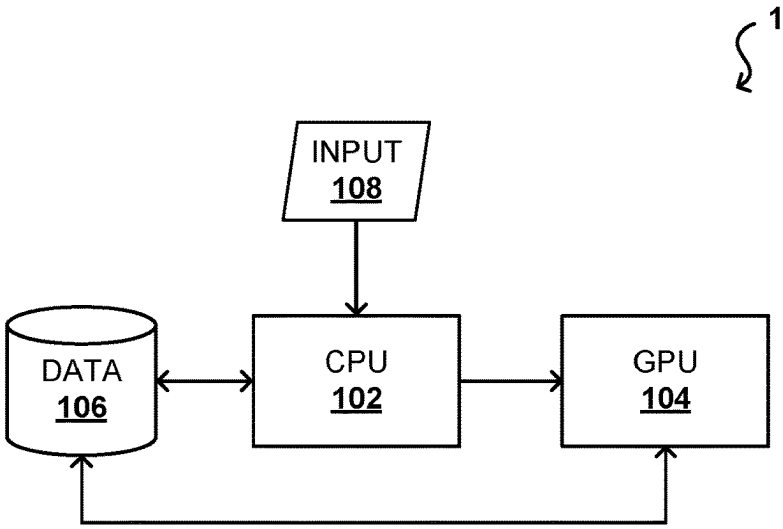

100
INPUT
108
DATA
106
CPU
102
GPU
104

FIG. 1A

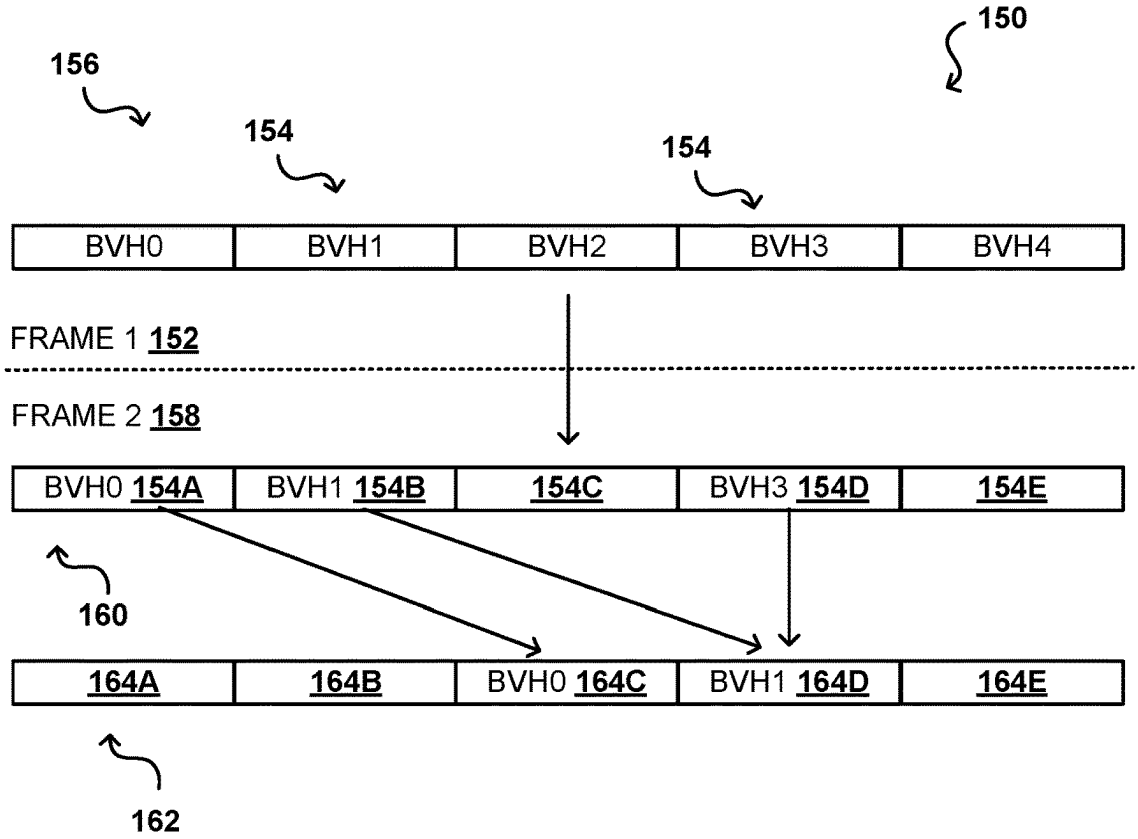

150
156
154
154
BVH0
BVH1
BVH2
BVH3
BVH4
FRAME 1 152
FRAME 2 158
BVH0 154A
BVH1 154B
154C
BVH3 154D
154E
160
164A
164B
BVH0 164C
BVH1 164D
164E
162

FIG. 1B

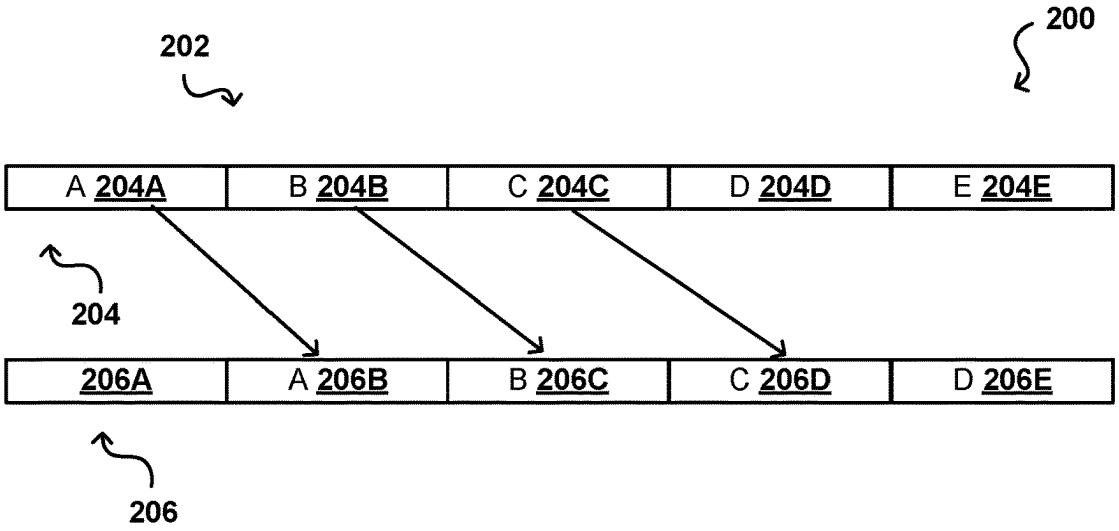

202
200
A 204A
B 204B
C 204C
D 204D
E 204E
204
206A
A 206B
B 206C
C 206D
D 206E
206

FIG. 2A

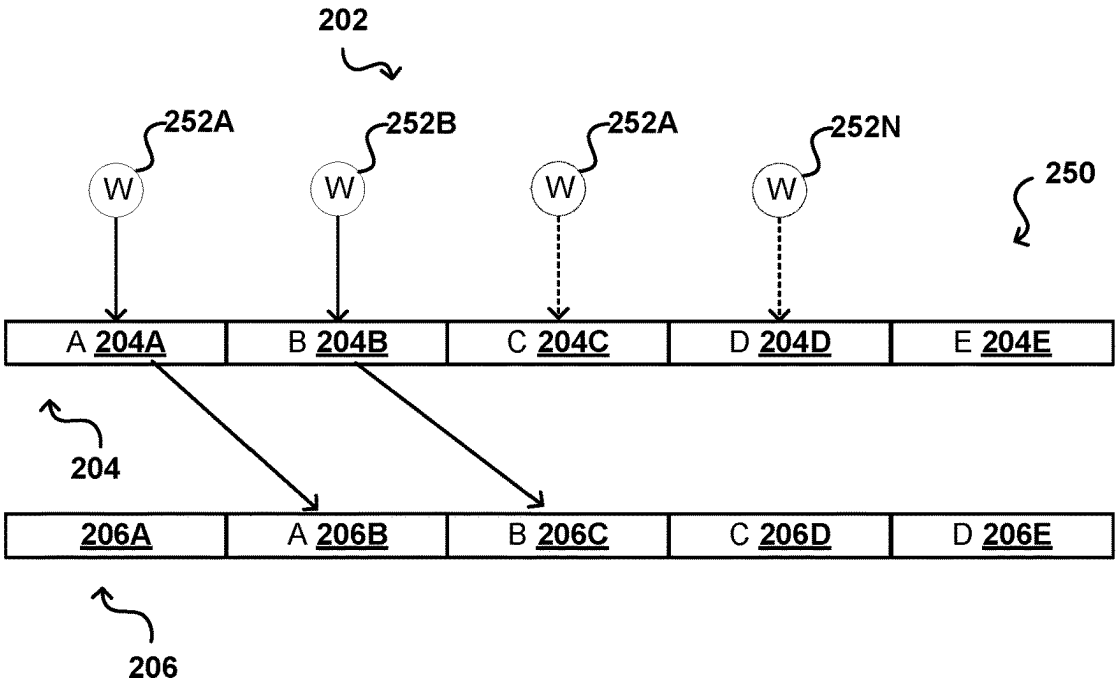
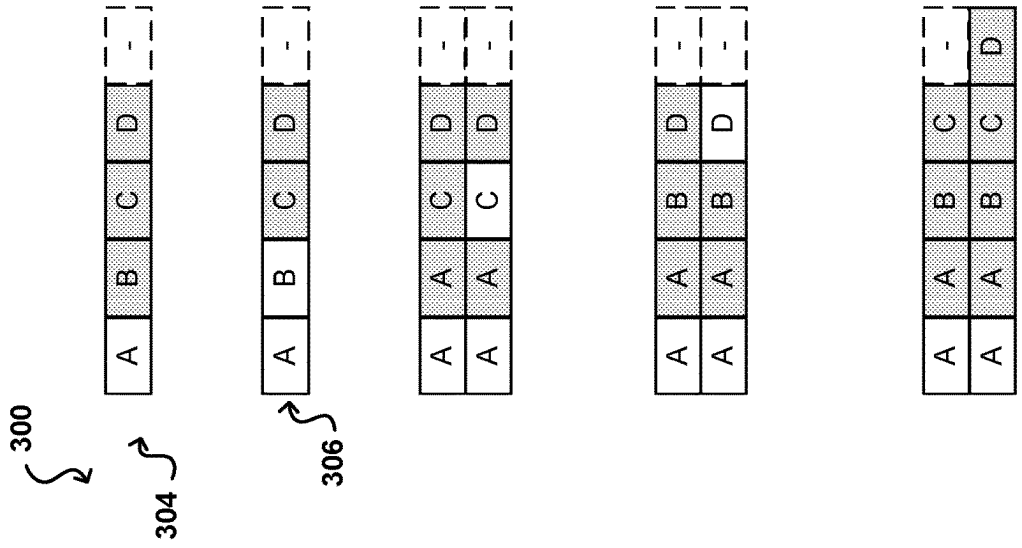

202
252A
252B
252A
252N
W
W
W
W
250
A 204A
B 204B
C 204C
D 204D
E 204E
204
206A
A 206B
B 206C
C 206D
D 206E
206

FIG. 2B


300
304
306
A
B
C
D
-

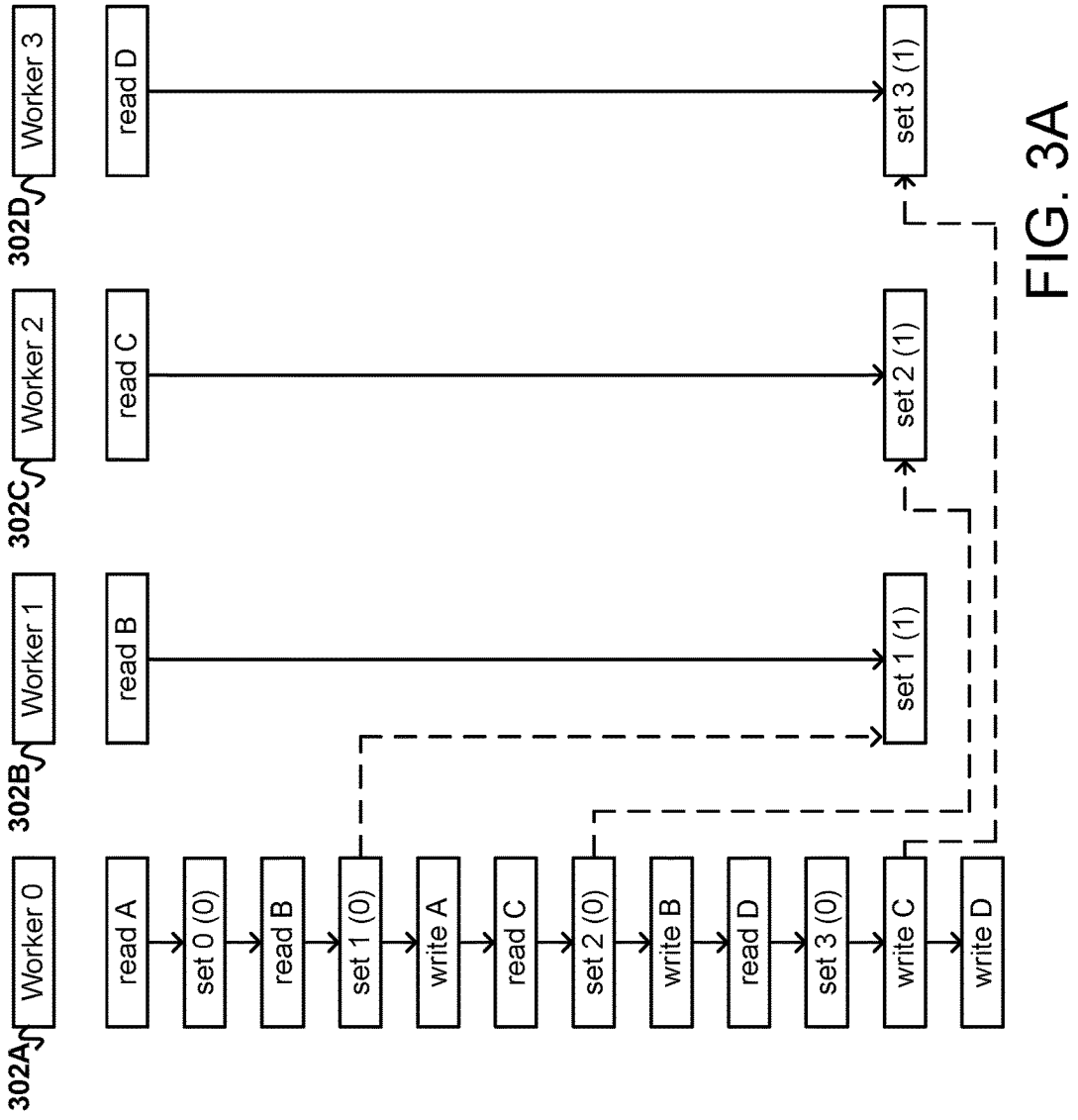

302A
Worker 0
read A
set 0 (0)
read B
set 1 (0)
write A
read C
set 2 (0)
write B
read D
set 3 (0)
write C
write D
302B
Worker 1
read B
set 1 (1)
302C
Worker 2
read C
set 2 (1)
302D
Worker 3
read D
set 3 (1)

FIG. 3A

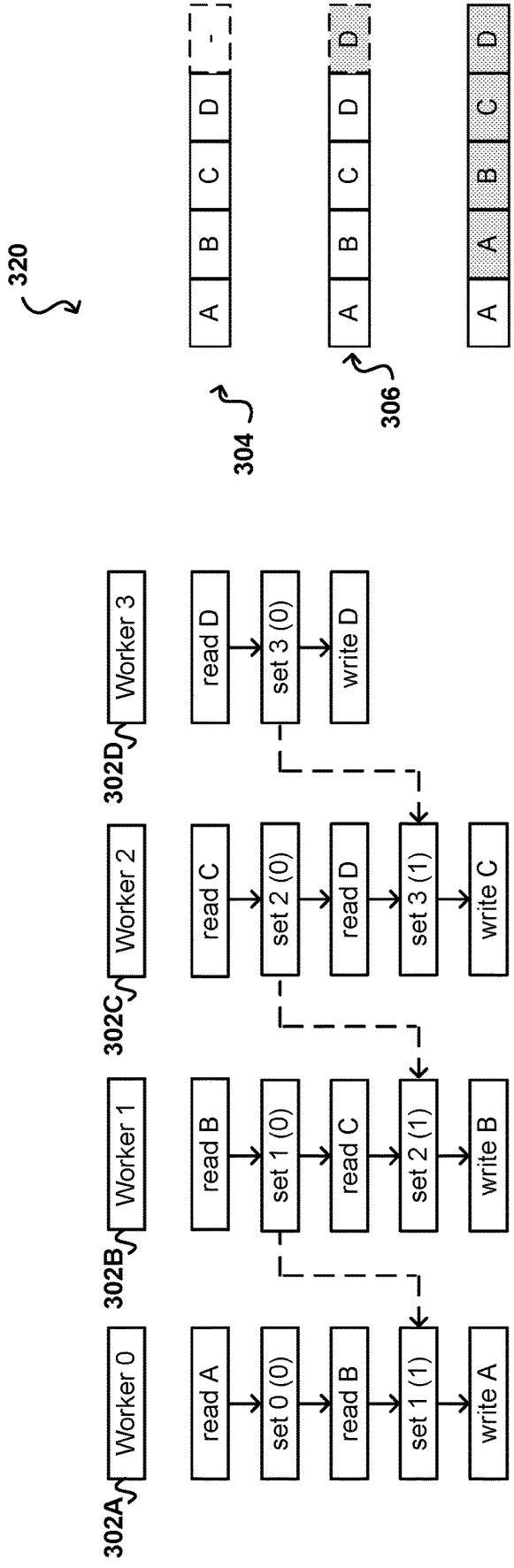

320
A
B
C
D
-
304
A
B
C
D
D
306
A
A
B
C
D
302A
Worker 0
read A
set 0 (0)
read B
set 1 (1)
write A
302B
Worker 1
read B
set 1 (0)
read C
set 2 (1)
write B
302C
Worker 2
read C
set 2 (0)
read D
set 3 (1)
write C
302D
Worker 3
read D
set 3 (0)
write D

FIG. 3B

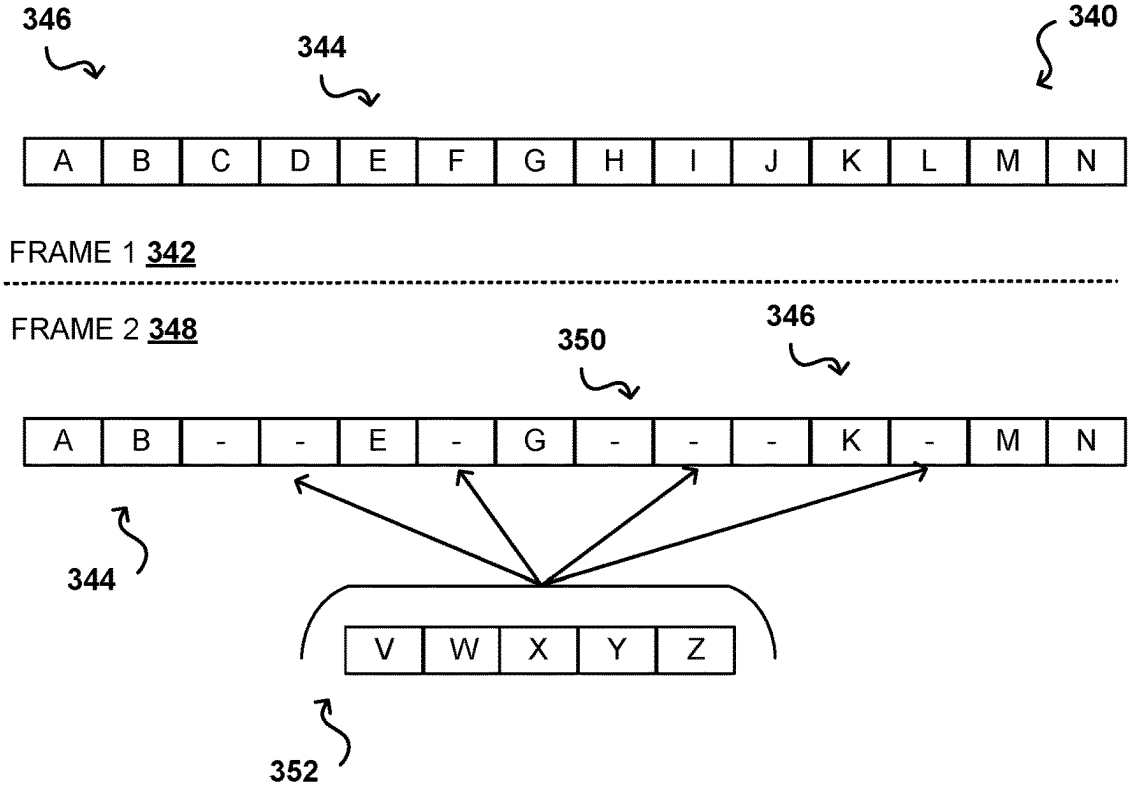

346
344
340
A B C D E F G H I J K L M N
FRAME 1 342
FRAME 2 348
350
346
A B - - E - G - - - K - M N
344
V W X Y Z
352

FIG. 3C

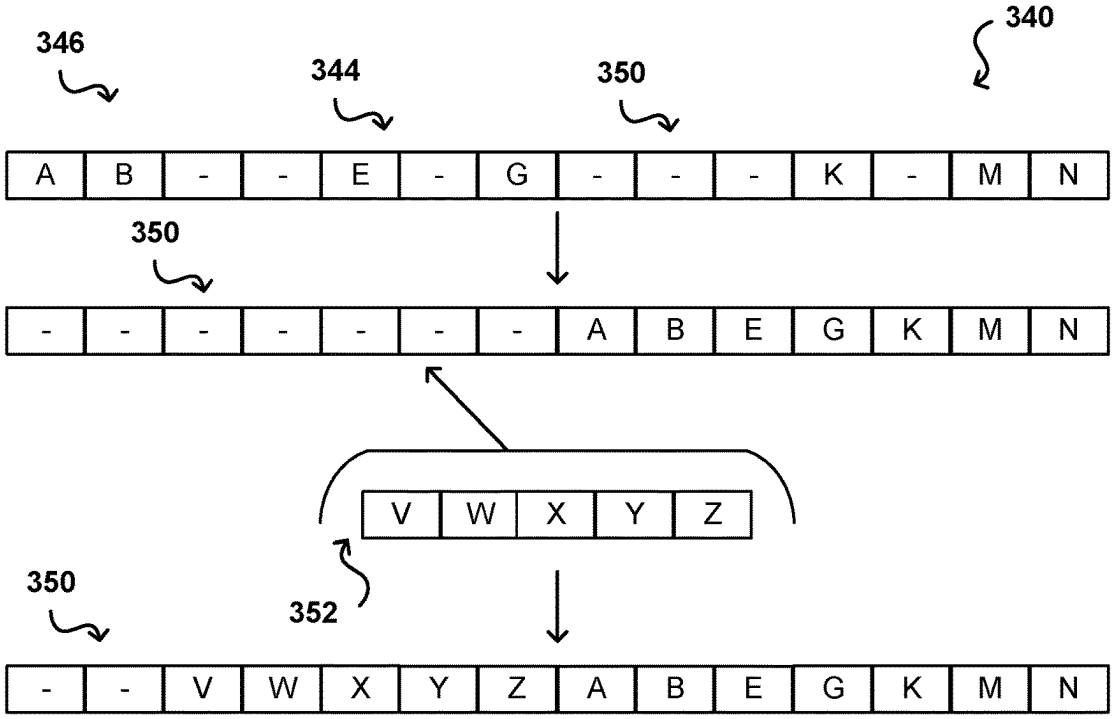

346
344
350
340
A B - - E - G - - - K - M N
350
- - - - - - - A B E G K M N
V W X Y Z
352
350
- - V W X Y Z A B E G K M N

FIG. 3D

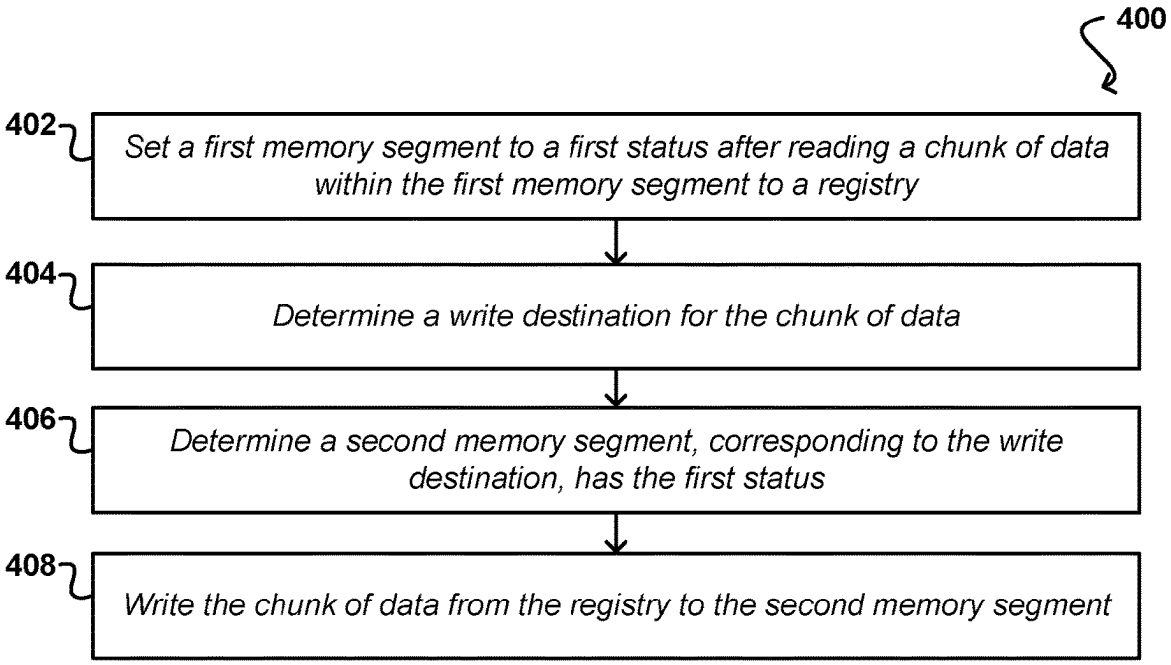

400
402
Set a first memory segment to a first status after reading a chunk of data within the first memory segment to a registry
404
Determine a write destination for the chunk of data
406
Determine a second memory segment, corresponding to the write destination, has the first status
408
Write the chunk of data from the registry to the second memory segment

FIG. 4A

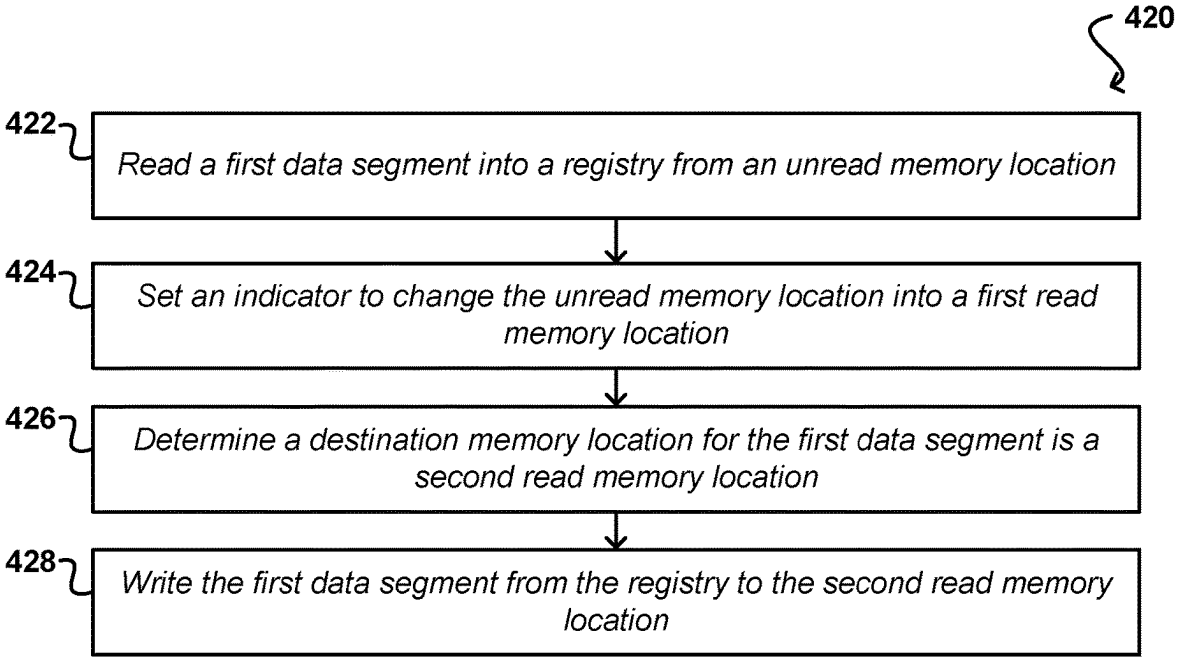

420
422
Read a first data segment into a registry from an unread memory location
424
Set an indicator to change the unread memory location into a first read memory location
426
Determine a destination memory location for the first data segment is a second read memory location
428
Write the first data segment from the registry to the second read memory location

FIG. 4B

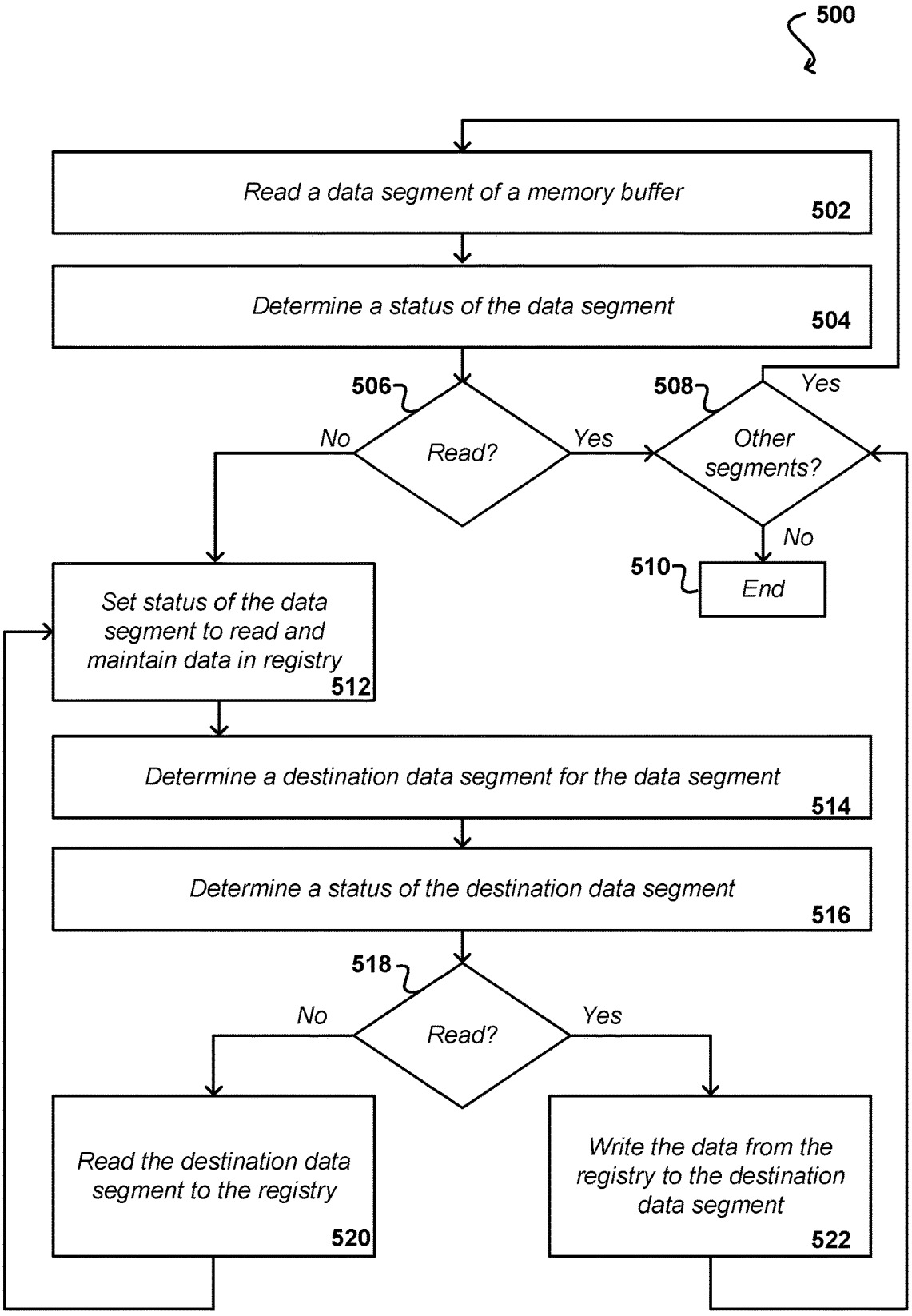

500
Read a data segment of a memory buffer
502
Determine a status of the data segment
504
506
Read?
No
Yes
508
Other segments?
Yes
No
510
End
Set status of the data segment to read and maintain data in registry
512
Determine a destination data segment for the data segment
514
Determine a status of the destination data segment
516
518
Read?
No
Yes
Read the destination data segment to the registry
520
Write the data from the registry to the destination data segment
522

FIG. 5

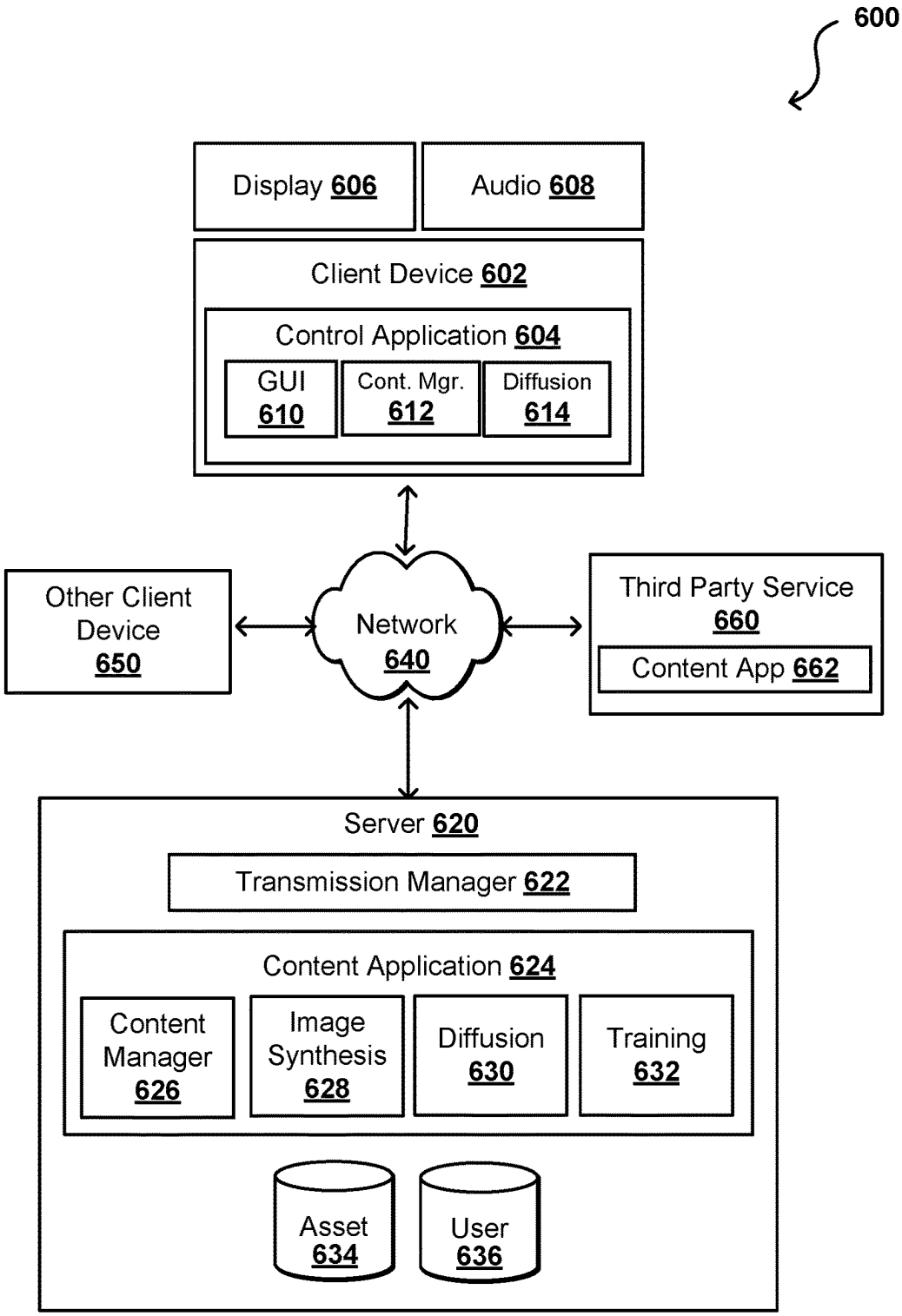

600
Display 606
Audio 608
Client Device 602
Control Application 604
GUI 610
Cont. Mgr. 612
Diffusion 614
Other Client Device 650
Network 640
Third Party Service 660
Content App 662
Server 620
Transmission Manager 622
Content Application 624
Content Manager 626
Image Synthesis 628
Diffusion 630
Training 632
Asset 634
User 636

FIG. 6

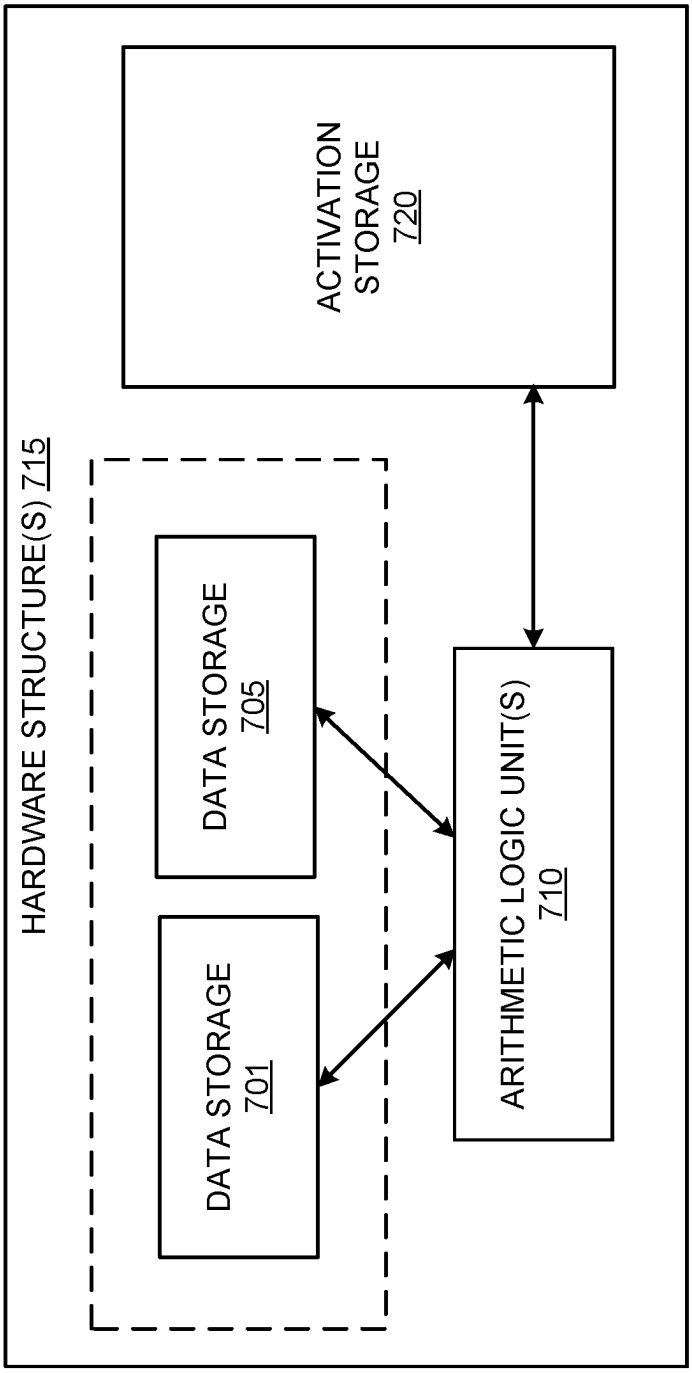

ACTIVATION STORAGE 720
HARDWARE STRUCTURE(S) 715
DATA STORAGE 705
DATA STORAGE 701
ARITHMETIC LOGIC UNIT(S) 710

FIG. 7A

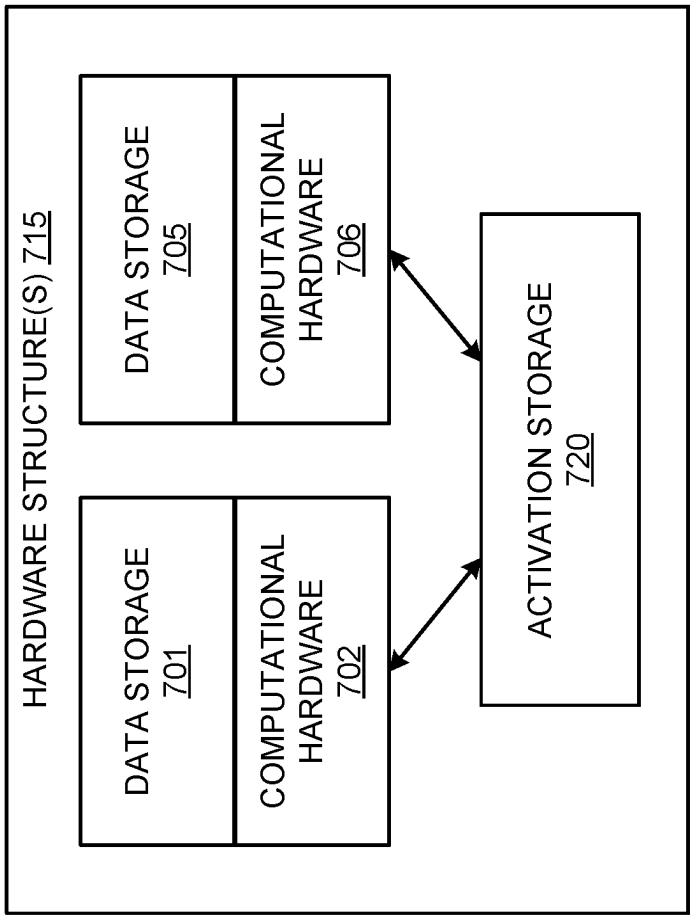

HARDWARE STRUCTURE(S) 715
DATA STORAGE 701
COMPUTATIONAL HARDWARE 702
DATA STORAGE 705
COMPUTATIONAL HARDWARE 706
ACTIVATION STORAGE 720

FIG. 7B

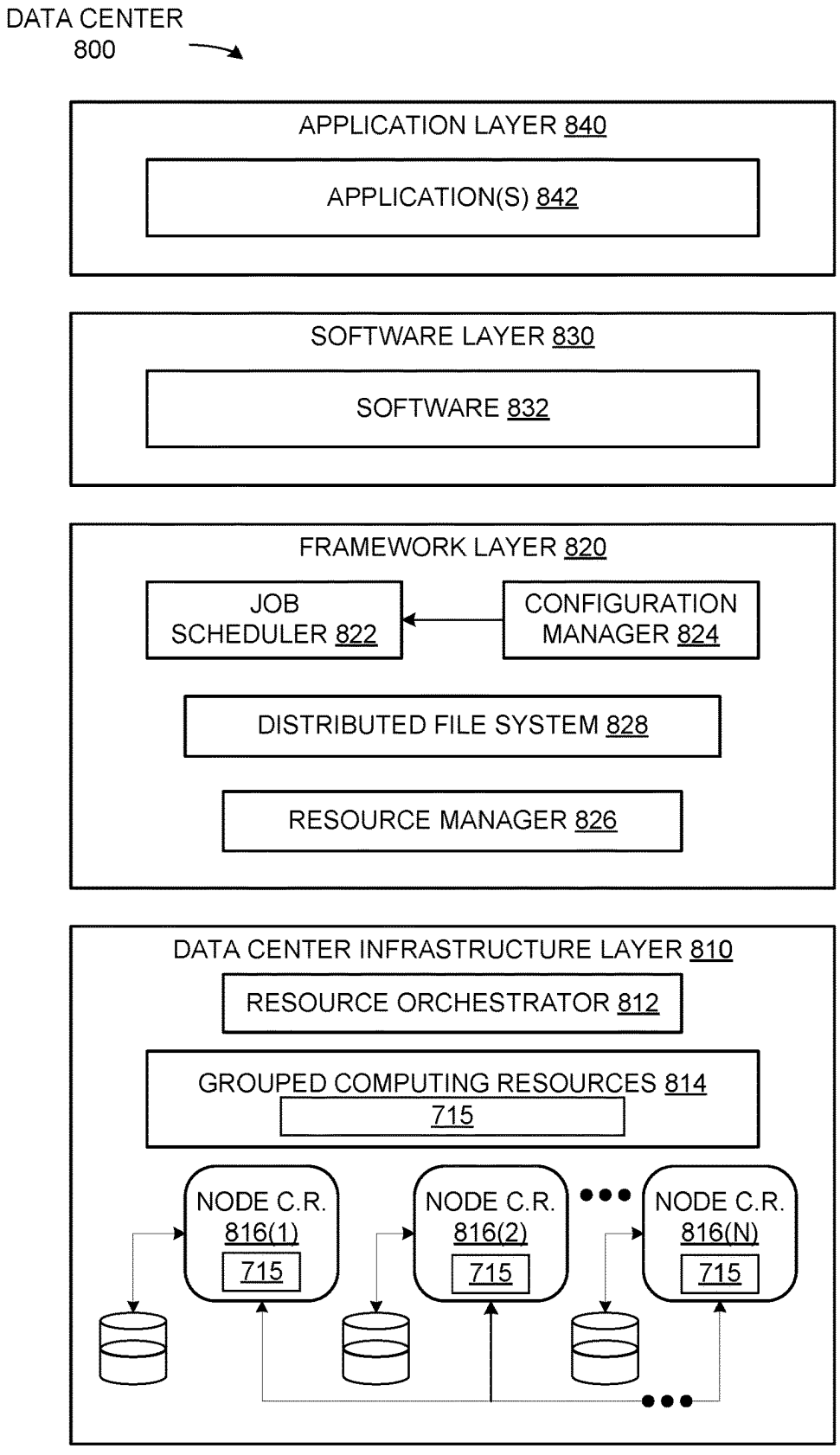

DATA CENTER
800
APPLICATION LAYER 840
APPLICATION(S) 842
SOFTWARE LAYER 830
SOFTWARE 832
FRAMEWORK LAYER 820
JOB SCHEDULER 822
CONFIGURATION MANAGER 824
DISTRIBUTED FILE SYSTEM 828
RESOURCE MANAGER 826
DATA CENTER INFRASTRUCTURE LAYER 810
RESOURCE ORCHESTRATOR 812
GROUPED COMPUTING RESOURCES 814
715
NODE C.R. 816(1)
715
NODE C.R. 816(2)
715
NODE C.R. 816(N)
715

FIG. 8

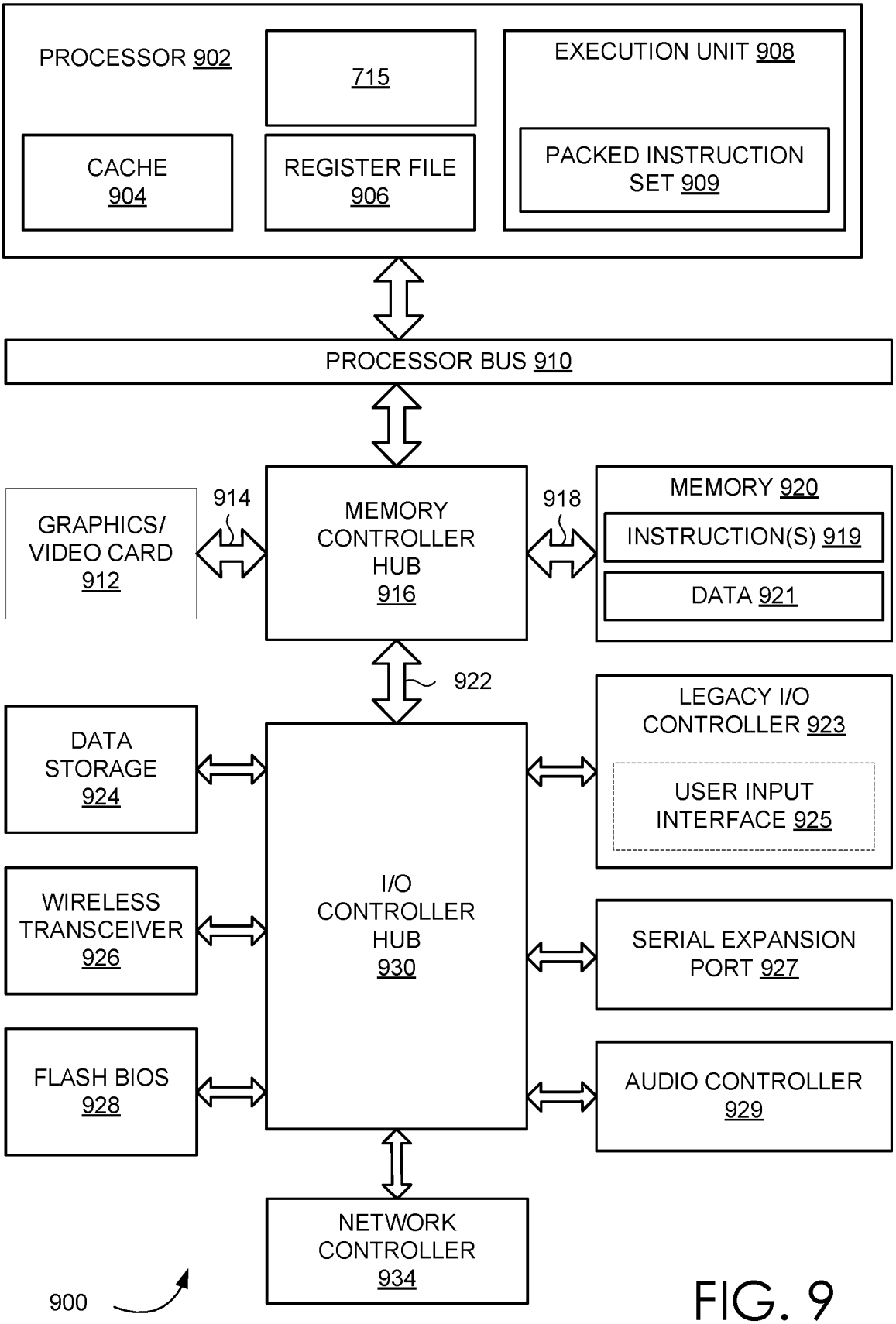

PROCESSOR 902
715
EXECUTION UNIT 908
CACHE 904
REGISTER FILE 906
PACKED INSTRUCTION SET 909
PROCESSOR BUS 910
GRAPHICS/ VIDEO CARD 912
914
MEMORY CONTROLLER HUB 916
918
MEMORY 920
INSTRUCTION(S) 919
DATA 921
922
DATA STORAGE 924
LEGACY I/O CONTROLLER 923
USER INPUT INTERFACE 925
WIRELESS TRANSCEIVER 926
I/O CONTROLLER HUB 930
SERIAL EXPANSION PORT 927
FLASH BIOS 928
AUDIO CONTROLLER 929
NETWORK CONTROLLER 934
900

FIG. 9

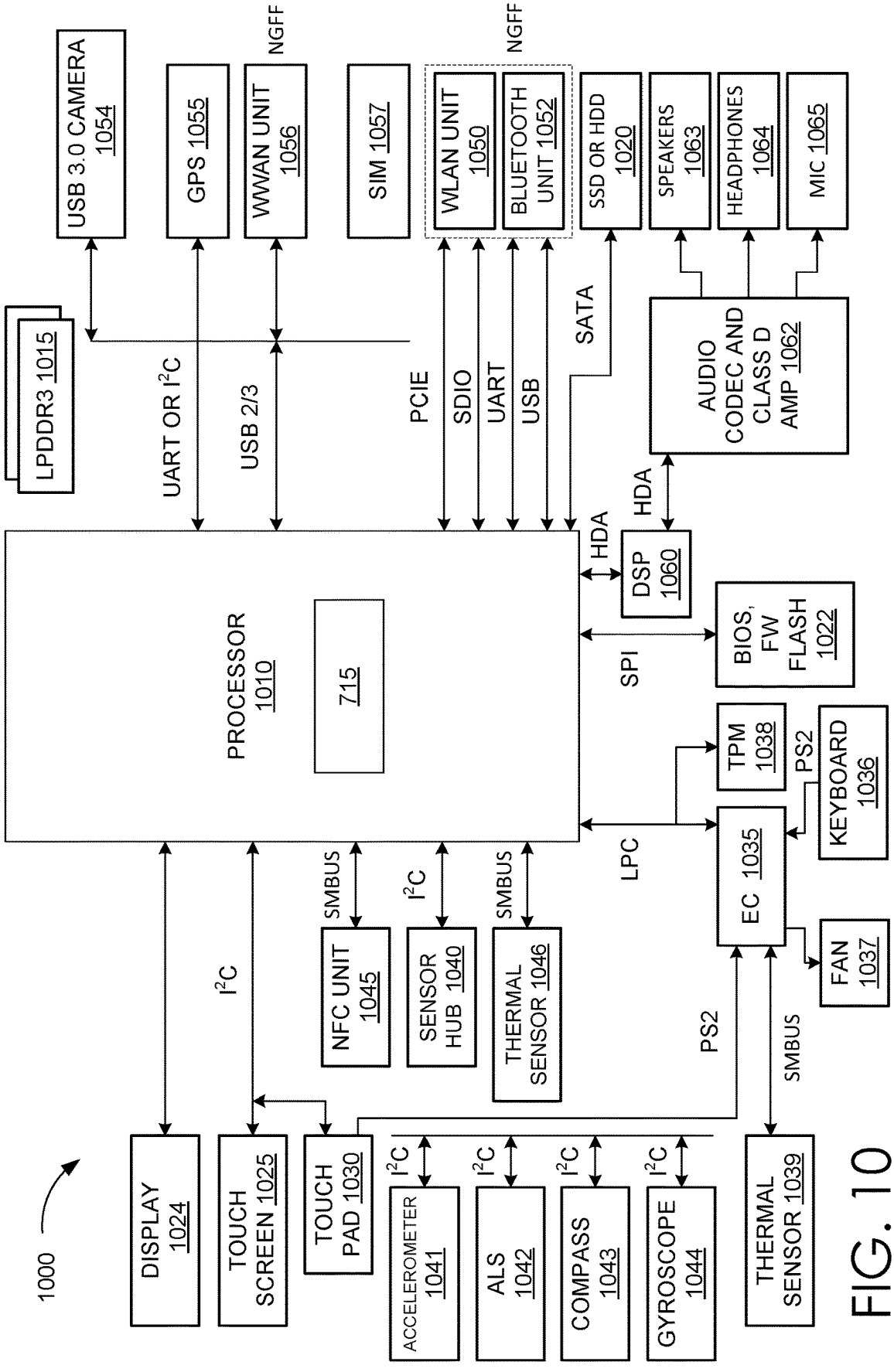

1000
DISPLAY 1024
TOUCH SCREEN 1025
TOUCH PAD 1030
ACCELEROMETER 1041
ALS 1042
COMPASS 1043
GYROSCOPE 1044
THERMAL SENSOR 1039
I2C
NFC UNIT 1045
SENSOR HUB 1040
THERMAL SENSOR 1046
SMBUS
I2C
SMBUS
PROCESSOR 1010
715
LPC
EC 1035
PS2
SMBUS
FAN 1037
TPM 1038
PS2
KEYBOARD 1036
SPI
BIOS, FW FLASH 1022
HDA
DSP 1060
HDA
AUDIO CODEC AND CLASS D AMP 1062
SATA
PCIE
SDIO
UART
USB
UART OR I2C
USB 2/3
LPDDR3 1015
USB 3.0 CAMERA 1054
GPS 1055
NGFF
WWAN UNIT 1056
SIM 1057
NGFF
WLAN UNIT 1050
BLUETOOTH UNIT 1052
SSD OR HDD 1020
SPEAKERS 1063
HEADPHONES 1064
MIC 1065

FIG. 10

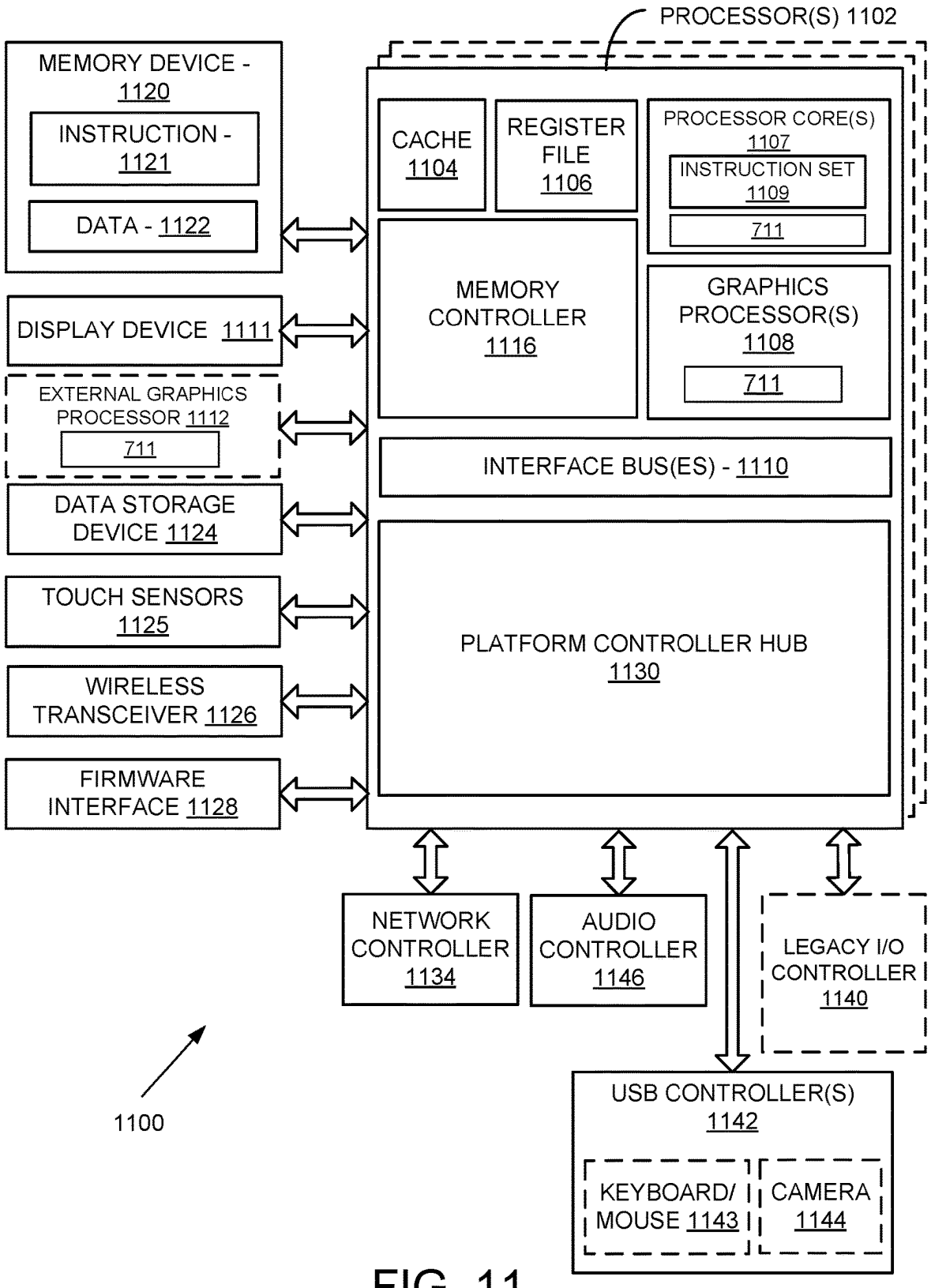

PROCESSOR(S) 1102
MEMORY DEVICE - 1120
INSTRUCTION - 1121
DATA - 1122
CACHE 1104
REGISTER FILE 1106
PROCESSOR CORE(S) 1107
INSTRUCTION SET 1109
711
MEMORY CONTROLLER 1116
GRAPHICS PROCESSOR(S) 1108
711
DISPLAY DEVICE 1111
EXTERNAL GRAPHICS PROCESSOR 1112
711
INTERFACE BUS(ES) - 1110
DATA STORAGE DEVICE 1124
TOUCH SENSORS 1125
PLATFORM CONTROLLER HUB 1130
WIRELESS TRANSCEIVER 1126
FIRMWARE INTERFACE 1128
NETWORK CONTROLLER 1134
AUDIO CONTROLLER 1146
LEGACY I/O CONTROLLER 1140
USB CONTROLLER(S) 1142
KEYBOARD/ MOUSE 1143
CAMERA 1144
1100

FIG. 11

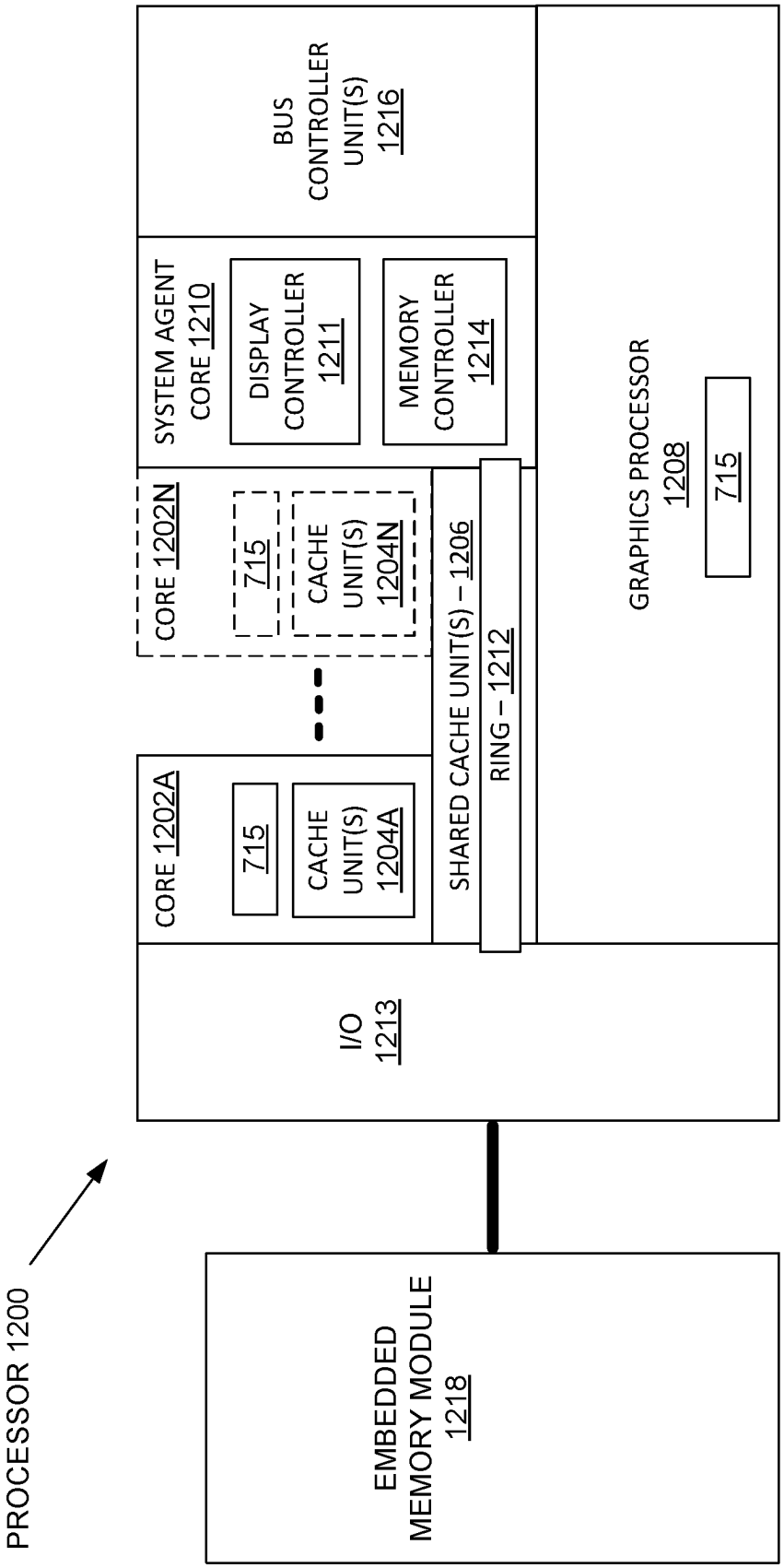

PROCESSOR 1200
EMBEDDED MEMORY MODULE 1218
I/O 1213
CORE 1202A
715
CACHE UNIT(S) 1204A
CORE 1202N
715
CACHE UNIT(S) 1204N
SHARED CACHE UNIT(S) – 1206
RING – 1212
SYSTEM AGENT CORE 1210
DISPLAY CONTROLLER 1211
MEMORY CONTROLLER 1214
BUS CONTROLLER UNIT(S) 1216
GRAPHICS PROCESSOR 1208
715

FIG. 12

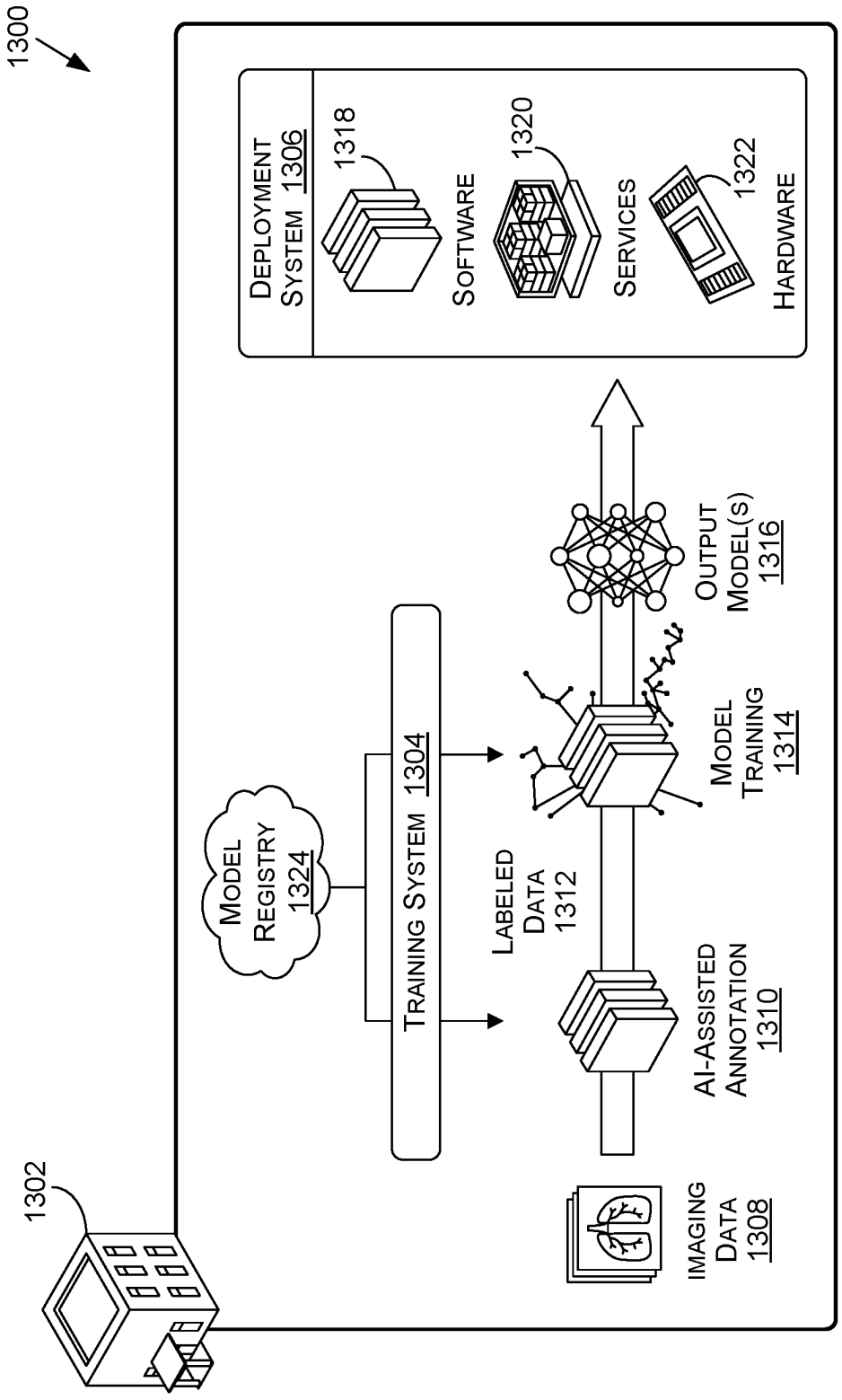

1300
1302
DEPLOYMENT SYSTEM 1306
1318
SOFTWARE
1320
SERVICES
1322
HARDWARE
IMAGING DATA 1308
AI-ASSISTED ANNOTATION 1310
LABELED DATA 1312
MODEL TRAINING 1314
OUTPUT MODEL(S) 1316
TRAINING SYSTEM 1304
MODEL REGISTRY 1324

FIG. 13

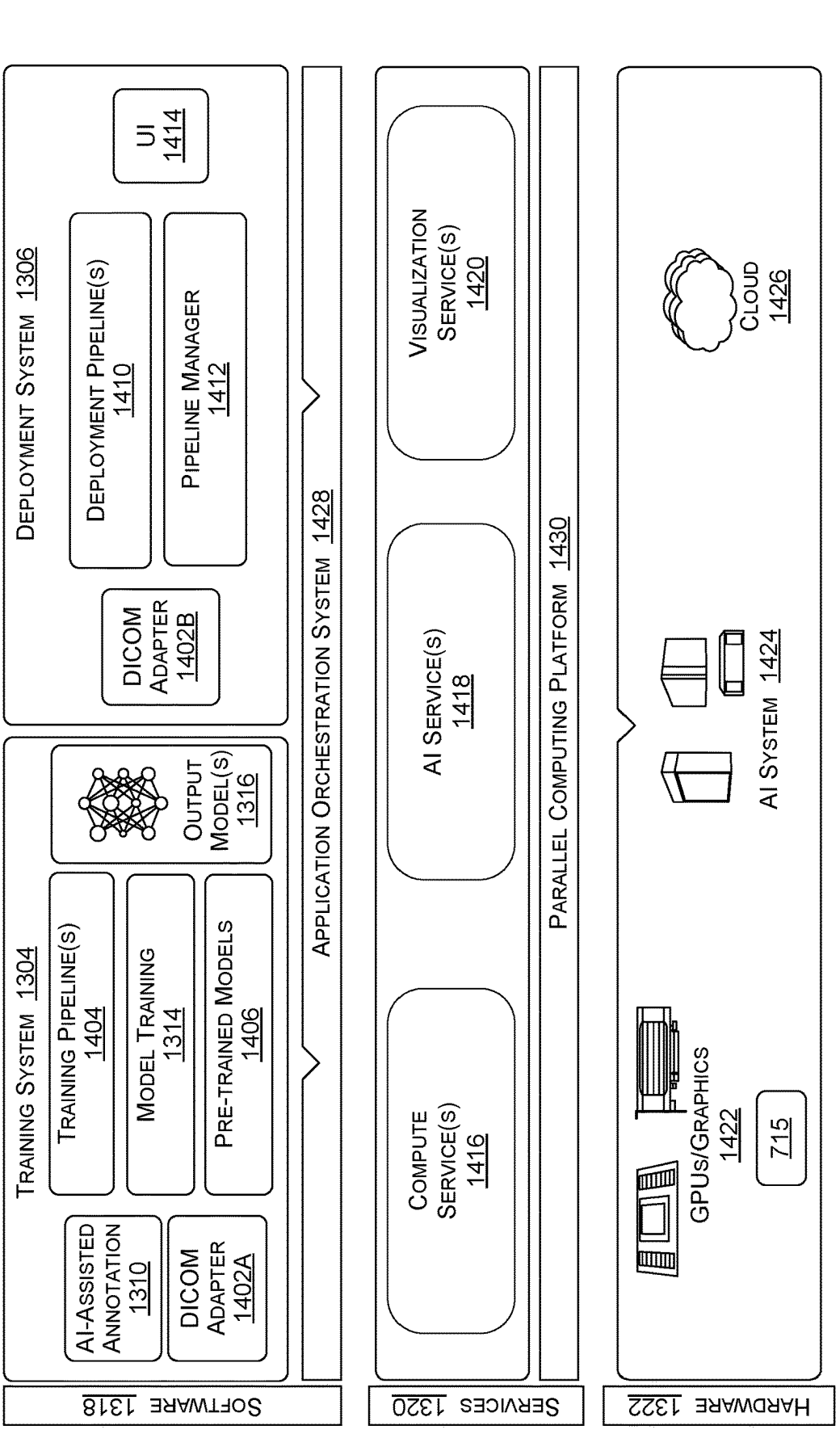

1400
SOFTWARE 1318
SERVICES 1320
HARDWARE 1322
DEPLOYMENT SYSTEM 1306
UI 1414
DEPLOYMENT PIPELINE(S) 1410
PIPELINE MANAGER 1412
DICOM ADAPTER 1402B
TRAINING SYSTEM 1304
AI-ASSISTED ANNOTATION 1310
DICOM ADAPTER 1402A
TRAINING PIPELINE(S) 1404
MODEL TRAINING 1314
PRE-TRAINED MODELS 1406
OUTPUT MODEL(S) 1316
APPLICATION ORCHESTRATION SYSTEM 1428
COMPUTE SERVICE(S) 1416
AI SERVICE(S) 1418
VISUALIZATION SERVICE(S) 1420
PARALLEL COMPUTING PLATFORM 1430
GPUS/GRAPHICS 1422
715
AI SYSTEM 1424
CLOUD 1426

FIG. 14

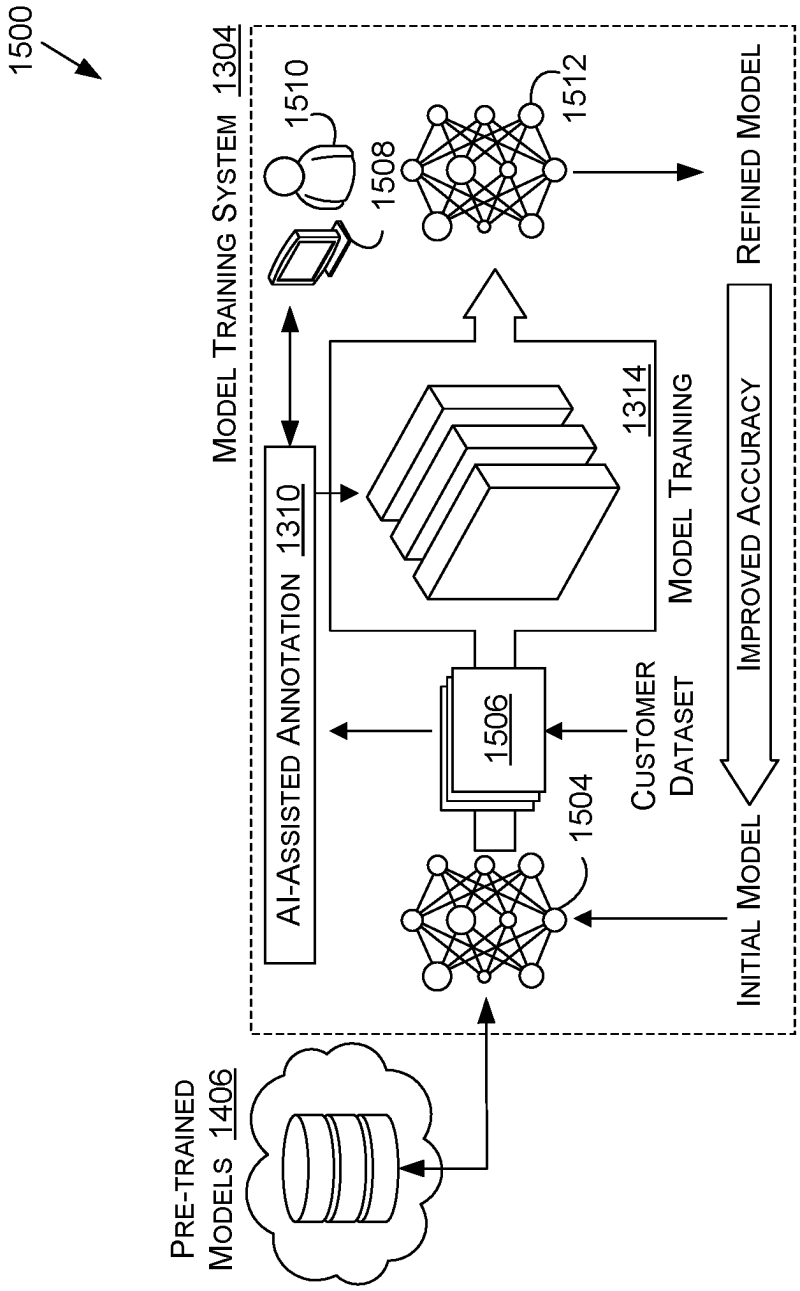

1500
Model Training System 1304
1510
1508
1512
Refined Model
AI-Assisted Annotation 1310
1314
Model Training
1506
Customer Dataset
Improved Accuracy
1504
Initial Model
Pre-trained Models 1406

FIG. 15A

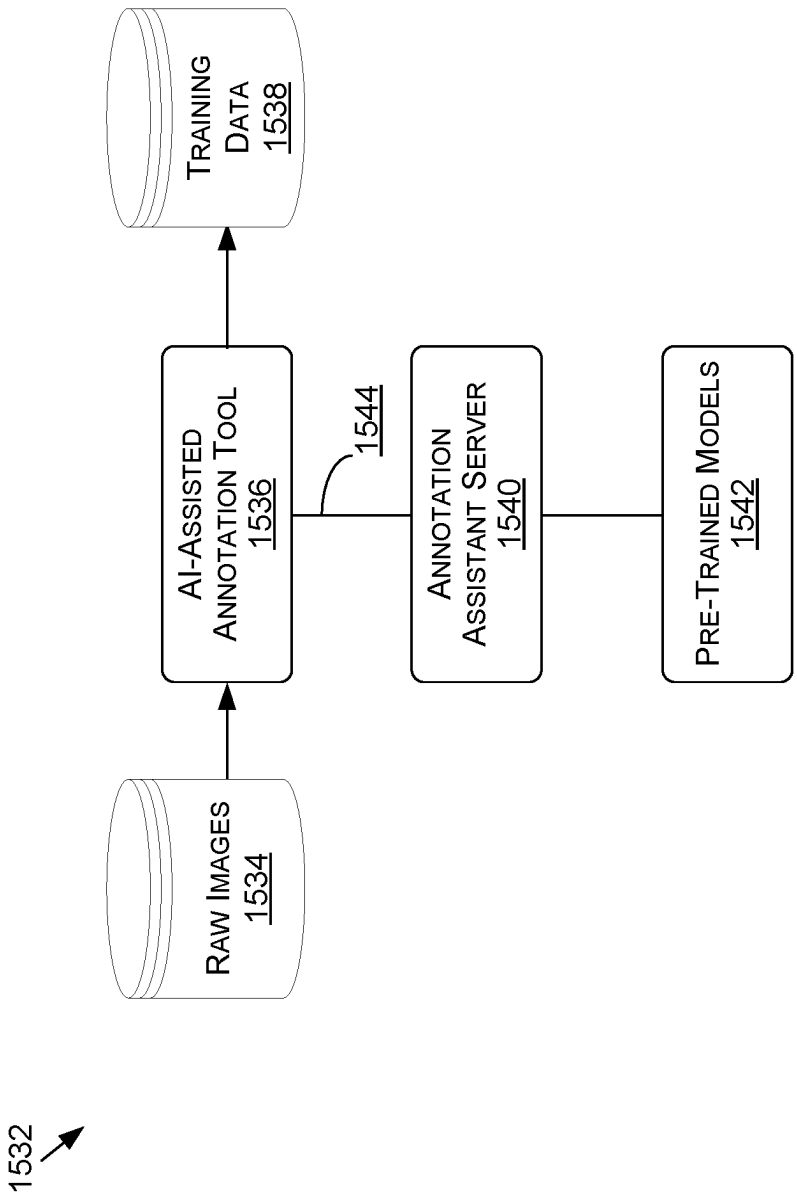

1532
RAW IMAGES
1534
AI-ASSISTED ANNOTATION TOOL
1536
TRAINING DATA
1538
1544
ANNOTATION ASSISTANT SERVER
1540
PRE-TRAINED MODELS
1542

FIG. 15B

IN-PLACE DATA MANAGEMENT WITHIN MEMORY BUFFERS

BACKGROUND

[0001] Graphics processing units (GPUs) may store information within a memory buffer including a variety of sub-buffers with arbitrary sizes. For some operations, such as rendering, it may be useful to reuse portions of the memory buffer in subsequent rendering processes. For example, a scene may have components that remain static (visually consistent) over a number of frames, like backgrounds or non-moving characters, and computing resources can be conserved by reusing the components rather than fully rebuilding a scene. Copy operations may map the information within the memory buffer from a source location to a destination location. However, when rendering or adding information to the destination location, certain sub-buffers may be occupied, causing overlap between the source and destination locations, leading to read-after-write errors.

BRIEF DESCRIPTION OF THE DRAWINGS

[0002] Various embodiments in accordance with the present disclosure will be described with reference to the drawings, in which:

[0003] FIG. **1**A illustrates an example schematic representation of a task being offloaded from a central processing unit (CPU) to a graphics processing unit (GPU), in accordance with various embodiments;

[0004] FIG. **1**B illustrates an example schematic representation of a memory buffer moving data segments from a source location to a destination location, in accordance with various embodiments;

[0005] FIG. **2**A illustrates an example schematic representation of a memory buffer moving data segments from a source location to a destination location, in accordance with various embodiments;

[0006] FIG. **2**B illustrates an example schematic representation of workers selecting a data segment in a memory buffer to move the data segments from a source location to a destination location, in accordance with various embodiments;

[0007] FIG. **3**A illustrates an example representation of a data movement task for a memory buffer using one or more workers, in accordance with various embodiments;

[0008] FIG. **3**B illustrates an example representation of a data movement task for a memory buffer using one or more workers, in accordance with various embodiments;

[0009] FIG. **3**C illustrates an example representation of a memory buffer reusing data segments from a first state in a second state, in accordance with various embodiments;

[0010] FIG. **3**D illustrates an example representation of a data movement task for a memory buffer to group reused data segments from a first state in a second state, in accordance with various embodiments;

[0011] FIG. **4**A illustrates an example process for moving chunks of data between memory segments, in accordance with various embodiments;

[0012] FIG. **4**B illustrates an example process for moving chunks of data between memory segments, in accordance with various embodiments;

[0013] FIG. **5** illustrates an example process for moving chunks of data between memory segments, in accordance with various embodiments;

[0014] FIG. **6** illustrates components of a distributed system that can be utilized to update or perform inferencing using a machine learning model, according to at least one embodiment;

[0015] FIG. **7**A illustrates inference and/or training logic, according to at least one embodiment;

[0016] FIG. **7**B illustrates inference and/or training logic, according to at least one embodiment;

[0017] FIG. **8** illustrates an example data center system, according to at least one embodiment;

[0018] FIG. **9** illustrates a computer system, according to at least one embodiment;

[0019] FIG. **10** illustrates a computer system, according to at least one embodiment;

[0020] FIG. **11** illustrates at least portions of a graphics processor, according to one or more embodiments;

[0021] FIG. **12** illustrates at least portions of a graphics processor, according to one or more embodiments;

[0022] FIG. **13** is an example data flow diagram for an advanced computing pipeline, in accordance with at least one embodiment;

[0023] FIG. **14** is a system diagram for an example system for training, adapting, instantiating and deploying machine learning models in an advanced computing pipeline, in accordance with at least one embodiment; and

[0024] FIGS. **15**A and **15**B illustrate a data flow diagram for a process to train a machine learning model, as well as client-server architecture to enhance annotation tools with pre-trained annotation models, in accordance with at least one embodiment.

DETAILED DESCRIPTION

[0025] In the following description, various embodiments will be described. For purposes of explanation, specific configurations and details are set forth in order to provide a thorough understanding of the embodiments. However, it will also be apparent to one skilled in the art that the embodiments may be practiced without the specific details. Furthermore, well-known features may be omitted or simplified in order not to obscure the embodiment being described.

[0026] The systems and methods described herein may be used by, without limitation, non-autonomous vehicles or machines, semi-autonomous vehicles or machines (e.g., in an in-cabin infotainment or digital or driver virtual assistant application)), autonomous vehicles or machines, piloted and un-piloted robots or robotic platforms, warehouse vehicles, off-road vehicles, vehicles coupled to one or more trailers, flying vessels, boats, shuttles, emergency response vehicles, motorcycles, electric or motorized bicycles, aircraft, construction vehicles, trains, underwater craft, remotely operated vehicles such as drones, and/or other vehicle types. Further, the systems and methods described herein may be used for a variety of purposes, by way of example and without limitation, for machine control, machine locomotion, machine driving, synthetic data generation, model training or updating, perception, augmented reality, virtual reality, mixed reality, robotics, security and surveillance, simulation and digital twinning, autonomous or semi-autonomous machine applications, deep learning, environment simulation, object or actor simulation and/or digital twin-

ning, data center processing, conversational artificial intelligence (AI), generative AI with large language models (LLMs), light transport simulation (e.g., ray-tracing, path tracing, etc.), collaborative content creation for 3D assets, cloud computing and/or any other suitable applications.

[0027] Disclosed embodiments may be comprised in a variety of different systems such as automotive systems (e.g., a control system for an autonomous or semi-autonomous machine, a perception system for an autonomous or semi-autonomous machine), systems implemented using a robot, aerial systems, medial systems, boating systems, smart area monitoring systems, systems for performing deep learning operations, systems for performing simulation operations, systems for performing digital twin operations, systems implemented using an edge device, systems incorporating one or more virtual machines (VMs), systems for performing synthetic data generation operations, systems implemented at least partially in a data center, systems for performing conversational AI operations, systems for performing generative AI operations using LLMs, systems for performing light transport simulation, systems for performing collaborative content creation for 3D assets, systems implemented at least partially using cloud computing resources, and/or other types of systems.

[0028] Approaches in accordance with various embodiments are directed toward parallel movement of data segments within a memory buffer. Systems and methods may implement workers to read a data segment into individual registries and then write the data segments to a different location within the memory buffer without the use of a second memory buffer. As a result, embodiments allow for a single read/write operation to move different data segments. The movement of the data segments may be part of a rendering operation to compact and/or move the data segments and to provide contiguous segments for new information, such as new data structures like bounding volume hierarchies (BVH). In at least one embodiment, a set of workers may race to read a first data segment. The winner of the race copies the information to its own registry, sets a bit to indicate to other workers the data has been stored within the registry, reads the destination data segment, determines whether the destination has been read (e.g., evaluates the bit), and, if the segment is available, writes the first data segment to the destination. If the segment has not yet been read, then the worker will pick up that segment, store the data within its registry, set the bit, and then proceed to write the first data segment. Each process of reading, storing, setting the bits, deleting/overwriting, etc., may be continued as different workers race to read different data segments. The process may provide for a substantially parallel process to compact or otherwise move data segments without restrictions on the destination addresses and/or an additional memory buffer to duplicate the data. Various embodiments may also describe one or more portions of the systems and methods herein as performing multiple passes separated by system-wide synchronizations.

[0029] Various embodiments may be used with memory processes related to graphics processing units (GPUs). GPUs may include a memory buffer with a number of sub-buffers of arbitrary size. In at least one embodiment, the sub-buffers may be configured to be of cache line or coarser granularity. Systems and methods discussed herein are used to rearrange different memory objects to arbitrary new offsets. The process may be described as being in-place, i.e., the destination location of one sub-buffer is allowed to overlap the source location of another. Additionally, the process may be executed in parallel with multiple threads. I one or more embodiments, the process includes dividing the memory buffer into fixed-size chunks (e.g., cache line sized or larger). Thereafter, an array of zero-initialized Boolean flags is allocated. Each chunk may correspond to a flag. In operation, a set bit marks a status of the corresponding chunk, for example indicating whether the chunk has been read into GPU registers, and based on the status, a worker may be informed whether the chunk may be safety overwritten.

[0030] Systems and methods may deploy a set of workers to read a source memory location into a registry location, establish a status for a corresponding chunk of the source memory location, and then write the information to a destination memory location. In at least one embodiment, the set of workers may "race" to different memory locations and set the status with an atomic-or operation. When the worker reaches the location, the result of the atomic-or may determine whether or not the worker "won" the race to the chunk (e.g., a "0" output may indicate a win, while a "1" output may indicate a loss). The winning worker is designated responsibility for writing the information within the chunk of memory to the destination location. Similarly, the process may also be repeated at the destination location. For example, the worker may read the destination chunk, set the corresponding status with an atomic-or, and overwrite the destination chunk.

[0031] Various other such functions can be used as well within the scope of the various embodiments as would be apparent to one of ordinary skill in the art in light of the teachings and suggestions contained herein.

[0032] FIG. **1**A illustrates an example environment **100** that may be used with embodiments of the present disclosure. In this example, a central processing unit (CPU) **102** may send instructions and/or offload various operations to a GPU **104**. For example, the CPU may offload operations that benefit from parallelization, such as graphics rendering, machine learning tasks, and/or the like. In this example, the CPU **102** may be used to execute one or more operations that receive data from a database **106**, which in an example related to graphics processing may include different data objects, such as BVH objects as one non-limiting example. Responsive to an input **108** from a user, the CPU **102** may provide the data to the GPU **104**, or instruct the GPU **104** to retrieve the data, in order to accomplish a task associated with the input **108**. In one non-limiting example related to graphics processing and rendering, the input **108** may correspond to a user moving a digital avatar within a rendered scene, and the instructions provided to the GPU **104** may be associated with rendering new content within one or more subsequent frames responsive to the input **108**.

[0033] Continuing with the example of rendering images, such as for graphics applications, the process of rendering new objects within a series of frames may be computationally inexpensive when there is static or substantially static geometry or where motion is related to rigid body movement. Additionally, reskinning an animation may also be computationally inexpensive. For these types of application, it may be easy and consume comparatively less processing resources to perform a full scene rebuild. However, for more complicated scenes or more detailed geometry, it is undesirable to fully build a scene because of the increased and

potentially prohibitively expensive computational cost. The scenes may include one or more features or objects that can be reused between frames. By reusing portions of the scene and only building a subset of the total scene, computational costs may be reduced while also improving output performance for users. Systems and methods of the present disclosure may facilitate partial scene rebuilds with improved memory management. For example, a standard command to copy a location within a memory buffer may, by default, directly write chunks of data to a "same" location. That is, a position or location within of a source may be copied directly to the corresponding position or location at the destination. However, this direct copy may lead to errors, as shown in FIG. **1**B, which illustrates a schematic diagram **150** of a partial rebuild of a scene. In this location, a first frame **152** may include one or more objects stored within chunks **154** of a memory buffer **156**. The objects in this example are represented by BVH**0**-BVH**4**, but it should be appreciated that various embodiments may use different data objects, more data objects, fewer data objects, or combinations thereof.

[0034] As the scene progresses, a second frame **158** may be rendered. The second frame **158** may share one or more of the objects within the memory buffer **156**, and as a result, it may be advantageous to reuse or duplicate the objects. In the illustrated example, BVH**0**, BVH**1**, and BVH**3** are all reused in the second frame **158**. Because the second frame **158** may have additional objects that may be stored in the remaining chunks **154**, it may be advantageous to group or move the reused objects toward different arbitrary locations, thereby freeing up bigger "blocks" of space for the new objects. For example, BVH**0** may be positioned within a first chunk **154**A, BVH**1** may be positioned within a second chunk **154**B, and BVH**3** may be positioned within a fourth chunk **154**D, leaving both a third chunk **154**C and a fifth chunk **154**D "empty" or "open." Traditional methods may have problems with movement of chunks to arbitrary locations or may try to copy information to its current location, but at a source destination. For example, a source location **160** may include the different objects within the respective chunks **154**A-**154**D and a destination location **162** may include corresponding destination chunks **164**A-**164**D. As shown by the arrows, BVH**0** is moved from the chunk **154**A to the chunk **164**C and BVH**1** is moved from the chunk **154**B to the chunk **164**D. However, moving BVH**3** to its "same" location in the destination location **164** also directs the chunk **154**D to the chunk **164**D, thereby leading to a read-after-write error. Systems and methods of the present disclosure address this problem, among others, to cause partial or indirect builds of different scenes between different locations. For example, embodiments may deploy an application program interface (API) that receives a call for a batch of builds, with the inputs defined on the GPU side memory buffers, thereby decreasing overhead for the CPU. Various embodiments may use a set of workers to identify a state of a chunk, set a status identifier, and then take responsibility for copying information to a register and writing the information to a destination location. In this manner, chunks may be moved to arbitrary locations to reduce a likelihood of read-after-write errors, among various other benefits. Moreover, within certain computational models, there may also be a low chance of read-after-write errors.

[0035] FIG. **2**A illustrates an example data movement task **200** that may be used with embodiments of the present disclosure. In this example, the data movement task **200** illustrates movements of chunks **202** of data from a source location **204** to a destination location **206**, similar to the configuration shown in FIG. **1**B. The illustrated example shows movement of the chunks **202** over one slot to the right, but it should be appreciated that systems and methods of the present disclosure may be used to perform arbitrary movement of chunks and all chunk movement may not be the same. For example, a first chunk may move one slot while a second chunk moves three slots.

[0036] The chunks **202** are represented by different letters in this example and are shown to be the same sizes, but the chunks **202** may be any size, with some chunks **202** being larger than others. For the movement task, the chunk "A" is moved from the source location **204**A to the destination location **206**B, the chunk "B" is moved from the source location **204**B to the destination location **206**B, and so forth. When each chunk is moved the same amount and there are no gaps or spaces, the problems shown in FIG. **1**B may be avoided. However, as shown in FIG. **1**B, gaps, irregular movement, and the like may cause read-after-write errors or other problems.

[0037] FIG. **2**B illustrates an example data movement task **250** that may be used with embodiments of the present disclosure. In this example, a set of workers **252** may "race" to different chunks **202** of data and then, upon winning their race, copy the data representative of the chunks **202** into a registry, set a status identifier for the data location/chunk, and then proceed to identify and write the copied chunk **202** to a destination location. As shown, a first worker **252**A wins the race to the chunk A at the source location **204**A, reads the information into a registry, and then proceeds to write the information to the destination location **206**B. As described herein, the writing to the destination location **206**B may only be performed if it determined that a status identifier for the destination location **206**B corresponds to an indication that the destination location **206**B may be written to, such as an indication that data within the destination location **206**B has been read and stored within a registry for writing to another location. The second worker **252**B, having lost the race to the first worker **252**A, may then win the race to the chunk B at the source location **204**B. Subsequent races to different chunks may then be won by the first worker **252**A or any number of additional workers **252**N used to execute the data movement task.

[0038] Various embodiments of the present disclosure may be deployed to execute one or more movement tasks, which may include identifying reused data between different frames, determining destination locations for the reused data, copying the reused data, and then writing the reused data to the determined destination locations. In at least one embodiment, the memory buffer is divided into fixed-size chunks (e.g., cache line sized or larger). An array may be allocated with zero-initialized Boolean flags, with a given flag corresponding to a given chunk of data, and therefore, to a given memory location. In at least one embodiment, a set bit marks that the corresponding chunk has been read into GPU registers by a worker thread-group and may be safely overwritten. In embodiments there is a mapping from a source chunk address to its destination, which may be referred to as calcDestIdx(sourceIdx) in the pseudocode presented herein. For example, given a list of sub-buffer

descriptions with a source and destination address and a byte size, systems and methods can pre-sort the list by the source address and then perform a binary search. For example and without limitation, the algorithm may be implemented as:

```
[0039] _global_ void defragKernel(BitArray pickedup, Chunk* buffer)
[0040] {
  [0041] int sourceIdx=workerIdx( );
  [0042] // Returns dest idx for a source chunk, or −1 if the source is empty
  [0043] int destIdx=calcDestIdx (sourceIdx);
  [0044] if (destIdx==sourceIdx||destIdx==−1) return;
  [0045] Chunk chunk=buffer [sourceIdx];
  [0046] int result=atomicOr1Bit (pickedup, sourceIdx);
  [0047] while (!result)
  [0048] {
    [0049] //Successfully picked-up chunk which is relocate
    [0050] sourceIdx=destIdx;
    [0051] Chunk nextChunk=buffer[sourceIdx];
    [0052] result=atomicOr1Bit (pickedup, sourceIdx);
    [0053] buffer[destIdx]=chunk;
    [0054] destIdx=calcDestIdx (sourceIdx);
    [0055] if (destIdx==sourceIdx||destIdx==−1) return;
    [0056] chunk=nextChunk;
  [0057] }
[0058] }
```

[0059] As shown, each worker is assigned a chunk in the memory buffer; the assigned worker then proceeds to read that chunk, and set the corresponding flag with an atomic-or. If the result of the atomic is 0, the worker was the first to read that chunk, and is responsible for writing the chunk out to its destination. The process may also be repeated before writing out the destination chunk. For example, the worker reads the destination chunk, sets the corresponding flag with an atomic-or, overwrites the destination chunk, and now is responsible for relocating the chunk. Embodiments may also be used with sparse memory applications, such as executing the process for cachelines within a known small region where the input and output overlap or for taking a prefix sum and then launching workers for cachelines that are occupied.

[0060] FIG. **3**A illustrates an example data movement task **300** that may be used with embodiments of the present disclosure. In this example, workers **302**A-**302**D are initialized to execute the task. While four workers **302**A-**302**D are illustrated, there may be more or fewer workers. As will be appreciated, more workers (e.g., more workers than chunks of data) may waste resources because at least some of the workers will be idle. However, too few workers may increase a time to complete the task. In at least one embodiment, the number of workers may be specified, such as by using a control that may be adjusted by a user, or may be determined by the GPU. Additionally, a threshold number of workers may be set (e.g., an upper threshold, a lower threshold, etc.) and then the GPU may determine or select a number of workers based on the threshold. Additionally, a task type may be used to specify a number of workers. For example, certain tasks (e.g., critical tasks or high importance tasks) may select a number of workers equal to a number of data segment. However, other tasks may use a multiplier of the number of data segments, such as one half or one third, among other reasonable options.

[0061] A memory buffer **304** is illustrated at different points in the movement task. As shown, different chunks **306** of data (denotated by the letters A-D) within the memory buffer **304** may move responsive to the movement task. Additionally, different chunks **306** are shaded based on their status. For example, shaded chunks **306** may correspond to chunks **306** that have not yet been read and/or have not yet been written. Certain examples show the memory buffer **304** "stacked" with another buffer to illustrate movement between a source location and a destination location, as discussed herein.

[0062] In this example, the first worker **302**A may win a race to read the chunk **306** corresponding to A (also referred to as A for simplicity), with the corresponding memory buffer **304** illustrating the chunk **306** corresponding to A as being unshaded. Each of the workers **302**A-**302**D may have "raced" to complete the task of reading A, however, because the first worker **302**A has won, the first worker **302**A assumes responsibilities with reading and writing the chunk **306** associated with A.

[0063] The first worker **302**A in this example, after reading A, may then set the atomic-or for A, which in this case corresponds to position **0** (e.g., set **0** indicating to set the bit with the 0 position) within the memory buffer **304**. The value or status indicator is initially set to "0" to indicate that the chunk **306** associated with A has not been read. After reading the data, the bit may be changed. Accordingly, if the second worker **302**B were to attempt to read A, the second worker **302**B would evaluate the status and determine that A has already been read and then move forward to race to a different chunk.

[0064] In this example, the next chunk **306** that the workers **302**A-**302**D race to is the chunk **306** that corresponds to B, which in this example is shown in position **1** (e.g., set **1**). Again, the first worker **302**A has won this race and proceeds to read B and then determines that B has not been read, due to the "0" value of the bit. The first worker **302**A may then store the information associated with B into the registry, and then sets the value of the bit to "1" to indicate the status as being read. Accordingly, as shown in the example by the dashed line, when the second worker **302**B tries to read B, the second worker **302**B will see that position **1** has a value of "1" to indicate that another worker has picked up the chunk **306**. In the corresponding memory buffer **304**, the chunk **306** associated with B is shown unshaded because B was initially not read by any workers. In subsequent operations, as shown in FIG. **3**A, B is shown as shaded to indicate reading/writing of the corresponding chunk **306**.

[0065] The illustrated example of FIG. **3**A further illustrates a process where the first worker **302**A writes A to the destination at position **1**. In this example the chunk **306** corresponding to A is now shaded in the source and destination locations for the memory buffer **304**. As noted herein, because it was determined that the chunk **306** corresponding to B had been read, the destination location associated with B (e.g., position **1**) can be overwritten for the data associated with the chunk **306** corresponding to A. The chunk **306** corresponding to B remains in the registry for the first worker **302**A, where it was read in a previous operation. The process may continue for the first worker **302**A as long as the first worker **302**A continues to "win" the races to each of the chunks **306**. For example, the first worker **302**A may pro-

ceed to read the chunk **306** corresponding to C, identify the value of position **2** as "0" to indicate the chunk has not yet been read, set the value of position **2** to "1" to indicate that the data chunk has been read, write the chunk **306** corresponding to B to its new location at position **2**, and then proceed to read D. The process may continue to repeat itself until the first worker **302**A loses the race to a given chunk **306** and/or until there are no remaining chunks **306**. As shown in this example, the remaining workers **302**B-**302**D each lose their respective races and then check the values for the given positions **1**, **2**, and **3**. The values for these positions is shown as being "1" to indicate that another worker has picked up the chunk. Accordingly, the first worker **302**A in this example reads and writes each of the chunks **306** to move the chunks A, B, C, and D from positions **0**, **1**, **2**, **3** to positions **1**, **2**, **3**, **4**, respectively.

[0066] Various embodiments of the present disclosure may loop/recurse and do an arbitrary number of operations, not just up to a fifth segment as shown in FIG. **3**A. Accordingly, the process may be continued for a given number of data segments, for example, continuing through the process until an empty destination segment is encountered or until a race is lost for an occupied destination segment, which may cause a worker to terminate.

[0067] The embodiment illustrated in FIG. **3**A discussed herein depicts the perspective of a four-way race for each chunk between the workers **302**A-**302**D. However, FIG. **3**A may also be described with respect to an implementation in which each worker **302**A-**302**D is initialized to target a different work item. For example, the first worker **302**A may target the chunk **306** represented by A to try and read and locate the chunk A, the second worker **302**B may target the chunk **306** represented by B to try and read and locate the chunk B, and so forth. As a result, the example may include an atomic race between two workers, as represented by the arrows associated with the "set" boxes (e.g., set **1**(**1**), set **2**(**1**), etc.). The implementation may then be described as an artifact of the example source-destination mapping. That is, there would not be a "race" between workers at all if there were not source-destinations laps.

[0068] FIG. **3**B illustrates an example data movement task **320** that may be used with embodiments of the present disclosure. In this example, the first worker **302**A does not win each of the races, and as a result, the other workers **302**B-**302**D are involved in the process of reading different chunks, setting values for identifiers, and then writing the data to a destination location. The first worker **302**A of this example reads the chunk **306** corresponding to A and determines the value for the position **0** is "0" to indicate the race has been won. The worker **302**A may, after reading and storing the data for the chunk **306** corresponding to A, then set the flag to "1" so that other workers racing to the chunk **306** will be informed the data has been read. It should be appreciated that because this is the first chunk of the process, it may be assumed that no data has been read.

[0069] The first worker **302**A continues the race to the next chunk **306** corresponding to B, and checks its status identifier value to determine whether the data has been previously read. Upon checking, as shown in the example, the value for the position **1** corresponding to B is set to "1" to indicate that the chunk **306** has been read. For example, the dashed line shows that the second worker **302**B won the race to read the chunk **306** corresponding to B and determined that the value of the position **1** was "0" to indicate that the chunk **306** could be picked up by the second worker **306**B. The second worker **306**B may then change the value of position **1** to "1" to indicate the chunk **306** has been picked up, and as a result, when the first worker **302**A reads the chunk **306** corresponding to B, the first worker **302**A may determine that the chunk **306**B corresponding to B is read and then proceed to write A to its destination location. As noted herein, the first worker **302**A writes A to its destination location, which in this example overwrites position **1** corresponding to B, because it is known, by the first worker **302**A, that the destination location has been picked up and stored within a registry. Accordingly, the first worker **302**A proceeds with writing the information to the destination location. If, however, it was determined that the position **1** had data that had not yet been picked up (e.g., a value of "0"), then the first worker **302**A may have read the chunk **306** corresponding to B into its registry, as shown in FIG. **3**A.

[0070] The illustrated example repeats the process with additional workers **302**C and **302**D. Other examples may include more or fewer workers, with the understanding that more workers than chunks may be inefficient. In this case, because there are only four different chunks **306**, having five or more workers would likely lead to idle workers and inefficiencies. As noted herein, the GPU may determine or set the number of workers based on information regarding how much data is to be moved and/or one or more parameters may be established, such as minimum and/or maximum threshold numbers of workers **302**.

[0071] The example continues with the third worker **302**C reading the chunk **306** corresponding to C, determining its value is "0" indicating C has not yet been picked up, and then setting the value to "1" indicating C has been picked up. As a result, when the second worker **302**B reads the chunk **306** corresponding to C, the second worker **302**B may know that the destination location has been stored in a registry (e.g., that position **2** is available for writing) and proceed to write B to position **2**. A similar process proceeds with the fourth worker **302**D regarding reading the chunk **306** corresponding to D in position **3**, setting the value, informing the third worker **302**C to write the chunk **306** corresponding to C in position **3**, and then writing the chunk **306** corresponding to D in position **4**.

[0072] Systems and methods may determine a status for a piece of data to ensure the data is set before overwriting the data within a memory location. For example, the systems and methods may include implementing an atomic-set of the destination data status, reading the destination data, and then overwriting the destination data. If, according to the atomic-set result, the worker was first to read the data, the process may be repeated for the new destination. As another example, embodiments may first try reading the status with a regular memory load, which may use fewer resources than the atomic). Then, if it were determined that the status was set, the atomic may be skipped, the destination data may be read, and then the destination memory may be overwritten because it would be determined that another worker has already taken care of the data that was in the destination memory. However, if the status was not set, then the atomic-set process could be used.

[0073] In at least one embodiment, systems and methods may be used to group different chunks to form contiguous memory units with a buffer. For example, different portions of a scene may be reused between different frames. How-

ever, the selected chunks of memory may form sparse "gaps" within the buffer, which may be too small to accommodate larger sequences of new chunks added to the scene within subsequent frames or operations. Various embodiments may use the indirect rebuild systems and methods described herein to group the reused memory chunks into a contiguous or semi-contiguous unit within the buffer in order to accommodate larger newly added chunks. FIG. **3**C illustrates an example grouping task **340** that may be used with embodiments of the present disclosure. In this example, a first frame **342** includes different chunks **344** (identified by letters A-N) within a memory buffer **346**. As discussed herein, the example of changes between subsequent frames, such as within a video sequence or during a graphic processing operation, is provided by way of non-limiting example and various features of the present embodiments may be used in a variety of different applications. The example further illustrates a second frame **348**, which may be a subsequent frame compared to the first frame **342**, where only a portion of the chunks **344** associated with the buffer **346** are reused. In this example, the chunks **344** corresponding to A, B, E, G, K, M, and N are being reused in the second frame **348**. Reusing only a portion of the chunks creates gaps or spaces **350** that are represented by "-".

[0074] The second frame **348** may introduce one or more new data objects **352**, which may be presented as a contiguous sequence. In this example, the data object **354** includes chunks **344** that are represented by V, W, X, Y, and Z. The chunks **344** are a contiguous set of five, but the buffer **346** does not include five contiguous chunks in the current gaps **350**. Additionally, merely moving or shifting the chunks **344** in a single direction by one step will not provide sufficient space for the data object **352**. Systems and methods of the present disclosure address this problem, among others, through the use of indirect builds that move chunks an arbitrary distance within the memory buffer in order to group or otherwise collect reused chunks into a contiguous sequence.

[0075] FIG. **3**D shows the grouping task **340** where different chunks **344** are shifted to form a contiguous sequence, which may be used with embodiments of the present disclosure. In this example, the memory buffer **346** includes the chunks **344** represented by A, B, E, G, K, M, N and the associated gaps **350**. By executing systems and methods of the present disclosure, as discussed herein, the different chunks **344** may be read, have their flags set, and then moved to a destination location within a single read/write operation. Additionally, as shown in this example, the movement may be an arbitrary number of positions that is selected in order to group the different chunks **344** and create a larger gap **350**. For example, prior to the movement, the chunk **344** corresponding to A is in position **0** and the chunk **344** corresponding to B is in position **1**, but after the move the chunk **344** corresponding to A is now in position **7** and the chunk **344** corresponding to B is in position **8**. The movement operation may be performed using any reasonable number of workers using the process described in FIGS. **3**A and **3**B, as one example.

[0076] The movement of the reused chunks **344** produces the continuous gap **350** corresponding to positions **0** through **6**. Instead of having gaps of two, one, three, and one chunk prior to the movement, there is now a gap with seven chunks **344**. Accordingly, the data object **352** may now be positioned within the gap **350**, which in this example maintains the continuity of the sequence by placing the data object in positions **2** through **6**. Systems and methods of the present disclosure may be used to group or cluster different chunks within memory in order to maintain continuous sequences and/or to provide space to insert new data objects.

[0077] FIG. **4**A illustrates an example flow chart for an example process **400** to move chunks of memory to different memory segments that may be used with embodiments of the present disclosure. It should be understood that for this and other processes presented herein that there can be additional, fewer, or alternative operations performed in similar or alternative order, or at least partially in parallel, within the scope of various embodiments unless otherwise specifically stated. In this example, a first memory segment is set to a first status after reading a chunk of data within the first memory segment to a registry **402**. The first memory segment may be a chunk of data within a memory buffer that is read by a worker that has "won" a race to be the first of a set of workers to read the first memory segment. In at least one embodiment, the first status may refer to an atomic-or that provides an indication to other workers indicative of whether or not the first memory segment has been read and/or has had the information stored within the first memory segment stored within a registry. The first status may be identified by other workers by setting a particular value, such as "1" to identify a memory segment that has been read.

[0078] In at least one embodiment, a write destination for the chunk of data may be determined **404**. The write destination may be different from a source destination (e.g., a current location for the chunk of data) and may be "shifted," compared to a current location or position. The write destination may be determined based on a variety of factors, such as a desire to group or combine different chunks of memory, an input command, and/or the like. The write destination may further be associated with one or more mapping algorithms, such as a sort and binary search or an array of destination indices.

[0079] A second memory segment at the write destination may be evaluated to determine whether the second memory segment has the first status **406**. As discussed herein, the first status may refer to an indicator that the data within the associated memory segment has been read into a registry. If the data has been read into the registry, then that may trigger an overwrite operation at the second memory segment. In at least one embodiment, the chunk of data may be written from the registry to the second memory segment **408**. In this manner, data may be read from one location, have its status flagged, and then be written to a different location.

[0080] FIG. **4**B illustrates an example flow chart of an example process **420** to move chunks of memory to different memory locations that can be used with at least one embodiment. In this example, a first data segment is read into a registry from an unread memory location **422**. For example, a worker may win a race to read a first data segment, determine the first data segment is unread, such as by evaluating a status identifier, and then read the first data segment into the registry. After reading the first data segment into the registry, an indicator for the first data segment (e.g., an indicator for an associated memory location) may be set to change a status of the first data segment from unread to read **424**. By changing the status of the first data segment, other workers or the same worker may be able to determine

whether it is safe to overwrite the memory location associated with the first data segment. Additionally, if multiple workers are trying to find data segments to read, the identifier may prevent a worker from reading a data segment that has already been read, thereby reducing duplication of a task that wastes computing resources.

[0081] In at least one embodiment, a destination memory location for the first data segment may be determined to be a second read memory location **426**. For example, the worker that reads the first data segment may receive instructions or determine a destination location for the first data segment. The worker may then check the indicator associated with the destination location to determine whether other data, if any, within that data segment has been read. If so, the indication of a read location may permit the worker to overwrite the data within the destination location. Accordingly, upon determining that the destination location is a second read memory location, the first data segment may be written from the registry to the second read memory location **428**. In this manner, embodiments of the present disclosure may be used to read through different data segments, identify data segments that may be overwritten, and then move data segments to the identified data segment locations.

[0082] FIG. **5** illustrates an example flow chart of an example process **500** to read through a memory buffer to identify and move data segments that may be used with at least one embodiment. In this example, a data segment of a memory buffer is read **502**. The data segment may be read by one or more workers that are executing a data movement task, among other options. The one or more workers may be deployed by a GPU responsive to one or more processing operations. In at least one embodiment, the number of workers deployed may be determined by the GPU based on information about the memory buffer and/or data segments. Additionally, in at least one embodiment, a user may tune or otherwise establish a desired number of workers. The number of workers may be based on a variety of different factors. As one example, the number of workers may be set to the number of data segments to be moved. As another example, the number of workers may be set to some multiple of the number of data segments to be moved, such as half as many, a third as many, and/or the like.

[0083] In this example, a status of the data segment is determined **504**. The status may correspond to whether or not data associated with the data segment has been read into a registry of one or more associated workers. It may be determined whether or not the status of the data segment corresponds to read (or some other equivalent to indicate the data is stored in another location, at least temporarily) **506**. If so, then the process for the particular worker making the determination may stop and then it may be determined whether there are other segments as part of the task **508**. If so, then a new data segment may be read. If not, then the worker(s) may terminate the process **510**.

[0084] If the data segment has not yet been read, then the status of the data segment may be set to read and the data from the data segment may be maintained within the registry, at least temporarily **512**. A destination data segment may then be determined **514**. The destination segment may correspond to a different memory location associated with the buffer. A status of the destination data segment may be determined **516**. For example, as noted herein, a bit or flag may be read to determine whether the bit or flag has been set to a value indicative of being read. In at least one embodiment, a determination is made whether the destination data segment has been read **518**. If not, then the worker may read the data from the destination data segment into the registry **520** and then return to set the value of the destination data segment. If the destination data segment has already been read, then data from the registry associated with the data segment may be written to the destination data segment **522**. In at least one embodiment, the process may be iteratively repeated for each data segment of the task associated with the memory buffer.

[0085] As discussed, aspects of various approaches presented herein can be lightweight enough to execute on a device such as a client device, such as a personal computer or gaming console, in real time. Such processing can be performed on, or for, content that is generated on, or received by, that client device or received from an external source, such as streaming data or other content received over at least one network. In some instances, the processing and/or determination of this content may be performed by one of these other devices, systems, or entities, then provided to the client device (or another such recipient) for presentation or another such use.

[0086] As an example, FIG. **6** illustrates an example network configuration **600** that can be used to provide, generate, modify, encode, process, and/or transmit image data or other such content. In at least one embodiment, a client device **602** can generate or receive data for a session using components of a control application **604** on client device **602** and data stored locally on that client device. In at least one embodiment, a content application **624** executing on a server **620** (e.g., a cloud server or edge server) may initiate a session associated with at least one client device **602**, as may utilize a session manager and user data stored in a user database **636**, and can cause content such as one or more digital assets (e.g., object representations) from an asset repository **634** to be determined by a content manager **626**. A content manager **626** may work with an image synthesis module **628** to generate or synthesize new objects, digital assets, or other such content to be provided for presentation via the client device **602**. In at least one embodiment, this image synthesis module **628** can use one or more neural networks, or machine learning models, which can be trained or updated using a training module **632** or system that is on, or in communication with, the server **620**. This can include training and/or using a diffusion model **630** to generate content tiles that can be used by an image synthesis module **628**, for example, to apply a non-repeating texture to a region of an environment for which image or video data is to be presented via a client device **602**. At least a portion of the generated content may be transmitted to the client device **602** using an appropriate transmission manager **622** to send by download, streaming, or another such transmission channel. An encoder may be used to encode and/or compress at least some of this data before transmitting to the client device **602**. In at least one embodiment, the client device **602** receiving such content can provide this content to a corresponding control application **604**, which may also or alternatively include a graphical user interface **610**, content manager **612**, and image synthesis or diffusion module **614** for use in providing, synthesizing, modifying, or using content for presentation (or other purposes) on or by the client device **602**. A decoder may also be used to decode data received over the network(s) **640** for presentation via client device **602**, such as image or video content through a

display **606** and audio, such as sounds and music, through at least one audio playback device **608**, such as speakers or headphones. In at least one embodiment, at least some of this content may already be stored on, rendered on, or accessible to client device **602** such that transmission over network **640** is not required for at least that portion of content, such as where that content may have been previously downloaded or stored locally on a hard drive or optical disk. In at least one embodiment, a transmission mechanism such as data streaming can be used to transfer this content from server **620**, or user database **636**, to client device **602**. In at least one embodiment, at least a portion of this content can be obtained, enhanced, and/or streamed from another source, such as a third party service **660** or other client device **650**, that may also include a content application **662** for generating, enhancing, or providing content. In at least one embodiment, portions of this functionality can be performed using multiple computing devices, or multiple processors within one or more computing devices, such as may include a combination of CPUs and GPUs.

[0087] In this example, these client devices can include any appropriate computing devices, as may include a desktop computer, notebook computer, set-top box, streaming device, gaming console, smartphone, tablet computer, VR headset, AR goggles, wearable computer, or a smart television. Each client device can submit a request across at least one wired or wireless network, as may include the Internet, an Ethernet, a local area network (LAN), or a cellular network, among other such options. In this example, these requests can be submitted to an address associated with a cloud provider, who may operate or control one or more electronic resources in a cloud provider environment, such as may include a data center or server farm. In at least one embodiment, the request may be received or processed by at least one edge server, that sits on a network edge and is outside at least one security layer associated with the cloud provider environment. In this way, latency can be reduced by enabling the client devices to interact with servers that are in closer proximity, while also improving security of resources in the cloud provider environment.

[0088] In at least one embodiment, such a system can be used for performing graphical rendering operations. In other embodiments, such a system can be used for other purposes, such as for providing image or video content to test or validate autonomous machine applications, or for performing deep learning operations. In at least one embodiment, such a system can be implemented using an edge device, or may incorporate one or more Virtual Machines (VMs). In at least one embodiment, such a system can be implemented at least partially in a data center or at least partially using cloud computing resources.

Inference and Training Logic

[0089] FIG. **7**A illustrates inference and/or training logic **715** used to perform inferencing and/or training operations associated with one or more embodiments. Details regarding inference and/or training logic **715** are provided below in conjunction with FIGS. **7**A and/or **7**B.

[0090] In at least one embodiment, inference and/or training logic **715** may include, without limitation, code and/or data storage **701** to store forward and/or output weight and/or input/output data, and/or other parameters to configure neurons or layers of a neural network trained and/or used for inferencing in aspects of one or more embodiments. In at least one embodiment, training logic **715** may include, or be coupled to code and/or data storage **701** to store graph code or other software to control timing and/or order, in which weight and/or other parameter information is to be loaded to configure, logic, including integer and/or floating point units (collectively, arithmetic logic units (ALUs). In at least one embodiment, code, such as graph code, loads weight or other parameter information into processor ALUs based on an architecture of a neural network to which the code corresponds. In at least one embodiment, code and/or data storage **701** stores weight parameters and/or input/output data of each layer of a neural network trained or used in conjunction with one or more embodiments during forward propagation of input/output data and/or weight parameters during training and/or inferencing using aspects of one or more embodiments. In at least one embodiment, any portion of code and/or data storage **701** may be included with other on-chip or off-chip data storage, including a processor's L**1**, L**2**, or L**3** cache or system memory.

[0091] In at least one embodiment, any portion of code and/or data storage **701** may be internal or external to one or more processors or other hardware logic devices or circuits. In at least one embodiment, code and/or data storage **701** may be cache memory, dynamic randomly addressable memory ("DRAM"), static randomly addressable memory ("SRAM"), non-volatile memory (e.g., Flash memory), or other storage. In at least one embodiment, choice of whether code and/or data storage **701** is internal or external to a processor, for example, or comprised of DRAM, SRAM, Flash or some other storage type may depend on available storage on-chip versus off-chip, latency requirements of training and/or inferencing functions being performed, batch size of data used in inferencing and/or training of a neural network, or some combination of these factors.

[0092] In at least one embodiment, inference and/or training logic **715** may include, without limitation, a code and/or data storage **705** to store backward and/or output weight and/or input/output data corresponding to neurons or layers of a neural network trained and/or used for inferencing in aspects of one or more embodiments. In at least one embodiment, code and/or data storage **705** stores weight parameters and/or input/output data of each layer of a neural network trained or used in conjunction with one or more embodiments during backward propagation of input/output data and/or weight parameters during training and/or inferencing using aspects of one or more embodiments. In at least one embodiment, training logic **715** may include, or be coupled to code and/or data storage **705** to store graph code or other software to control timing and/or order, in which weight and/or other parameter information is to be loaded to configure, logic, including integer and/or floating point units (collectively, arithmetic logic units (ALUs). In at least one embodiment, code, such as graph code, loads weight or other parameter information into processor ALUs based on an architecture of a neural network to which the code corresponds. In at least one embodiment, any portion of code and/or data storage **705** may be included with other on-chip or off-chip data storage, including a processor's L**1**, L**2**, or L**3** cache or system memory. In at least one embodiment, any portion of code and/or data storage **705** may be internal or external to on one or more processors or other hardware logic devices or circuits. In at least one embodiment, code and/or data storage **705** may be cache memory, DRAM, SRAM, non-volatile memory (e.g., Flash memory), or other

storage. In at least one embodiment, choice of whether code and/or data storage **705** is internal or external to a processor, for example, or comprised of DRAM, SRAM, Flash or some other storage type may depend on available storage on-chip versus off-chip, latency requirements of training and/or inferencing functions being performed, batch size of data used in inferencing and/or training of a neural network, or some combination of these factors.

[0093] In at least one embodiment, code and/or data storage **701** and code and/or data storage **705** may be separate storage structures. In at least one embodiment, code and/or data storage **701** and code and/or data storage **705** may be same storage structure. In at least one embodiment, code and/or data storage **701** and code and/or data storage **705** may be partially same storage structure and partially separate storage structures. In at least one embodiment, any portion of code and/or data storage **701** and code and/or data storage **705** may be included with other on-chip or off-chip data storage, including a processor's L**1**, L**2**, or L**3** cache or system memory.

[0094] In at least one embodiment, inference and/or training logic **715** may include, without limitation, one or more arithmetic logic unit(s) ("ALU(s)") **710**, including integer and/or floating point units, to perform logical and/or mathematical operations based, at least in part on, or indicated by, training and/or inference code (e.g., graph code), a result of which may produce activations (e.g., output values from layers or neurons within a neural network) stored in an activation storage **720** that are functions of input/output and/or weight parameter data stored in code and/or data storage **701** and/or code and/or data storage **705**. In at least one embodiment, activations stored in activation storage **720** are generated according to linear algebraic and or matrix-based mathematics performed by ALU(s) **710** in response to performing instructions or other code, wherein weight values stored in code and/or data storage **705** and/or code and/or data storage **701** are used as operands along with other values, such as bias values, gradient information, momentum values, or other parameters or hyperparameters, any or all of which may be stored in code and/or data storage **705** or code and/or data storage **701** or another storage on or off-chip.

[0095] In at least one embodiment, ALU(s) **710** are included within one or more processors or other hardware logic devices or circuits, whereas in another embodiment, ALU(s) **710** may be external to a processor or other hardware logic device or circuit that uses them (e.g., a co-processor). In at least one embodiment, ALU(s) **710** may be included within a processor's execution units or otherwise within a bank of ALUs accessible by a processor's execution units either within same processor or distributed between different processors of different types (e.g., central processing units, graphics processing units, fixed function units, etc.). In at least one embodiment, code and/or data storage **701**, code and/or data storage **705**, and activation storage **720** may be on same processor or other hardware logic device or circuit, whereas in another embodiment, they may be in different processors or other hardware logic devices or circuits, or some combination of same and different processors or other hardware logic devices or circuits. In at least one embodiment, any portion of activation storage **720** may be included with other on-chip or off-chip data storage, including a processor's L**1**, L**2**, or L**3** cache or system memory. Furthermore, inferencing and/or training code may be stored with other code accessible to a processor or other hardware logic or circuit and fetched and/or processed using a processor's fetch, decode, scheduling, execution, retirement and/or other logical circuits.

[0096] In at least one embodiment, activation storage **720** may be cache memory, DRAM, SRAM, non-volatile memory (e.g., Flash memory), or other storage. In at least one embodiment, activation storage **720** may be completely or partially within or external to one or more processors or other logical circuits. In at least one embodiment, choice of whether activation storage **720** is internal or external to a processor, for example, or comprised of DRAM, SRAM, Flash or some other storage type may depend on available storage on-chip versus off-chip, latency requirements of training and/or inferencing functions being performed, batch size of data used in inferencing and/or training of a neural network, or some combination of these factors. In at least one embodiment, inference and/or training logic **715** illustrated in FIG. **7**A may be used in conjunction with an application-specific integrated circuit ("ASIC"), such as Tensorflow® Processing Unit from Google, an inference processing unit (IPU) from Graphcore™, or a Nervana® (e.g., "Lake Crest") processor from Intel Corp. In at least one embodiment, inference and/or training logic **715** illustrated in FIG. **7**A may be used in conjunction with central processing unit ("CPU") hardware, graphics processing unit ("GPU") hardware or other hardware, such as field programmable gate arrays ("FPGAs").

[0097] FIG. **7**B illustrates inference and/or training logic **715**, according to at least one or more embodiments. In at least one embodiment, inference and/or training logic **715** may include, without limitation, hardware logic in which computational resources are dedicated or otherwise exclusively used in conjunction with weight values or other information corresponding to one or more layers of neurons within a neural network. In at least one embodiment, inference and/or training logic **715** illustrated in FIG. **7**B may be used in conjunction with an application-specific integrated circuit (ASIC), such as Tensorflow® Processing Unit from Google, an inference processing unit (IPU) from Graphcore™, or a Nervana® (e.g., "Lake Crest") processor from Intel Corp. In at least one embodiment, inference and/or training logic **715** illustrated in FIG. **7**B may be used in conjunction with central processing unit (CPU) hardware, graphics processing unit (GPU) hardware or other hardware, such as field programmable gate arrays (FPGAs). In at least one embodiment, inference and/or training logic **715** includes, without limitation, code and/or data storage **701** and code and/or data storage **705**, which may be used to store code (e.g., graph code), weight values and/or other information, including bias values, gradient information, momentum values, and/or other parameter or hyperparameter information. In at least one embodiment illustrated in FIG. **7**B, each of code and/or data storage **701** and code and/or data storage **705** is associated with a dedicated computational resource, such as computational hardware **702** and computational hardware **706**, respectively. In at least one embodiment, each of computational hardware **702** and computational hardware **706** comprises one or more ALUs that perform mathematical functions, such as linear algebraic functions, only on information stored in code and/or data storage **701** and code and/or data storage **705**, respectively, result of which is stored in activation storage **720**.

[0098] In at least one embodiment, each of code and/or data storage **701** and **705** and corresponding computational hardware **702** and **706**, respectively, correspond to different layers of a neural network, such that resulting activation from one "storage/computational pair **701**/**702**" of code and/or data storage **701** and computational hardware **702** is provided as an input to "storage/computational pair **705**/**706**" of code and/or data storage **705** and computational hardware **706**, in order to mirror conceptual organization of a neural network. In at least one embodiment, each of storage/computational pairs **701**/**702** and **705**/**706** may correspond to more than one neural network layer. In at least one embodiment, additional storage/computation pairs (not shown) subsequent to or in parallel with storage computation pairs **701**/**702** and **705**/**706** may be included in inference and/or training logic **715**.

Data Center

[0099] FIG. **8** illustrates an example data center **800**, in which at least one embodiment may be used. In at least one embodiment, data center **800** includes a data center infrastructure layer **810**, a framework layer **820**, a software layer **830**, and an application layer **840**.

[0100] In at least one embodiment, as shown in FIG. **8**, data center infrastructure layer **810** may include a resource orchestrator **812**, grouped computing resources **814**, and node computing resources ("node C.R.s") **816**(**1**)-**816**(N), where "N" represents any whole, positive integer. In at least one embodiment, node C.R.s **816**(**1**)-**816**(N) may include, but are not limited to, any number of central processing units ("CPUs") or other processors (including accelerators, field programmable gate arrays (FPGAs), graphics processors, etc.), memory devices (e.g., dynamic read-only memory), storage devices (e.g., solid state or disk drives), network input/output ("NW I/O") devices, network switches, virtual machines ("VMs"), power modules, and cooling modules, etc. In at least one embodiment, one or more node C.R.s from among node C.R.s **816**(**1**)-**816**(N) may be a server having one or more of above-mentioned computing resources.

[0101] In at least one embodiment, grouped computing resources **814** may include separate groupings of node C.R.s housed within one or more racks (not shown), or many racks housed in data centers at various geographical locations (also not shown). Separate groupings of node C.R.s within grouped computing resources **814** may include grouped compute, network, memory or storage resources that may be configured or allocated to support one or more workloads. In at least one embodiment, several node C.R.s including CPUs or processors may be grouped within one or more racks to provide compute resources to support one or more workloads. In at least one embodiment, one or more racks may also include any number of power modules, cooling modules, and network switches, in any combination.

[0102] In at least one embodiment, resource orchestrator **812** may configure or otherwise control one or more node C.R.s **816**(**1**)-**816**(N) and/or grouped computing resources **814**. In at least one embodiment, resource orchestrator **812** may include a software design infrastructure ("SDI") management entity for data center **800**. In at least one embodiment, resource orchestrator **812** may include hardware, software or some combination thereof.

[0103] In at least one embodiment, as shown in FIG. **8**, framework layer **820** includes a job scheduler **822**, a configuration manager **824**, a resource manager **826** and a distributed file system **828**. In at least one embodiment, framework layer **820** may include a framework to support software **832** of software layer **830** and/or one or more application(s) **842** of application layer **840**. In at least one embodiment, software **832** or application(s) **842** may respectively include web-based service software or applications, such as those provided by Amazon Web Services, Google Cloud and Microsoft Azure. In at least one embodiment, framework layer **820** may be, but is not limited to, a type of free and open-source software web application framework such as Apache Spark™ (hereinafter "Spark") that may use distributed file system **828** for large-scale data processing (e.g., "big data"). In at least one embodiment, job scheduler **822** may include a Spark driver to facilitate scheduling of workloads supported by various layers of data center **800**. In at least one embodiment, configuration manager **824** may be capable of configuring different layers such as software layer **830** and framework layer **820** including Spark and distributed file system **828** for supporting large-scale data processing. In at least one embodiment, resource manager **826** may be capable of managing clustered or grouped computing resources mapped to or allocated for support of distributed file system **828** and job scheduler **822**. In at least one embodiment, clustered or grouped computing resources may include grouped computing resource **814** at data center infrastructure layer **810**. In at least one embodiment, resource manager **826** may coordinate with resource orchestrator **812** to manage these mapped or allocated computing resources.

[0104] In at least one embodiment, software **832** included in software layer **830** may include software used by at least portions of node C.R.s **816**(**1**)-**816**(N), grouped computing resources **814**, and/or distributed file system **828** of framework layer **820**. The one or more types of software may include, but are not limited to, Internet web page search software, e-mail virus scan software, database software, and streaming video content software.

[0105] In at least one embodiment, application(s) **842** included in application layer **840** may include one or more types of applications used by at least portions of node C.R.s **816**(**1**)-**816**(N), grouped computing resources **814**, and/or distributed file system **828** of framework layer **820**. One or more types of applications may include, but are not limited to, any number of a genomics application, a cognitive compute, and a machine learning application, including training or inferencing software, machine learning framework software (e.g., PyTorch, TensorFlow, Caffe, etc.) or other machine learning applications used in conjunction with one or more embodiments.

[0106] In at least one embodiment, any of configuration manager **824**, resource manager **826**, and resource orchestrator **812** may implement any number and type of self-modifying actions based on any amount and type of data acquired in any technically feasible fashion. In at least one embodiment, self-modifying actions may relieve a data center operator of data center **800** from making possibly bad configuration decisions and possibly avoiding underused and/or poor performing portions of a data center.

[0107] In at least one embodiment, data center **800** may include tools, services, software or other resources to train one or more machine learning models or predict or infer information using one or more machine learning models according to one or more embodiments described herein.

For example, in at least one embodiment, a machine learning model may be trained by calculating weight parameters according to a neural network architecture using software and computing resources described above with respect to data center **800**. In at least one embodiment, trained machine learning models corresponding to one or more neural networks may be used to infer or predict information using resources described above with respect to data center **800** by using weight parameters calculated through one or more training techniques described herein.

[0108] In at least one embodiment, data center may use CPUs, application-specific integrated circuits (ASICs), GPUs, FPGAs, or other hardware to perform training and/or inferencing using above-described resources. Moreover, one or more software and/or hardware resources described above may be configured as a service to allow users to train or performing inferencing of information, such as image recognition, speech recognition, or other artificial intelligence services.

[0109] Inference and/or training logic **715** are used to perform inferencing and/or training operations associated with one or more embodiments. Details regarding inference and/or training logic **715** are provided below in conjunction with FIGS. **7**A and/or **7**B. In at least one embodiment, inference and/or training logic **715** may be used in system FIG. **8** for inferencing or predicting operations based, at least in part, on weight parameters calculated using neural network training operations, neural network functions and/or architectures, or neural network use cases described herein.

[0110] Such components can be used for data movement.

Computer Systems

[0111] FIG. **9** is a block diagram illustrating an exemplary computer system, which may be a system with interconnected devices and components, a system-on-a-chip (SOC) or some combination thereof **900** formed with a processor that may include execution units to execute an instruction, according to at least one embodiment. In at least one embodiment, computer system **900** may include, without limitation, a component, such as a processor **902** to employ execution units including logic to perform algorithms for process data, in accordance with present disclosure, such as in embodiment described herein. In at least one embodiment, computer system **900** may include processors, such as PENTIUM® Processor family, Xeon™, Itanium®, XScale™ and/or StrongARM™, Intel® Core™, or Intel® Nervana™ microprocessors available from Intel Corporation of Santa Clara, California, although other systems (including PCs having other microprocessors, engineering workstations, set-top boxes and like) may also be used. In at least one embodiment, computer system **900** may execute a version of WINDOWS' operating system available from Microsoft Corporation of Redmond, Wash., although other operating systems (UNIX and Linux for example), embedded software, and/or graphical user interfaces, may also be used.

[0112] Embodiments may be used in other devices such as handheld devices and embedded applications. Some examples of handheld devices include cellular phones, Internet Protocol devices, digital cameras, personal digital assistants ("PDAs"), and handheld PCs. In at least one embodiment, embedded applications may include a microcontroller, a digital signal processor ("DSP"), system on a chip, network computers ("NetPCs"), set-top boxes, network hubs, wide area network ("WAN") switches, or any other system that may perform one or more instructions in accordance with at least one embodiment.

[0113] In at least one embodiment, computer system **900** may include, without limitation, processor **902** that may include, without limitation, one or more execution units **908** to perform machine learning model training and/or inferencing according to techniques described herein. In at least one embodiment, computer system **900** is a single processor desktop or server system, but in another embodiment computer system **900** may be a multiprocessor system. In at least one embodiment, processor **902** may include, without limitation, a complex instruction set computing ("CISC") microprocessor, a reduced instruction set computing ("RISC") microprocessor, a very long instruction word ("VLIW") computing microprocessor, a processor implementing a combination of instruction sets, or any other processor device, such as a digital signal processor, for example. In at least one embodiment, processor **902** may be coupled to a processor bus **910** that may transmit data signals between processor **902** and other components in computer system **900**.

[0114] In at least one embodiment, processor **902** may include, without limitation, a Level **1** ("L**1**") internal cache memory ("cache") **904**. In at least one embodiment, processor **902** may have a single internal cache or multiple levels of internal cache. In at least one embodiment, cache memory may reside external to processor **902**. Other embodiments may also include a combination of both internal and external caches depending on particular implementation and needs. In at least one embodiment, register file **906** may store different types of data in various registers including, without limitation, integer registers, floating point registers, status registers, and instruction pointer register.

[0115] In at least one embodiment, execution unit **908**, including, without limitation, logic to perform integer and floating point operations, also resides in processor **902**. In at least one embodiment, processor **902** may also include a microcode ("ucode") read only memory ("ROM") that stores microcode for certain macro instructions. In at least one embodiment, execution unit **908** may include logic to handle a packed instruction set **909**. In at least one embodiment, by including packed instruction set **909** in an instruction set of a general-purpose processor **902**, along with associated circuitry to execute instructions, operations used by many multimedia applications may be performed using packed data in a general-purpose processor **902**. In one or more embodiments, many multimedia applications may be accelerated and executed more efficiently by using full width of a processor's data bus for performing operations on packed data, which may eliminate need to transfer smaller units of data across processor's data bus to perform one or more operations one data element at a time.

[0116] In at least one embodiment, execution unit **908** may also be used in microcontrollers, embedded processors, graphics devices, DSPs, and other types of logic circuits. In at least one embodiment, computer system **900** may include, without limitation, a memory **920**. In at least one embodiment, memory **920** may be implemented as a Dynamic Random Access Memory ("DRAM") device, a Static Random Access Memory ("SRAM") device, flash memory device, or other memory device. In at least one embodiment,

memory **920** may store instruction(s) **919** and/or data **921** represented by data signals that may be executed by processor **902**.

[0117] In at least one embodiment, system logic chip may be coupled to processor bus **910** and memory **920**. In at least one embodiment, system logic chip may include, without limitation, a memory controller hub ("MCH") **916**, and processor **902** may communicate with MCH **916** via processor bus **910**. In at least one embodiment, MCH **916** may provide a high bandwidth memory path **918** to memory **920** for instruction and data storage and for storage of graphics commands, data and textures. In at least one embodiment, MCH **916** may direct data signals between processor **902**, memory **920**, and other components in computer system **900** and to bridge data signals between processor bus **910**, memory **920**, and a system I/O **922**. In at least one embodiment, system logic chip may provide a graphics port for coupling to a graphics controller. In at least one embodiment, MCH **916** may be coupled to memory **920** through a high bandwidth memory path **918** and graphics/video card **912** may be coupled to MCH **916** through an Accelerated Graphics Port ("AGP") interconnect **914**.

[0118] In at least one embodiment, computer system **900** may use system I/O **922** that is a proprietary hub interface bus to couple MCH **916** to I/O controller hub ("ICH") **930**. In at least one embodiment, ICH **930** may provide direct connections to some I/O devices via a local I/O bus. In at least one embodiment, local I/O bus may include, without limitation, a high-speed I/O bus for connecting peripherals to memory **920**, chipset, and processor **902**. Examples may include, without limitation, an audio controller **929**, a firmware hub ("flash BIOS") **928**, a wireless transceiver **926**, a data storage **924**, a legacy I/O controller **923** containing user input and keyboard interfaces **925**, a serial expansion port **927**, such as Universal Serial Bus ("USB"), and a network controller **934**. Data storage **924** may comprise a hard disk drive, a floppy disk drive, a CD-ROM device, a flash memory device, or other mass storage device.

[0119] In at least one embodiment, FIG. **9** illustrates a system, which includes interconnected hardware devices or "chips", whereas in other embodiments, FIG. **9** may illustrate an exemplary System on a Chip ("SoC"). In at least one embodiment, devices may be interconnected with proprietary interconnects, standardized interconnects (e.g., PCIe) or some combination thereof. In at least one embodiment, one or more components of computer system **900** are interconnected using compute express link (CXL) interconnects.

[0120] Inference and/or training logic **715** are used to perform inferencing and/or training operations associated with one or more embodiments. Details regarding inference and/or training logic **715** are provided below in conjunction with FIGS. **7**A and/or **7**B. In at least one embodiment, inference and/or training logic **715** may be used in system FIG. **9** for inferencing or predicting operations based, at least in part, on weight parameters calculated using neural network training operations, neural network functions and/or architectures, or neural network use cases described herein.

[0121] Such components can be used for data movement.

[0122] FIG. **10** is a block diagram illustrating an electronic device **1000** for utilizing a processor **1010**, according to at least one embodiment. In at least one embodiment, electronic device **1000** may be, for example and without limitation, a notebook, a tower server, a rack server, a blade server, a laptop, a desktop, a tablet, a mobile device, a phone, an embedded computer, or any other suitable electronic device.

[0123] In at least one embodiment, system **1000** may include, without limitation, processor **1010** communicatively coupled to any suitable number or kind of components, peripherals, modules, or devices. In at least one embodiment, processor **1010** coupled using a bus or interface, such as a 1° C. bus, a System Management Bus ("SMBus"), a Low Pin Count (LPC) bus, a Serial Peripheral Interface ("SPI"), a High Definition Audio ("HDA") bus, a Serial Advance Technology Attachment ("SATA") bus, a Universal Serial Bus ("USB") (versions 1, 2, 3), or a Universal Asynchronous Receiver/Transmitter ("UART") bus. In at least one embodiment, FIG. **10** illustrates a system, which includes interconnected hardware devices or "chips", whereas in other embodiments, FIG. **10** may illustrate an exemplary System on a Chip ("SoC"). In at least one embodiment, devices illustrated in FIG. **10** may be interconnected with proprietary interconnects, standardized interconnects (e.g., PCIe) or some combination thereof. In at least one embodiment, one or more components of FIG. **10** are interconnected using compute express link (CXL) interconnects.

[0124] In at least one embodiment, FIG. **10** may include a display **1024**, a touch screen **1025**, a touch pad **1030**, a Near Field Communications unit ("NFC") **1045**, a sensor hub **1040**, a thermal sensor **1046**, an Express Chipset ("EC") **1035**, a Trusted Platform Module ("TPM") **1038**, BIOS/firmware/flash memory ("BIOS, FW Flash") **1022**, a DSP **1060**, a drive **1020** such as a Solid State Disk ("SSD") or a Hard Disk Drive ("HDD"), a wireless local area network unit ("WLAN") **1050**, a Bluetooth unit **1052**, a Wireless Wide Area Network unit ("WWAN") **1056**, a Global Positioning System (GPS) **1055**, a camera ("USB 3.0 camera") **1054** such as a USB 3.0 camera, and/or a Low Power Double Data Rate ("LPDDR") memory unit ("LPDDR3") **1015** implemented in, for example, LPDDR3 standard. These components may each be implemented in any suitable manner.

[0125] In at least one embodiment, other components may be communicatively coupled to processor **1010** through components discussed above. In at least one embodiment, an accelerometer **1041**, Ambient Light Sensor ("ALS") **1042**, compass **1043**, and a gyroscope **1044** may be communicatively coupled to sensor hub **1040**. In at least one embodiment, thermal sensor **1039**, a fan **1037**, a keyboard **1036**, and a touch pad **1030** may be communicatively coupled to EC **1035**. In at least one embodiment, speakers **1063**, headphones **1064**, and microphone ("mic") **1065** may be communicatively coupled to an audio unit ("audio codec and class d amp") **1062**, which may in turn be communicatively coupled to DSP **1060**. In at least one embodiment, audio unit **1062** may include, for example and without limitation, an audio coder/decoder ("codec") and a class D amplifier. In at least one embodiment, SIM card ("SIM") **1057** may be communicatively coupled to WWAN unit **1056**. In at least one embodiment, components such as WLAN unit **1050** and Bluetooth unit **1052**, as well as WWAN unit **1056** may be implemented in a Next Generation Form Factor ("NGFF").

[0126] Inference and/or training logic **715** are used to perform inferencing and/or training operations associated with one or more embodiments. Details regarding inference

and/or training logic **715** are provided below in conjunction with FIGS. **7**A and/or **7**B. In at least one embodiment, inference and/or training logic **715** may be used in system FIG. **10** for inferencing or predicting operations based, at least in part, on weight parameters calculated using neural network training operations, neural network functions and/or architectures, or neural network use cases described herein.

[0127] Such components can be used for data movement.

[0128] FIG. **11** is a block diagram of a processing system, according to at least one embodiment. In at least one embodiment, system **1100** includes one or more processor(s) **1102** and one or more graphics processor(s) **1108**, and may be a single processor desktop system, a multiprocessor workstation system, or a server system having a large number of processor(s) **1102** or processor core(s) **1107**. In at least one embodiment, system **1100** is a processing platform incorporated within a system-on-a-chip (SoC) integrated circuit for use in mobile, handheld, or embedded devices.

[0129] In at least one embodiment, system **1100** can include, or be incorporated within a server-based gaming platform, a game console, including a game and media console, a mobile gaming console, a handheld game console, or an online game console. In at least one embodiment, system **1100** is a mobile phone, smart phone, tablet computing device or mobile Internet device. In at least one embodiment, processing system **1100** can also include, coupled with, or be integrated within a wearable device, such as a smart watch wearable device, smart eyewear device, augmented reality device, or virtual reality device. In at least one embodiment, processing system **1100** is a television or set top box device having one or more processor(s) **1102** and a graphical interface generated by one or more graphics processor(s) **1108**.

[0130] In at least one embodiment, one or more processor (s) **1102** each include one or more processor core(s) **1107** to process instructions which, when executed, perform operations for system and user software. In at least one embodiment, each of one or more processor core(s) **1107** is configured to process a specific instruction set **1109**. In at least one embodiment, instruction set **1109** may facilitate Complex Instruction Set Computing (CISC), Reduced Instruction Set Computing (RISC), or computing via a Very Long Instruction Word (VLIW). In at least one embodiment, processor core(s) **1107** may each process a different instruction set **1109**, which may include instructions to facilitate emulation of other instruction sets. In at least one embodiment, processor core(s) **1107** may also include other processing devices, such a Digital Signal Processor (DSP).

[0131] In at least one embodiment, processor(s) **1102** includes cache memory **1104**. In at least one embodiment, processor(s) **1102** can have a single internal cache or multiple levels of internal cache. In at least one embodiment, cache memory is shared among various components of processor(s) **1102**. In at least one embodiment, processor(s) **1102** also uses an external cache (e.g., a Level-3 (L3) cache or Last Level Cache (LLC)) (not shown), which may be shared among processor core(s) **1107** using known cache coherency techniques. In at least one embodiment, register file **1106** is additionally included in processor(s) **1102** which may include different types of registers for storing different types of data (e.g., integer registers, floating point registers, status registers, and an instruction pointer register). In at least one embodiment, register file **1106** may include general-purpose registers or other registers.

[0132] In at least one embodiment, one or more processor (s) **1102** are coupled with one or more interface bus(es) **1110** to transmit communication signals such as address, data, or control signals between processor(s) **1102** and other components in system **1100**. In at least one embodiment, interface bus(es) **1110**, in one embodiment, can be a processor bus, such as a version of a Direct Media Interface (DMI) bus. In at least one embodiment, interface bus(es) **1110** is not limited to a DMI bus, and may include one or more Peripheral Component Interconnect buses (e.g., PCI, PCI Express), memory busses, or other types of interface busses. In at least one embodiment processor(s) **1102** include an integrated memory controller **1116** and a platform controller hub **1130**. In at least one embodiment, memory controller **1116** facilitates communication between a memory device and other components of system **1100**, while platform controller hub (PCH) **1130** provides connections to I/O devices via a local I/O bus.

[0133] In at least one embodiment, memory device **1120** can be a dynamic random access memory (DRAM) device, a static random access memory (SRAM) device, flash memory device, phase-change memory device, or some other memory device having suitable performance to serve as process memory. In at least one embodiment memory device **1120** can operate as system memory for system **1100**, to store data **1122** and instruction **1121** for use when one or more processor(s) **1102** executes an application or process. In at least one embodiment, memory controller **1116** also couples with an optional external graphics processor **1112**, which may communicate with one or more graphics processor(s) **1108** in processor(s) **1102** to perform graphics and media operations. In at least one embodiment, a display device **1111** can connect to processor(s) **1102**. In at least one embodiment display device **1111** can include one or more of an internal display device, as in a mobile electronic device or a laptop device or an external display device attached via a display interface (e.g., DisplayPort, etc.). In at least one embodiment, display device **1111** can include a head mounted display (HMD) such as a stereoscopic display device for use in virtual reality (VR) applications or augmented reality (AR) applications.

[0134] In at least one embodiment, platform controller hub **1130** enables peripherals to connect to memory device **1120** and processor(s) **1102** via a high-speed I/O bus. In at least one embodiment, I/O peripherals include, but are not limited to, an audio controller **1146**, a network controller **1134**, a firmware interface **1128**, a wireless transceiver **1126**, touch sensors **1125**, a data storage device **1124** (e.g., hard disk drive, flash memory, etc.). In at least one embodiment, data storage device **1124** can connect via a storage interface (e.g., SATA) or via a peripheral bus, such as a Peripheral Component Interconnect bus (e.g., PCI, PCI Express). In at least one embodiment, touch sensors **1125** can include touch screen sensors, pressure sensors, or fingerprint sensors. In at least one embodiment, wireless transceiver **1126** can be a Wi-Fi transceiver, a Bluetooth transceiver, or a mobile network transceiver such as a 3G, 4G, or Long Term Evolution (LTE) transceiver. In at least one embodiment, firmware interface **1128** enables communication with system firmware, and can be, for example, a unified extensible firmware interface (UEFI). In at least one embodiment, network controller **1134** can enable a network connection to

a wired network. In at least one embodiment, a high-performance network controller (not shown) couples with interface bus(es) **1110**. In at least one embodiment, audio controller **1146** is a multi-channel high definition audio controller. In at least one embodiment, system **1100** includes an optional legacy I/O controller **1140** for coupling legacy (e.g., Personal System 2 (PS/2)) devices to system. In at least one embodiment, platform controller hub **1130** can also connect to one or more Universal Serial Bus (USB) controller(s) **1142** connect input devices, such as keyboard and mouse **1143** combinations, a camera **1144**, or other USB input devices.

[0135] In at least one embodiment, an instance of memory controller **1116** and platform controller hub **1130** may be integrated into a discreet external graphics processor, such as external graphics processor **1112**. In at least one embodiment, platform controller hub **1130** and/or memory controller **1116** may be external to one or more processor(s) **1102**. For example, in at least one embodiment, system **1100** can include an external memory controller **1116** and platform controller hub **1130**, which may be configured as a memory controller hub and peripheral controller hub within a system chipset that is in communication with processor(s) **1102**.

[0136] Inference and/or training logic **715** are used to perform inferencing and/or training operations associated with one or more embodiments. Details regarding inference and/or training logic **715** are provided below in conjunction with FIGS. **7**A and/or **7**B. In at least one embodiment portions or all of inference and/or training logic **715** may be incorporated into graphics processor **1500**. For example, in at least one embodiment, training and/or inferencing techniques described herein may use one or more of ALUs embodied in a graphics processor. Moreover, in at least one embodiment, inferencing and/or training operations described herein may be done using logic other than logic illustrated in FIGS. **7**A and/or **7**B. In at least one embodiment, weight parameters may be stored in on-chip or off-chip memory and/or registers (shown or not shown) that configure ALUs of a graphics processor to perform one or more machine learning algorithms, neural network architectures, use cases, or training techniques described herein.

[0137] Such components can be used for data movement.

[0138] FIG. **12** is a block diagram of a processor **1200** having one or more processor core(s) **1202**A-**1202**N, an integrated memory controller **1214**, and an integrated graphics processor **1208**, according to at least one embodiment. In at least one embodiment, processor **1200** can include additional cores up to and including additional core **1202**N represented by dashed lined boxes. In at least one embodiment, each of processor core(s) **1202**A-**1202**N includes one or more internal cache unit(s) **1204**A-**1204**N. In at least one embodiment, each processor core also has access to one or more shared cached unit(s) **1206**.

[0139] In at least one embodiment, internal cache unit(s) **1204**A-**1204**N and shared cache unit(s) **1206** represent a cache memory hierarchy within processor **1200**. In at least one embodiment, cache unit(s) **1204**A-**1204**N may include at least one level of instruction and data cache within each processor core and one or more levels of shared mid-level cache, such as a Level 2 (L2), Level 3 (L3), Level 4 (L4), or other levels of cache, where a highest level of cache before external memory is classified as an LLC. In at least one embodiment, cache coherency logic maintains coherency between various cache unit(s) **1206** and **1204**A-**1204**N.

[0140] In at least one embodiment, processor **1200** may also include a set of one or more bus controller unit(s) **1216** and a system agent core **1210**. In at least one embodiment, one or more bus controller unit(s) **1216** manage a set of peripheral buses, such as one or more PCI or PCI express busses. In at least one embodiment, system agent core **1210** provides management functionality for various processor components. In at least one embodiment, system agent core **1210** includes one or more integrated memory controllers **1214** to manage access to various external memory devices (not shown).

[0141] In at least one embodiment, one or more of processor core(s) **1202**A-**1202**N include support for simultaneous multi-threading. In at least one embodiment, system agent core **1210** includes components for coordinating and processor core(s) **1202**A-**1202**N during multi-threaded processing. In at least one embodiment, system agent core **1210** may additionally include a power control unit (PCU), which includes logic and components to regulate one or more power states of processor core(s) **1202**A-**1202**N and graphics processor **1208**.

[0142] In at least one embodiment, processor **1200** additionally includes graphics processor **1208** to execute graphics processing operations. In at least one embodiment, graphics processor **1208** couples with shared cache unit(s) **1206**, and system agent core **1210**, including one or more integrated memory controllers **1214**. In at least one embodiment, system agent core **1210** also includes a display controller **1211** to drive graphics processor output to one or more coupled displays. In at least one embodiment, display controller **1211** may also be a separate module coupled with graphics processor **1208** via at least one interconnect, or may be integrated within graphics processor **1208**.

[0143] In at least one embodiment, a ring based interconnect unit **1212** is used to couple internal components of processor **1200**. In at least one embodiment, an alternative interconnect unit may be used, such as a point-to-point interconnect, a switched interconnect, or other techniques. In at least one embodiment, graphics processor **1208** couples with a ring based interconnect unit **1212** via an I/O link **1213**.

[0144] In at least one embodiment, I/O link **1213** represents at least one of multiple varieties of I/O interconnects, including an on package I/O interconnect which facilitates communication between various processor components and a high-performance embedded memory module **1218**, such as an eDRAM module. In at least one embodiment, each of processor core(s) **1202**A-**1202**N and graphics processor **1208** use embedded memory modules **1218** as a shared Last Level Cache.

[0145] In at least one embodiment, processor core(s) **1202**A-**1202**N are homogenous cores executing a common instruction set architecture. In at least one embodiment, processor core(s) **1202**A-**1202**N are heterogeneous in terms of instruction set architecture (ISA), where one or more of processor core(s) **1202**A-**1202**N execute a common instruction set, while one or more other cores of processor core(s) **1202**A-**1202**N executes a subset of a common instruction set or a different instruction set. In at least one embodiment, processor core(s) **1202**A-**1202**N are heterogeneous in terms of microarchitecture, where one or more cores having a relatively higher power consumption couple with one or more power cores having a lower power consumption. In at

least one embodiment, processor **1200** can be implemented on one or more chips or as an SoC integrated circuit.

[0146] Inference and/or training logic **715** are used to perform inferencing and/or training operations associated with one or more embodiments. Details regarding inference and/or training logic **715** are provided below in conjunction with FIGS. **7**A and/or **7**B. In at least one embodiment portions or all of inference and/or training logic **715** may be incorporated into processor **1200**. For example, in at least one embodiment, training and/or inferencing techniques described herein may use one or more of ALUs embodied in graphics processor **1208**, graphics core(s) **1202**A-**1202**N, or other components in FIG. **12**. Moreover, in at least one embodiment, inferencing and/or training operations described herein may be done using logic other than logic illustrated in FIGS. **7**A and/or **7**B. In at least one embodiment, weight parameters may be stored in on-chip or off-chip memory and/or registers (shown or not shown) that configure ALUs of graphics processor **1200** to perform one or more machine learning algorithms, neural network architectures, use cases, or training techniques described herein.

[0147] Such components can be used for data movement.

Virtualized Computing Platform

[0148] FIG. **13** is an example data flow diagram for a process **1300** of generating and deploying an image processing and inferencing pipeline, in accordance with at least one embodiment. In at least one embodiment, process **1300** may be deployed for use with imaging devices, processing devices, and/or other device types at one or more facilities **1302**. Process **1300** may be executed within a training system **1304** and/or a deployment system **1306**. In at least one embodiment, training system **1304** may be used to perform training, deployment, and implementation of machine learning models (e.g., neural networks, object detection algorithms, computer vision algorithms, etc.) for use in deployment system **1306**. In at least one embodiment, deployment system **1306** may be configured to offload processing and compute resources among a distributed computing environment to reduce infrastructure requirements at facility **1302**. In at least one embodiment, one or more applications in a pipeline may use or call upon services (e.g., inference, visualization, compute, AI, etc.) of deployment system **1306** during execution of applications.

[0149] In at least one embodiment, some of applications used in advanced processing and inferencing pipelines may use machine learning models or other AI to perform one or more processing steps. In at least one embodiment, machine learning models may be trained at facility **1302** using data **1308** (such as imaging data) generated at facility **1302** (and stored on one or more picture archiving and communication system (PACS) servers at facility **1302**), may be trained using imaging or sequencing data **1308** from another facility (ies), or a combination thereof. In at least one embodiment, training system **1304** may be used to provide applications, services, and/or other resources for generating working, deployable machine learning models for deployment system **1306**.

[0150] In at least one embodiment, model registry **1324** may be backed by object storage that may support versioning and object metadata. In at least one embodiment, object storage may be accessible through, for example, a cloud storage compatible application programming interface (API) from within a cloud platform. In at least one embodiment, machine learning models within model registry **1324** may uploaded, listed, modified, or deleted by developers or partners of a system interacting with an API. In at least one embodiment, an API may provide access to methods that allow users with appropriate credentials to associate models with applications, such that models may be executed as part of execution of containerized instantiations of applications.

[0151] In at least one embodiment, training system **1304** (FIG. **13**) may include a scenario where facility **1302** is training their own machine learning model, or has an existing machine learning model that needs to be optimized or updated. In at least one embodiment, imaging data **1308** generated by imaging device(s), sequencing devices, and/or other device types may be received. In at least one embodiment, once imaging data **1308** is received, AI-assisted annotation **1310** may be used to aid in generating annotations corresponding to imaging data **1308** to be used as ground truth data for a machine learning model. In at least one embodiment, AI-assisted annotation **1310** may include one or more machine learning models (e.g., convolutional neural networks (CNNs)) that may be trained to generate annotations corresponding to certain types of imaging data **1308** (e.g., from certain devices). In at least one embodiment, AI-assisted annotation **1310** may then be used directly, or may be adjusted or fine-tuned using an annotation tool to generate ground truth data. In at least one embodiment, AI-assisted annotation **1310**, labeled data **1312**, or a combination thereof may be used as ground truth data for training a machine learning model. In at least one embodiment, a trained machine learning model may be referred to as output model(s) **1316**, and may be used by deployment system **1306**, as described herein.

[0152] In at least one embodiment, a training pipeline may include a scenario where facility **1302** needs a machine learning model for use in performing one or more processing tasks for one or more applications in deployment system **1306**, but facility **1302** may not currently have such a machine learning model (or may not have a model that is optimized, efficient, or effective for such purposes). In at least one embodiment, an existing machine learning model may be selected from a model registry **1324**. In at least one embodiment, model registry **1324** may include machine learning models trained to perform a variety of different inference tasks on imaging data. In at least one embodiment, machine learning models in model registry **1324** may have been trained on imaging data from different facilities than facility **1302** (e.g., facilities remotely located). In at least one embodiment, machine learning models may have been trained on imaging data from one location, two locations, or any number of locations. In at least one embodiment, when being trained on imaging data from a specific location, training may take place at that location, or at least in a manner that protects confidentiality of imaging data or restricts imaging data from being transferred off-premises. In at least one embodiment, once a model is trained—or partially trained—at one location, a machine learning model may be added to model registry **1324**. In at least one embodiment, a machine learning model may then be retrained, or updated, at any number of other facilities, and a retrained or updated model may be made available in model registry **1324**. In at least one embodiment, a machine learning model may then be selected from model registry **1324**—and referred to as output model(s) **1316**—and may

be used in deployment system **1306** to perform one or more processing tasks for one or more applications of a deployment system.

[0153] In at least one embodiment, a scenario may include facility **1302** requiring a machine learning model for use in performing one or more processing tasks for one or more applications in deployment system **1306**, but facility **1302** may not currently have such a machine learning model (or may not have a model that is optimized, efficient, or effective for such purposes). In at least one embodiment, a machine learning model selected from model registry **1324** may not be fine-tuned or optimized for imaging data **1308** generated at facility **1302** because of differences in populations, robustness of training data used to train a machine learning model, diversity in anomalies of training data, and/or other issues with training data. In at least one embodiment, AI-assisted annotation **1310** may be used to aid in generating annotations corresponding to imaging data **1308** to be used as ground truth data for retraining or updating a machine learning model. In at least one embodiment, labeled data **1312** may be used as ground truth data for training a machine learning model. In at least one embodiment, retraining or updating a machine learning model may be referred to as model training **1314**. In at least one embodiment, model training **1314**—e.g., AI-assisted annotation **1310**, labeled data **1312**, or a combination thereof—may be used as ground truth data for retraining or updating a machine learning model. In at least one embodiment, a trained machine learning model may be referred to as output model(s) **1316**, and may be used by deployment system **1306**, as described herein.

[0154] In at least one embodiment, deployment system **1306** may include software **1318**, services **1320**, hardware **1322**, and/or other components, features, and functionality. In at least one embodiment, deployment system **1306** may include a software "stack," such that software **1318** may be built on top of services **1320** and may use services **1320** to perform some or all of processing tasks, and services **1320** and software **1318** may be built on top of hardware **1322** and use hardware **1322** to execute processing, storage, and/or other compute tasks of deployment system **1306**. In at least one embodiment, software **1318** may include any number of different containers, where each container may execute an instantiation of an application. In at least one embodiment, each application may perform one or more processing tasks in an advanced processing and inferencing pipeline (e.g., inferencing, object detection, feature detection, segmentation, image enhancement, calibration, etc.). In at least one embodiment, an advanced processing and inferencing pipeline may be defined based on selections of different containers that are desired or required for processing imaging data **1308**, in addition to containers that receive and configure imaging data for use by each container and/or for use by facility **1302** after processing through a pipeline (e.g., to convert outputs back to a usable data type). In at least one embodiment, a combination of containers within software **1318** (e.g., that make up a pipeline) may be referred to as a virtual instrument (as described in more detail herein), and a virtual instrument may leverage services **1320** and hardware **1322** to execute some or all processing tasks of applications instantiated in containers.

[0155] In at least one embodiment, a data processing pipeline may receive input data (e.g., imaging data **1308**) in a specific format in response to an inference request (e.g., a request from a user of deployment system **1306**). In at least one embodiment, input data may be representative of one or more images, video, and/or other data representations generated by one or more imaging devices. In at least one embodiment, data may undergo pre-processing as part of data processing pipeline to prepare data for processing by one or more applications. In at least one embodiment, post-processing may be performed on an output of one or more inferencing tasks or other processing tasks of a pipeline to prepare an output data for a next application and/or to prepare output data for transmission and/or use by a user (e.g., as a response to an inference request). In at least one embodiment, inferencing tasks may be performed by one or more machine learning models, such as trained or deployed neural networks, which may include output model(s) **1316** of training system **1304**.

[0156] In at least one embodiment, tasks of data processing pipeline may be encapsulated in a container(s) that each represents a discrete, fully functional instantiation of an application and virtualized computing environment that is able to reference machine learning models. In at least one embodiment, containers or applications may be published into a private (e.g., limited access) area of a container registry (described in more detail herein), and trained or deployed models may be stored in model registry **1324** and associated with one or more applications. In at least one embodiment, images of applications (e.g., container images) may be available in a container registry, and once selected by a user from a container registry for deployment in a pipeline, an image may be used to generate a container for an instantiation of an application for use by a user's system.

[0157] In at least one embodiment, developers (e.g., software developers, clinicians, doctors, etc.) may develop, publish, and store applications (e.g., as containers) for performing image processing and/or inferencing on supplied data. In at least one embodiment, development, publishing, and/or storing may be performed using a software development kit (SDK) associated with a system (e.g., to ensure that an application and/or container developed is compliant with or compatible with a system). In at least one embodiment, an application that is developed may be tested locally (e.g., at a first facility, on data from a first facility) with an SDK which may support at least some of services **1320** as a system (e.g., system **1200** of FIG. **12**). In at least one embodiment, because DICOM objects may contain anywhere from one to hundreds of images or other data types, and due to a variation in data, a developer may be responsible for managing (e.g., setting constructs for, building pre-processing into an application, etc.) extraction and preparation of incoming data. In at least one embodiment, once validated by system **1300** (e.g., for accuracy), an application may be available in a container registry for selection and/or implementation by a user to perform one or more processing tasks with respect to data at a facility (e.g., a second facility) of a user.

[0158] In at least one embodiment, developers may then share applications or containers through a network for access and use by users of a system (e.g., system **1300** of FIG. **13**). In at least one embodiment, completed and validated applications or containers may be stored in a container registry and associated machine learning models may be stored in model registry **1324**. In at least one embodiment, a requesting entity—who provides an inference or image processing request—may browse a container registry and/or

model registry **1324** for an application, container, dataset, machine learning model, etc., select a desired combination of elements for inclusion in data processing pipeline, and submit an imaging processing request. In at least one embodiment, a request may include input data (and associated patient data, in some examples) that is necessary to perform a request, and/or may include a selection of application(s) and/or machine learning models to be executed in processing a request. In at least one embodiment, a request may then be passed to one or more components of deployment system **1306** (e.g., a cloud) to perform processing of data processing pipeline. In at least one embodiment, processing by deployment system **1306** may include referencing selected elements (e.g., applications, containers, models, etc.) from a container registry and/or model registry **1324**. In at least one embodiment, once results are generated by a pipeline, results may be returned to a user for reference (e.g., for viewing in a viewing application suite executing on a local, on-premises workstation or terminal).

[0159] In at least one embodiment, to aid in processing or execution of applications or containers in pipelines, services **1320** may be leveraged. In at least one embodiment, services **1320** may include compute services, artificial intelligence (AI) services, visualization services, and/or other service types. In at least one embodiment, services **1320** may provide functionality that is common to one or more applications in software **1318**, so functionality may be abstracted to a service that may be called upon or leveraged by applications. In at least one embodiment, functionality provided by services **1320** may run dynamically and more efficiently, while also scaling well by allowing applications to process data in parallel (e.g., using a parallel computing platform **1230** (FIG. **12**)). In at least one embodiment, rather than each application that shares a same functionality offered by services **1320** being required to have a respective instance of services **1320**, services **1320** may be shared between and among various applications. In at least one embodiment, services may include an inference server or engine that may be used for executing detection or segmentation tasks, as non-limiting examples. In at least one embodiment, a model training service may be included that may provide machine learning model training and/or retraining capabilities. In at least one embodiment, a data augmentation service may further be included that may provide GPU accelerated data (e.g., DICOM, RIS, CIS, REST compliant, RPC, raw, etc.) extraction, resizing, scaling, and/or other augmentation. In at least one embodiment, a visualization service may be used that may add image rendering effects—such as ray-tracing, rasterization, denoising, sharpening, etc.—to add realism to two-dimensional (2D) and/or three-dimensional (3D) models. In at least one embodiment, virtual instrument services may be included that provide for beam-forming, segmentation, inferencing, imaging, and/or support for other applications within pipelines of virtual instruments.

[0160] In at least one embodiment, where services **1320** includes an AI service (e.g., an inference service), one or more machine learning models may be executed by calling upon (e.g., as an API call) an inference service (e.g., an inference server) to execute machine learning model(s), or processing thereof, as part of application execution. In at least one embodiment, where another application includes one or more machine learning models for segmentation tasks, an application may call upon an inference service to execute machine learning models for performing one or more of processing operations associated with segmentation tasks. In at least one embodiment, software **1318** implementing advanced processing and inferencing pipeline that includes segmentation application and anomaly detection application may be streamlined because each application may call upon a same inference service to perform one or more inferencing tasks.

[0161] In at least one embodiment, hardware **1322** may include GPUs, CPUs, graphics cards, an AI/deep learning system (e.g., an AI supercomputer, such as NVIDIA's DGX), a cloud platform, or a combination thereof. In at least one embodiment, different types of hardware **1322** may be used to provide efficient, purpose-built support for software **1318** and services **1320** in deployment system **1306**. In at least one embodiment, use of GPU processing may be implemented for processing locally (e.g., at facility **1302**), within an AI/deep learning system, in a cloud system, and/or in other processing components of deployment system **1306** to improve efficiency, accuracy, and efficacy of image processing and generation. In at least one embodiment, software **1318** and/or services **1320** may be optimized for GPU processing with respect to deep learning, machine learning, and/or high-performance computing, as non-limiting examples. In at least one embodiment, at least some of computing environment of deployment system **1306** and/or training system **1304** may be executed in a datacenter one or more supercomputers or high performance computing systems, with GPU optimized software (e.g., hardware and software combination of NVIDIA's DGX System). In at least one embodiment, hardware **1322** may include any number of GPUs that may be called upon to perform processing of data in parallel, as described herein. In at least one embodiment, cloud platform may further include GPU processing for GPU-optimized execution of deep learning tasks, machine learning tasks, or other computing tasks. In at least one embodiment, cloud platform (e.g., NVIDIA's NGC) may be executed using an AI/deep learning supercomputer(s) and/or GPU-optimized software (e.g., as provided on NVIDIA's DGX Systems) as a hardware abstraction and scaling platform. In at least one embodiment, cloud platform may integrate an application container clustering system or orchestration system (e.g., KUBERNETES) on multiple GPUs to enable seamless scaling and load balancing.

[0162] FIG. **14** is a system diagram for an example system **1400** for generating and deploying an imaging deployment pipeline, in accordance with at least one embodiment. In at least one embodiment, system **1400** may be used to implement process **1300** of FIG. **13** and/or other processes including advanced processing and inferencing pipelines. In at least one embodiment, system **1400** may include training system **1304** and deployment system **1306**. In at least one embodiment, training system **1304** and deployment system **1306** may be implemented using software **1318**, services **1320**, and/or hardware **1322**, as described herein.

[0163] In at least one embodiment, system **1400** (e.g., training system **1304** and/or deployment system **1306**) may implemented in a cloud computing environment (e.g., using cloud **1426**). In at least one embodiment, system **1400** may be implemented locally with respect to a healthcare services facility, or as a combination of both cloud and local computing resources. In at least one embodiment, access to APIs in cloud **1426** may be restricted to authorized users through

enacted security measures or protocols. In at least one embodiment, a security protocol may include web tokens that may be signed by an authentication (e.g., AuthN, AuthZ, Gluecon, etc.) service and may carry appropriate authorization. In at least one embodiment, APIs of virtual instruments (described herein), or other instantiations of system **1400**, may be restricted to a set of public IPs that have been vetted or authorized for interaction.

[0164] In at least one embodiment, various components of system **1400** may communicate between and among one another using any of a variety of different network types, including but not limited to local area networks (LANs) and/or wide area networks (WANs) via wired and/or wireless communication protocols. In at least one embodiment, communication between facilities and components of system **1400** (e.g., for transmitting inference requests, for receiving results of inference requests, etc.) may be communicated over data bus(ses), wireless data protocols (Wi-Fi), wired data protocols (e.g., Ethernet), etc.

[0165] In at least one embodiment, training system **1304** may execute training pipelines **1404**, similar to those described herein with respect to FIG. **13**. In at least one embodiment, where one or more machine learning models are to be used in deployment pipeline(s) **1410** by deployment system **1306**, training pipelines **1404** may be used to train or retrain one or more (e.g. pre-trained) models, and/or implement one or more of pre-trained models **1406** (e.g., without a need for retraining or updating). In at least one embodiment, as a result of training pipelines **1404**, output model(s) **1316** may be generated. In at least one embodiment, training pipelines **1404** may include any number of processing steps, such as but not limited to imaging data (or other input data) conversion or adaption In at least one embodiment, for different machine learning models used by deployment system **1306**, different training pipelines **1404** may be used. In at least one embodiment, training pipeline **1404** similar to a first example described with respect to FIG. **13** may be used for a first machine learning model, training pipeline **1404** similar to a second example described with respect to FIG. **13** may be used for a second machine learning model, and training pipeline **1404** similar to a third example described with respect to FIG. **13** may be used for a third machine learning model. In at least one embodiment, any combination of tasks within training system **1304** may be used depending on what is required for each respective machine learning model. In at least one embodiment, one or more of machine learning models may already be trained and ready for deployment so machine learning models may not undergo any processing by training system **1304**, and may be implemented by deployment system **1306**.

[0166] In at least one embodiment, output model(s) **1316** and/or pre-trained models **1406** may include any types of machine learning models depending on implementation or embodiment. In at least one embodiment, and without limitation, machine learning models used by system **1400** may include machine learning model(s) using linear regression, logistic regression, decision trees, support vector machines (SVM), Naïve Bayes, k-nearest neighbor (Knn), K means clustering, random forest, dimensionality reduction algorithms, gradient boosting algorithms, neural networks (e.g., auto-encoders, convolutional, recurrent, perceptrons, Long/Short Term Memory (LSTM), Hopfield, Boltzmann, deep belief, deconvolutional, generative adversarial, liquid state machine, etc.), and/or other types of machine learning models.

[0167] In at least one embodiment, training pipelines **1404** may include AI-assisted annotation, as described in more detail herein with respect to at least FIG. **14**B. In at least one embodiment, labeled data **1312** (e.g., traditional annotation) may be generated by any number of techniques. In at least one embodiment, labels or other annotations may be generated within a drawing program (e.g., an annotation program), a computer aided design (CAD) program, a labeling program, another type of program suitable for generating annotations or labels for ground truth, and/or may be hand drawn, in some examples. In at least one embodiment, ground truth data may be synthetically produced (e.g., generated from computer models or renderings), real produced (e.g., designed and produced from real-world data), machine-automated (e.g., using feature analysis and learning to extract features from data and then generate labels), human annotated (e.g., labeler, or annotation expert, defines location of labels), and/or a combination thereof. In at least one embodiment, for each instance of imaging data **1308** (or other data type used by machine learning models), there may be corresponding ground truth data generated by training system **1304**. In at least one embodiment, AI-assisted annotation may be performed as part of deployment pipeline(s) **1410**; either in addition to, or in lieu of AI-assisted annotation included in training pipelines **1404**. In at least one embodiment, system **1400** may include a multi-layer platform that may include a software layer (e.g., software **1318**) of diagnostic applications (or other application types) that may perform one or more medical imaging and diagnostic functions. In at least one embodiment, system **1400** may be communicatively coupled to (e.g., via encrypted links) PACS server networks of one or more facilities. In at least one embodiment, system **1400** may be configured to access and referenced data from PACS servers to perform operations, such as training machine learning models, deploying machine learning models, image processing, inferencing, and/or other operations.

[0168] In at least one embodiment, a software layer may be implemented as a secure, encrypted, and/or authenticated API through which applications or containers may be invoked (e.g., called) from an external environment(s) (e.g., facility **1302**). In at least one embodiment, applications may then call or execute one or more services **1320** for performing compute, AI, or visualization tasks associated with respective applications, and software **1318** and/or services **1320** may leverage hardware **1322** to perform processing tasks in an effective and efficient manner. In at least one embodiment, communications sent to, or received by, a training system **1304** and a deployment system **1306** may occur using a pair of DICOM adapters **1402**A, **1402**B.

[0169] In at least one embodiment, deployment system **1306** may execute deployment pipeline(s) **1410**. In at least one embodiment, deployment pipeline(s) **1410** may include any number of applications that may be sequentially, non-sequentially, or otherwise applied to imaging data (and/or other data types) generated by imaging devices, sequencing devices, genomics devices, etc.-including AI-assisted annotation, as described above. In at least one embodiment, as described herein, a deployment pipeline(s) **1410** for an individual device may be referred to as a virtual instrument for a device (e.g., a virtual ultrasound instrument, a virtual

CT scan instrument, a virtual sequencing instrument, etc.). In at least one embodiment, for a single device, there may be more than one deployment pipeline(s) **1410** depending on information desired from data generated by a device. In at least one embodiment, where detections of anomalies are desired from an MRI machine, there may be a first deployment pipeline(s) **1410**, and where image enhancement is desired from output of an MRI machine, there may be a second deployment pipeline(s) **1410**.

[0170] In at least one embodiment, an image generation application may include a processing task that includes use of a machine learning model. In at least one embodiment, a user may desire to use their own machine learning model, or to select a machine learning model from model registry **1324**. In at least one embodiment, a user may implement their own machine learning model or select a machine learning model for inclusion in an application for performing a processing task. In at least one embodiment, applications may be selectable and customizable, and by defining constructs of applications, deployment and implementation of applications for a particular user are presented as a more seamless user experience. In at least one embodiment, by leveraging other features of system **1400**—such as services **1320** and hardware **1322**—deployment pipeline(s) **1410** may be even more user friendly, provide for easier integration, and produce more accurate, efficient, and timely results.

[0171] In at least one embodiment, deployment system **1306** may include a user interface ("UI") **1414** (e.g., a graphical user interface, a web interface, etc.) that may be used to select applications for inclusion in deployment pipeline(s) **1410**, arrange applications, modify or change applications or parameters or constructs thereof, use and interact with deployment pipeline(s) **1410** during set-up and/or deployment, and/or to otherwise interact with deployment system **1306**. In at least one embodiment, although not illustrated with respect to training system **1304**, UI **1414** (or a different user interface) may be used for selecting models for use in deployment system **1306**, for selecting models for training, or retraining, in training system **1304**, and/or for otherwise interacting with training system **1304**.

[0172] In at least one embodiment, pipeline manager **1412** may be used, in addition to an application orchestration system **1428**, to manage interaction between applications or containers of deployment pipeline(s) **1410** and services **1320** and/or hardware **1322**. In at least one embodiment, pipeline manager **1412** may be configured to facilitate interactions from application to application, from application to services **1320**, and/or from application or service to hardware **1322**. In at least one embodiment, although illustrated as included in software **1318**, this is not intended to be limiting, and in some examples pipeline manager **1412** may be included in services **1320**. In at least one embodiment, application orchestration system **1428** (e.g., Kubernetes, DOCKER, etc.) may include a container orchestration system that may group applications into containers as logical units for coordination, management, scaling, and deployment. In at least one embodiment, by associating applications from deployment pipeline(s) **1410** (e.g., a reconstruction application, a segmentation application, etc.) with individual containers, each application may execute in a self-contained environment (e.g., at a kernel level) to increase speed and efficiency.

[0173] In at least one embodiment, each application and/or container (or image thereof) may be individually developed, modified, and deployed (e.g., a first user or developer may develop, modify, and deploy a first application and a second user or developer may develop, modify, and deploy a second application separate from a first user or developer), which may allow for focus on, and attention to, a task of a single application and/or container(s) without being hindered by tasks of another application(s) or container(s). In at least one embodiment, communication, and cooperation between different containers or applications may be aided by pipeline manager **1412** and application orchestration system **1428**. In at least one embodiment, so long as an expected input and/or output of each container or application is known by a system (e.g., based on constructs of applications or containers), application orchestration system **1428** and/or pipeline manager **1412** may facilitate communication among and between, and sharing of resources among and between, each of applications or containers. In at least one embodiment, because one or more of applications or containers in deployment pipeline(s) **1410** may share same services and resources, application orchestration system **1428** may orchestrate, load balance, and determine sharing of services or resources between and among various applications or containers. In at least one embodiment, a scheduler may be used to track resource requirements of applications or containers, current usage or planned usage of these resources, and resource availability. In at least one embodiment, a scheduler may thus allocate resources to different applications and distribute resources between and among applications in view of requirements and availability of a system. In some examples, a scheduler (and/or other component of application orchestration system **1428**) may determine resource availability and distribution based on constraints imposed on a system (e.g., user constraints), such as quality of service (QoS), urgency of need for data outputs (e.g., to determine whether to execute real-time processing or delayed processing), etc.

[0174] In at least one embodiment, services **1320** leveraged by and shared by applications or containers in deployment system **1306** may include compute service(s) **1416**, AI service(s) **1418**, visualization service(s) **1420**, and/or other service types. In at least one embodiment, applications may call (e.g., execute) one or more of services **1320** to perform processing operations for an application. In at least one embodiment, compute service(s) **1416** may be leveraged by applications to perform super-computing or other high-performance computing (HPC) tasks. In at least one embodiment, compute service(s) **1416** may be leveraged to perform parallel processing (e.g., using a parallel computing platform **1430**) for processing data through one or more of applications and/or one or more tasks of a single application, substantially simultaneously. In at least one embodiment, parallel computing platform **1430** (e.g., NVIDIA's CUDA) may enable general purpose computing on GPUs (GPGPU) (e.g., GPUs/Graphics **1422**). In at least one embodiment, a software layer of parallel computing platform **1430** may provide access to virtual instruction sets and parallel computational elements of GPUs, for execution of compute kernels. In at least one embodiment, parallel computing platform **1430** may include memory and, in some embodiments, a memory may be shared between and among multiple containers, and/or between and among different processing tasks within a single container. In at least one embodiment, inter-process communication (IPC) calls may be generated for multiple containers and/or for multiple

processes within a container to use same data from a shared segment of memory of parallel computing platform **1430** (e.g., where multiple different stages of an application or multiple applications are processing same information). In at least one embodiment, rather than making a copy of data and moving data to different locations in memory (e.g., a read/write operation), same data in same location of a memory may be used for any number of processing tasks (e.g., at a same time, at different times, etc.). In at least one embodiment, as data is used to generate new data as a result of processing, this information of a new location of data may be stored and shared between various applications. In at least one embodiment, location of data and a location of updated or modified data may be part of a definition of how a payload is understood within containers.

[0175] In at least one embodiment, AI service(s) **1418** may be leveraged to perform inferencing services for executing machine learning model(s) associated with applications (e.g., tasked with performing one or more processing tasks of an application). In at least one embodiment, AI service(s) **1418** may leverage AI system **1424** to execute machine learning model(s) (e.g., neural networks, such as CNNs) for segmentation, reconstruction, object detection, feature detection, classification, and/or other inferencing tasks. In at least one embodiment, applications of deployment pipeline(s) **1410** may use one or more of output model(s) **1316** from training system **1304** and/or other models of applications to perform inference on imaging data. In at least one embodiment, two or more examples of inferencing using application orchestration system **1428** (e.g., a scheduler) may be available. In at least one embodiment, a first category may include a high priority/low latency path that may achieve higher service level agreements, such as for performing inference on urgent requests during an emergency, or for a radiologist during diagnosis. In at least one embodiment, a second category may include a standard priority path that may be used for requests that may be non-urgent or where analysis may be performed at a later time. In at least one embodiment, application orchestration system **1428** may distribute resources (e.g., services **1320** and/or hardware **1322**) based on priority paths for different inferencing tasks of AI service(s) **1418**.

[0176] In at least one embodiment, shared storage may be mounted to AI service(s) **1418** within system **1400**. In at least one embodiment, shared storage may operate as a cache (or other storage device type) and may be used to process inference requests from applications. In at least one embodiment, when an inference request is submitted, a request may be received by a set of API instances of deployment system **1306**, and one or more instances may be selected (e.g., for best fit, for load balancing, etc.) to process a request. In at least one embodiment, to process a request, a request may be entered into a database, a machine learning model may be located from model registry **1324** if not already in a cache, a validation step may ensure appropriate machine learning model is loaded into a cache (e.g., shared storage), and/or a copy of a model may be saved to a cache. In at least one embodiment, a scheduler (e.g., of pipeline manager **1412**) may be used to launch an application that is referenced in a request if an application is not already running or if there are not enough instances of an application. In at least one embodiment, if an inference server is not already launched to execute a model, an inference server may be launched. Any number of inference servers may be launched per model. In at least one embodiment, in a pull model, in which inference servers are clustered, models may be cached whenever load balancing is advantageous. In at least one embodiment, inference servers may be statically loaded in corresponding, distributed servers.

[0177] In at least one embodiment, inferencing may be performed using an inference server that runs in a container. In at least one embodiment, an instance of an inference server may be associated with a model (and optionally a plurality of versions of a model). In at least one embodiment, if an instance of an inference server does not exist when a request to perform inference on a model is received, a new instance may be loaded. In at least one embodiment, when starting an inference server, a model may be passed to an inference server such that a same container may be used to serve different models so long as inference server is running as a different instance.

[0178] In at least one embodiment, during application execution, an inference request for a given application may be received, and a container (e.g., hosting an instance of an inference server) may be loaded (if not already), and a start procedure may be called. In at least one embodiment, pre-processing logic in a container may load, decode, and/or perform any additional pre-processing on incoming data (e.g., using a CPU(s) and/or GPU(s)). In at least one embodiment, once data is prepared for inference, a container may perform inference as necessary on data. In at least one embodiment, this may include a single inference call on one image (e.g., a hand X-ray), or may require inference on hundreds of images (e.g., a chest CT). In at least one embodiment, an application may summarize results before completing, which may include, without limitation, a single confidence score, pixel level-segmentation, voxel-level segmentation, generating a visualization, or generating text to summarize findings. In at least one embodiment, different models or applications may be assigned different priorities. For example, some models may have a real-time (TAT<1 min) priority while others may have lower priority (e.g., TAT<10 min). In at least one embodiment, model execution times may be measured from requesting institution or entity and may include partner network traversal time, as well as execution on an inference service.

[0179] In at least one embodiment, transfer of requests between services **1320** and inference applications may be hidden behind a software development kit (SDK), and robust transport may be provide through a queue. In at least one embodiment, a request will be placed in a queue via an API for an individual application/tenant ID combination and an SDK will pull a request from a queue and give a request to an application. In at least one embodiment, a name of a queue may be provided in an environment from where an SDK will pick it up. In at least one embodiment, asynchronous communication through a queue may be useful as it may allow any instance of an application to pick up work as it becomes available. Results may be transferred back through a queue, to ensure no data is lost. In at least one embodiment, queues may also provide an ability to segment work, as highest priority work may go to a queue with most instances of an application connected to it, while lowest priority work may go to a queue with a single instance connected to it that processes tasks in an order received. In at least one embodiment, an application may run on a GPU-accelerated instance generated in cloud **1426**, and an inference service may perform inferencing on a GPU.

[0180] In at least one embodiment, visualization service(s) **1420** may be leveraged to generate visualizations for viewing outputs of applications and/or deployment pipeline(s) **1410**. In at least one embodiment, GPUs/Graphics **1422** may be leveraged by visualization service(s) **1420** to generate visualizations. In at least one embodiment, rendering effects, such as ray-tracing, may be implemented by visualization service(s) **1420** to generate higher quality visualizations. In at least one embodiment, visualizations may include, without limitation, 2D image renderings, 3D volume renderings, 3D volume reconstruction, 2D tomographic slices, virtual reality displays, augmented reality displays, etc. In at least one embodiment, virtualized environments may be used to generate a virtual interactive display or environment (e.g., a virtual environment) for interaction by users of a system (e.g., doctors, nurses, radiologists, etc.). In at least one embodiment, visualization service(s) **1420** may include an internal visualizer, cinematics, and/or other rendering or image processing capabilities or functionality (e.g., ray tracing, rasterization, internal optics, etc.).

[0181] In at least one embodiment, hardware **1322** may include GPUs/Graphics **1422**, AI system **1424**, cloud **1426**, and/or any other hardware used for executing training system **1304** and/or deployment system **1306**. In at least one embodiment, GPUs/Graphics **1422** (e.g., NVIDIA's TESLA and/or QUADRO GPUs) may include any number of GPUs that may be used for executing processing tasks of compute service(s) **1416**, AI service(s) **1418**, visualization service(s) **1420**, other services, and/or any of features or functionality of software **1318**. For example, with respect to AI service(s) **1418**, GPUs/Graphics **1422** may be used to perform pre-processing on imaging data (or other data types used by machine learning models), post-processing on outputs of machine learning models, and/or to perform inferencing (e.g., to execute machine learning models). In at least one embodiment, cloud **1426**, AI system **1424**, and/or other components of system **1400** may use GPUs/Graphics **1422**. In at least one embodiment, cloud **1426** may include a GPU-optimized platform for deep learning tasks. In at least one embodiment, AI system **1424** may use GPUs, and cloud **1426**—or at least a portion tasked with deep learning or inferencing-may be executed using one or more AI systems **1424**. As such, although hardware **1322** is illustrated as discrete components, this is not intended to be limiting, and any components of hardware **1322** may be combined with, or leveraged by, any other components of hardware **1322**.

[0182] In at least one embodiment, AI system **1424** may include a purpose-built computing system (e.g., a supercomputer or an HPC) configured for inferencing, deep learning, machine learning, and/or other artificial intelligence tasks. In at least one embodiment, AI system **1424** (e.g., NVIDIA's DGX) may include GPU-optimized software (e.g., a software stack) that may be executed using a plurality of GPUs/Graphics **1422**, in addition to CPUs, RAM, storage, and/or other components, features, or functionality. In at least one embodiment, one or more AI systems **1424** may be implemented in cloud **1426** (e.g., in a data center) for performing some or all of AI-based processing tasks of system **1400**.

[0183] In at least one embodiment, cloud **1426** may include a GPU-accelerated infrastructure (e.g., NVIDIA's NGC) that may provide a GPU-optimized platform for executing processing tasks of system **1400**. In at least one embodiment, cloud **1426** may include an AI system **1424** for performing one or more of AI-based tasks of system **1400** (e.g., as a hardware abstraction and scaling platform). In at least one embodiment, cloud **1426** may integrate with application orchestration system **1428** leveraging multiple GPUs to enable seamless scaling and load balancing between and among applications and services **1320**. In at least one embodiment, cloud **1426** may tasked with executing at least some of services **1320** of system **1400**, including compute service(s) **1416**, AI service(s) **1418**, and/or visualization service(s) **1420**, as described herein. In at least one embodiment, cloud **1426** may perform small and large batch inference (e.g., executing NVIDIA's TENSOR RT), provide an accelerated parallel computing API and platform **1430** (e.g., NVIDIA's CUDA), execute application orchestration system **1428** (e.g., KUBERNETES), provide a graphics rendering API and platform (e.g., for ray-tracing, 2D graphics, 3D graphics, and/or other rendering techniques to produce higher quality cinematics), and/or may provide other functionality for system **1400**.

[0184] FIG. **15**A illustrates a data flow diagram for a process **1500** to train, retrain, or update a machine learning model, in accordance with at least one embodiment. In at least one embodiment, process **1500** may be executed using, as a non-limiting example, system **1400** of FIG. **14**. In at least one embodiment, process **1500** may leverage services and/or hardware as described herein. In at least one embodiment, refined models **1512** generated by process **1500** may be executed by a deployment system for one or more containerized applications in deployment pipelines.

[0185] In at least one embodiment, model training **1514** may include retraining or updating an initial model **1504** (e.g., a pre-trained model) using new training data (e.g., new input data, such as customer dataset **1506**, and/or new ground truth data associated with input data). In at least one embodiment, to retrain, or update, initial model **1504**, output or loss layer(s) of initial model **1504** may be reset, deleted, and/or replaced with an updated or new output or loss layer(s). In at least one embodiment, initial model **1504** may have previously fine-tuned parameters (e.g., weights and/or biases) that remain from prior training, so training or retraining **1514** may not take as long or require as much processing as training a model from scratch. In at least one embodiment, during model training **1514**, by having reset or replaced output or loss layer(s) of initial model **1504**, parameters may be updated and re-tuned for a new data set based on loss calculations associated with accuracy of output or loss layer(s) at generating predictions on new, customer dataset **1506**.

[0186] In at least one embodiment, pre-trained models **1506** may be stored in a data store, or registry. In at least one embodiment, pre-trained models **1506** may have been trained, at least in part, at one or more facilities other than a facility executing process **1500**. In at least one embodiment, to protect privacy and rights of patients, subjects, or clients of different facilities, pre-trained models **1506** may have been trained, on-premise, using customer or patient data generated on-premise. In at least one embodiment, pre-trained models **1306** may be trained using a cloud and/or other hardware, but confidential, privacy protected patient data may not be transferred to, used by, or accessible to any components of a cloud (or other off premise hardware). In at least one embodiment, where pre-trained models **1506** is trained at using patient data from more than one facility, pre-trained models **1506** may have been individually trained

for each facility prior to being trained on patient or customer data from another facility. In at least one embodiment, such as where a customer or patient data has been released of privacy concerns (e.g., by waiver, for experimental use, etc.), or where a customer or patient data is included in a public data set, a customer or patient data from any number of facilities may be used to train pre-trained models **1506** on-premise and/or off premise, such as in a datacenter or other cloud computing infrastructure.

[0187] In at least one embodiment, when selecting applications for use in deployment pipelines, a user may also select machine learning models to be used for specific applications. In at least one embodiment, a user may not have a model for use, so a user may select a pre-trained model to use with an application. In at least one embodiment, pre-trained model may not be optimized for generating accurate results on customer dataset **1506** of a facility of a user (e.g., based on patient diversity, demographics, types of medical imaging devices used, etc.). In at least one embodiment, prior to deploying a pre-trained model into a deployment pipeline for use with an application(s), pre-trained model may be updated, retrained, and/or fine-tuned for use at a respective facility.

[0188] In at least one embodiment, a user may select pre-trained model that is to be updated, retrained, and/or fine-tuned, and this pre-trained model may be referred to as initial model **1504** for a training system within process **1500**. In at least one embodiment, a customer dataset **1506** (e.g., imaging data, genomics data, sequencing data, or other data types generated by devices at a facility) may be used to perform model training (which may include, without limitation, transfer learning) on initial model **1504** to generate refined model **1512**. In at least one embodiment, ground truth data corresponding to customer dataset **1506** may be generated by training system **1304**. In at least one embodiment, ground truth data may be generated, at least in part, by clinicians, scientists, doctors, practitioners, at a facility.

[0189] In at least one embodiment, AI-assisted annotation may be used in some examples to generate ground truth data. In at least one embodiment, AI-assisted annotation (e.g., implemented using an AI-assisted annotation SDK) may leverage machine learning models (e.g., neural networks) to generate suggested or predicted ground truth data for a customer dataset. In at least one embodiment, a user may use annotation tools within a user interface (a graphical user interface (GUI)) on a computing device.

[0190] In at least one embodiment, user **1510** may interact with a GUI via computing device **1508** to edit or fine-tune (auto) annotations. In at least one embodiment, a polygon editing feature may be used to move vertices of a polygon to more accurate or fine-tuned locations.

[0191] In at least one embodiment, once customer dataset **1506** has associated ground truth data, ground truth data (e.g., from AI-assisted annotation, manual labeling, etc.) may be used by during model training to generate refined model **1512**. In at least one embodiment, customer dataset **1506** may be applied to initial model **1504** any number of times, and ground truth data may be used to update parameters of initial model **1504** until an acceptable level of accuracy is attained for refined model **1512**. In at least one embodiment, once refined model **1512** is generated, refined model **1512** may be deployed within one or more deployment pipelines at a facility for performing one or more processing tasks with respect to medical imaging data.

[0192] In at least one embodiment, refined model **1512** may be uploaded to pre-trained models in a model registry to be selected by another facility. In at least one embodiment, this process may be completed at any number of facilities such that refined model **1512** may be further refined on new datasets any number of times to generate a more universal model.

[0193] FIG. **15**B is an example illustration of a client-server architecture **1532** to enhance annotation tools with pre-trained annotation models, in accordance with at least one embodiment. In at least one embodiment, AI-assisted annotation tool **1536** may be instantiated based on a client-server architecture **1532**. In at least one embodiment, AI-assisted annotation tool **1536** in imaging applications may aid radiologists, for example, identify organs and abnormalities. In at least one embodiment, imaging applications may include software tools that help user **1510** to identify, as a non-limiting example, a few extreme points on a particular organ of interest in raw images **1534** (e.g., in a 3D MRI or CT scan) and receive auto-annotated results for all 2D slices of a particular organ. In at least one embodiment, results may be stored in a data store as training data **1538** and used as (for example and without limitation) ground truth data for training. In at least one embodiment, when computing device **1508** sends extreme points for AI-assisted annotation, a deep learning model, for example, may receive this data as input and return inference results of a segmented organ or abnormality. In at least one embodiment, pre-instantiated annotation tools, such as AI-assisted annotation tool **1536** in FIG. **15**B, may be enhanced by making API calls (e.g., API Call **1544**) to a server, such as an Annotation Assistant Server **1540** that may include a set of pre-trained models **1542** stored in an annotation model registry, for example. In at least one embodiment, an annotation model registry may store pre-trained models **1542** (e.g., machine learning models, such as deep learning models) that are pre-trained to perform AI-assisted annotation on a particular organ or abnormality. These models may be further updated by using training pipelines. In at least one embodiment, pre-installed annotation tools may be improved over time as new labeled data is added.

[0194] Various embodiments can be described by the following clauses:

[0195] 1. A computer-implemented method, comprising:

[0196] setting a first memory segment to a first status value based at least on a portion of data within the first memory segment being read to a registry;

[0197] determining a write destination for the portion of data;

[0198] determining a second memory segment, corresponding to the write destination, has a same status value as the first status value; and

[0199] based on the second memory segment having a same status value as the first status, writing the chunk of data from the registry to the second memory segment.

[0200] 2. The computer-implemented method of clause 1, further comprising:

[0201] determining a second write destination for a second portion of data from the second memory segment;

[0202] determining a third memory segment corresponding to the second write destination has a second status value;

[0203] reading a third portion of data from the third memory segment into the registry;

[0204] setting the third memory segment to a third status value; and

[0205] writing the second portion of data from the registry to the third memory segment.

[0206] 3. The computer-implemented method of clause 2, wherein the second status value corresponds to an unread status.

[0207] 4. The computer-implemented method of clause 1, wherein the first status value corresponds to a read status.

[0208] 5. The computer-implemented method of clause 1, wherein a first worker is used to set the first status value of the first memory segment and a second worker is used to set a status value of the second memory segment.

[0209] 6. The computer-implemented method of clause 1, wherein a second portion of data from the second memory segment is read into a second registry.

[0210] 7. The computer-implemented method of clause 1, further comprising:

[0211] determining a number of processing units to execute a data movement operation.

[0212] 8. The computer-implemented method of clause 7, further comprising:

[0213] determining a number of contiguous portions for a new data object;

[0214] determining an existing number of contiguous portions is less than the number; and

[0215] selecting the write destination based at least on the number, wherein the write destination forms the number of contiguous portions for the new data object.

[0216] 9. A processor, comprising:

[0217] one or more circuits to:

[0218] read a first data segment into a registry from an unread memory location;

[0219] set an indicator to change the unread memory location to a first read memory location;

[0220] determine a destination memory location for the first data segment is a second read memory location; and

[0221] write the first data segment from the registry to the second read memory location.

[0222] 10. The processor of clause 9, wherein two or more workers execute, in parallel, to read the unread memory location and the second read memory location.

[0223] 11. The processor of clause 9, wherein the destination memory location is determined based at least on an updated data object to be added to a memory buffer.

[0224] 12. The processor of clause 9, wherein the one or more circuits are further to:

[0225] determine a second destination memory location for a second data segment read from the second memory location;

[0226] determine the second destination memory location is unread;

[0227] read a third data segment from the second destination memory location;

[0228] set a second indicator to change the second destination memory location into a third read memory location; and

[0229] write the second data segment from the registry to the third read memory location.

[0230] 13. The processor of clause 9, wherein the one or more circuits are further to:

[0231] determine a total number of data segments for a task; and

[0232] determine a number of processing units to execute the task based on the total number of data segments.

[0233] 14. The processor of clause 9, wherein the processor is comprised in at least one of:

[0234] a system for performing simulation operations;

[0235] a system for performing simulation operations to test or validate autonomous machine applications;

[0236] a system for performing digital twin operations;

[0237] a system for performing light transport simulation;

[0238] a system for rendering graphical output;

[0239] a system for performing deep learning operations;

[0240] a system implemented using an edge device;

[0241] a system for generating or presenting virtual reality (VR) content;

[0242] a system for generating or presenting augmented reality (AR) content;

[0243] a system for generating or presenting mixed reality (MR) content;

[0244] a system incorporating one or more Virtual Machines (VMs);

[0245] a system for performing operations for a conversational AI application;

[0246] a system for performing operations for a generative AI application;

[0247] a system for performing operations using a language model;

[0248] a system for performing one or more generative content operations using a large language model (LLM);

[0249] a system implemented at least partially in a data center;

[0250] a system for performing hardware testing using simulation;

[0251] a system for performing one or more generative content operations using a language model;

[0252] a system for synthetic data generation;

[0253] a collaborative content creation platform for 3D assets; or

[0254] a system implemented at least partially using cloud computing resources.

[0255] 15. A system, comprising:

[0256] one or more processing circuits to read a portion of data from an unprocessed memory segment to a registry, change a first status identifier for the unprocessed memory segment, and to write the portion of data to a destination memory segment from the registry upon determining a second status identifier for the destination memory segment meets a predetermined value.

[0257] 16. The system of clause 15, wherein the predetermined value is indicative that a second portion of data associated with the destination memory segment is stored within a registry.

[0258] 17. The system of clause 15, wherein a first unit of processing reads the portion of data from the unprocessed memory segment and a second unit of processing sets the second status identifier.

[0259] 18. The system of clause 15, wherein a first unit of processing reads the portion of data from the unprocessed memory segment and the first unit of processing sets the second status identifier.

[0260] 19. The system of clause 15, wherein the one or more processing circuits are further to select the destination memory segment to form a contiguous block of available memory segments.

[0261] 20. The system of clause 15, wherein the system is one of:

[0262] a system for performing simulation operations;

[0263] a system for performing simulation operations to test or validate autonomous machine applications;

[0264] a system for performing digital twin operations;

[0265] a system for performing light transport simulation;

[0266] a system for rendering graphical output;

[0267] a system for performing deep learning operations;

[0268] a system implemented using an edge device;

[0269] a system for generating or presenting virtual reality (VR) content;

[0270] a system for generating or presenting augmented reality (AR) content;

[0271] a system for generating or presenting mixed reality (MR) content;

[0272] a system incorporating one or more Virtual Machines (VMs);

[0273] a system for performing operations for a conversational AI application;

[0274] a system for performing operations for a generative AI application;

[0275] a system for performing operations using a language model;

[0276] a system for performing one or more generative content operations using a large language model (LLM);

[0277] a system implemented at least partially in a data center;

[0278] a system for performing hardware testing using simulation;

[0279] a system for performing one or more generative content operations using a language model;

[0280] a system for synthetic data generation;

[0281] a collaborative content creation platform for 3D assets; or

[0282] a system implemented at least partially using cloud computing resources.

[0283] Other variations are within spirit of present disclosure. Thus, while disclosed techniques are susceptible to various modifications and alternative constructions, certain illustrated embodiments thereof are shown in drawings and have been described above in detail. It should be understood, however, that there is no intention to limit disclosure to specific form or forms disclosed, but on contrary, intention is to cover all modifications, alternative constructions, and equivalents falling within spirit and scope of disclosure, as defined in appended claims.

[0284] Use of terms "a" and "an" and "the" and similar referents in context of describing disclosed embodiments (especially in context of following claims) are to be construed to cover both singular and plural, unless otherwise indicated herein or clearly contradicted by context, and not as a definition of a term. Terms "comprising," "having," "including," and "containing" are to be construed as open-ended terms (meaning "including, but not limited to,") unless otherwise noted. Term "connected," when unmodified and referring to physical connections, is to be construed as partly or wholly contained within, attached to, or joined together, even if there is something intervening. Recitation of ranges of values herein are merely intended to serve as a shorthand method of referring individually to each separate value falling within range, unless otherwise indicated herein and each separate value is incorporated into specification as if it were individually recited herein. Use of term "set" (e.g., "a set of items") or "subset," unless otherwise noted or contradicted by context, is to be construed as a nonempty collection comprising one or more members. Further, unless otherwise noted or contradicted by context, term "subset" of a corresponding set does not necessarily denote a proper subset of corresponding set, but subset and corresponding set may be equal.

[0285] Conjunctive language, such as phrases of form "at least one of A, B, and C," or "at least one of A, B and C," unless specifically stated otherwise or otherwise clearly contradicted by context, is otherwise understood with context as used in general to present that an item, term, etc., may be either A or B or C, or any nonempty subset of set of A and B and C. For instance, in illustrative example of a set having three members, conjunctive phrases "at least one of A, B, and C" and "at least one of A, B and C" refer to any of following sets: {A}, {B}, {C}, {A, B}, {A, C}, {B, C}, {A, B, C}. Thus, such conjunctive language is not generally intended to imply that certain embodiments require at least one of A, at least one of B, and at least one of C each to be present. In addition, unless otherwise noted or contradicted by context, term "plurality" indicates a state of being plural (e.g., "a plurality of items" indicates multiple items). A plurality is at least two items, but can be more when so indicated either explicitly or by context. Further, unless stated otherwise or otherwise clear from context, phrase "based on" means "based at least in part on" and not "based solely on."

[0286] Operations of processes described herein can be performed in any suitable order unless otherwise indicated herein or otherwise clearly contradicted by context. In at least one embodiment, a process such as those processes described herein (or variations and/or combinations thereof) is performed under control of one or more computer systems configured with executable instructions and is implemented as code (e.g., executable instructions, one or more computer programs or one or more applications) executing collectively on one or more processors, by hardware or combinations thereof. In at least one embodiment, code is stored on a computer-readable storage medium, for example, in form of a computer program comprising a plurality of instructions executable by one or more processors. In at least one embodiment, a computer-readable storage medium is a non-transitory computer-readable storage medium that excludes transitory signals (e.g., a propagating transient electric or electromagnetic transmission) but includes non-transitory data storage circuitry (e.g., buffers, cache, and queues) within transceivers of transitory signals. In at least one embodiment, code (e.g., executable code or source code) is stored on a set of one or more non-transitory computer-readable storage media having stored thereon executable instructions (or other memory to store executable instructions) that, when executed (i.e., as a result of being

executed) by one or more processors of a computer system, cause computer system to perform operations described herein. A set of non-transitory computer-readable storage media, in at least one embodiment, comprises multiple non-transitory computer-readable storage media and one or more of individual non-transitory storage media of multiple non-transitory computer-readable storage media lack all of code while multiple non-transitory computer-readable storage media collectively store all of code. In at least one embodiment, executable instructions are executed such that different instructions are executed by different processors-for example, a non-transitory computer-readable storage medium store instructions and a main central processing unit ("CPU") executes some of instructions while a graphics processing unit ("GPU") executes other instructions. In at least one embodiment, different components of a computer system have separate processors and different processors execute different subsets of instructions.

[0287] Accordingly, in at least one embodiment, computer systems are configured to implement one or more services that singly or collectively perform operations of processes described herein and such computer systems are configured with applicable hardware and/or software that enable performance of operations. Further, a computer system that implements at least one embodiment of present disclosure is a single device and, in another embodiment, is a distributed computer system comprising multiple devices that operate differently such that distributed computer system performs operations described herein and such that a single device does not perform all operations.

[0288] Use of any and all examples, or exemplary language (e.g., "such as") provided herein, is intended merely to better illuminate embodiments of disclosure and does not pose a limitation on scope of disclosure unless otherwise claimed. No language in specification should be construed as indicating any non-claimed element as essential to practice of disclosure.

[0289] All references, including publications, patent applications, and patents, cited herein are hereby incorporated by reference to same extent as if each reference were individually and specifically indicated to be incorporated by reference and were set forth in its entirety herein.

[0290] In description and claims, terms "coupled" and "connected," along with their derivatives, may be used. It should be understood that these terms may be not intended as synonyms for each other. Rather, in particular examples, "connected" or "coupled" may be used to indicate that two or more elements are in direct or indirect physical or electrical contact with each other. "Coupled" may also mean that two or more elements are not in direct contact with each other, but yet still co-operate or interact with each other.

[0291] Unless specifically stated otherwise, it may be appreciated that throughout specification terms such as "processing," "computing," "calculating," "determining," or like, refer to action and/or processes of a computer or computing system, or similar electronic computing device, that manipulate and/or transform data represented as physical, such as electronic, quantities within computing system's registers and/or memories into other data similarly represented as physical quantities within computing system's memories, registers or other such information storage, transmission or display devices.

[0292] In a similar manner, term "processor" may refer to any device or portion of a device that processes electronic data from registers and/or memory and transform that electronic data into other electronic data that may be stored in registers and/or memory. As non-limiting examples, "processor" may be a CPU or a GPU. A "computing platform" may comprise one or more processors. As used herein, "software" processes may include, for example, software and/or hardware entities that perform work over time, such as tasks, threads, and intelligent agents. Also, each process may refer to multiple processes, for carrying out instructions in sequence or in parallel, continuously or intermittently. Terms "system" and "method" are used herein interchangeably insofar as system may embody one or more methods and methods may be considered a system.

[0293] In present document, references may be made to obtaining, acquiring, receiving, or inputting analog or digital data into a subsystem, computer system, or computer-implemented machine. Obtaining, acquiring, receiving, or inputting analog and digital data can be accomplished in a variety of ways such as by receiving data as a parameter of a function call or a call to an application programming interface. In some implementations, process of obtaining, acquiring, receiving, or inputting analog or digital data can be accomplished by transferring data via a serial or parallel interface. In another implementation, process of obtaining, acquiring, receiving, or inputting analog or digital data can be accomplished by transferring data via a computer network from providing entity to acquiring entity. References may also be made to providing, outputting, transmitting, sending, or presenting analog or digital data. In various examples, process of providing, outputting, transmitting, sending, or presenting analog or digital data can be accomplished by transferring data as an input or output parameter of a function call, a parameter of an application programming interface or interprocess communication mechanism.

[0294] Although discussion above sets forth example implementations of described techniques, other architectures may be used to implement described functionality, and are intended to be within scope of this disclosure. Furthermore, although specific distributions of responsibilities are defined above for purposes of discussion, various functions and responsibilities might be distributed and divided in different ways, depending on circumstances.

[0295] Furthermore, although subject matter has been described in language specific to structural features and/or methodological acts, it is to be understood that subject matter claimed in appended claims is not necessarily limited to specific features or acts described. Rather, specific features and acts are disclosed as exemplary forms of implementing the claims.

What is claimed is:

1. A computer-implemented method, comprising:

setting a first memory segment to a first status value based at least on a portion of data within the first memory segment being read to a registry;

determining a write destination for the portion of data;

determining a second memory segment, corresponding to the write destination, has a same status value as the first status value; and

based on the second memory segment having a same status value as the first status, writing the chunk of data from the registry to the second memory segment.

2. The computer-implemented method of claim **1**, further comprising:

determining a second write destination for a second portion of data from the second memory segment;

determining a third memory segment corresponding to the second write destination has a second status value;

reading a third portion of data from the third memory segment into the registry;

setting the third memory segment to a third status value; and

writing the second portion of data from the registry to the third memory segment.

3. The computer-implemented method of claim **2**, wherein the second status value corresponds to an unread status.

4. The computer-implemented method of claim **1**, wherein the first status value corresponds to a read status.

5. The computer-implemented method of claim **1**, wherein a first worker is used to set the first status value of the first memory segment and a second worker is used to set a status value of the second memory segment.

6. The computer-implemented method of claim **1**, wherein a second portion of data from the second memory segment is read into a second registry.

7. The computer-implemented method of claim **1**, further comprising:

determining a number of processing units to execute a data movement operation.

8. The computer-implemented method of claim **7**, further comprising:

determining a number of contiguous portions for a new data object;

determining an existing number of contiguous portions is less than the number; and

selecting the write destination based at least on the number, wherein the write destination forms the number of contiguous portions for the new data object.

9. A processor, comprising:

one or more circuits to:

read a first data segment into a registry from an unread memory location;

set an indicator to change the unread memory location to a first read memory location;

determine a destination memory location for the first data segment is a second read memory location; and

write the first data segment from the registry to the second read memory location.

10. The processor of claim **9**, wherein two or more workers execute, in parallel, to read the unread memory location and the second read memory location.

11. The processor of claim **9**, wherein the destination memory location is determined based at least on an updated data object to be added to a memory buffer.

12. The processor of claim **9**, wherein the one or more circuits are further to:

determine a second destination memory location for a second data segment read from the second memory location;

determine the second destination memory location is unread;

read a third data segment from the second destination memory location;

set a second indicator to change the second destination memory location into a third read memory location; and

write the second data segment from the registry to the third read memory location.

13. The processor of claim **9**, wherein the one or more circuits are further to:

determine a total number of data segments for a task; and

determine a number of processing units to execute the task based on the total number of data segments.

14. The processor of claim **9**, wherein the processor is comprised in at least one of:

a system for performing simulation operations;

a system for performing simulation operations to test or validate autonomous machine applications;

a system for performing digital twin operations;

a system for performing light transport simulation;

a system for rendering graphical output;

a system for performing deep learning operations;

a system implemented using an edge device;

a system for generating or presenting virtual reality (VR) content;

a system for generating or presenting augmented reality (AR) content;

a system for generating or presenting mixed reality (MR) content;

a system incorporating one or more Virtual Machines (VMs);

a system for performing operations for a conversational AI application;

a system for performing operations for a generative AI application;

a system for performing operations using a language model;

a system for performing one or more generative content operations using a large language model (LLM);

a system implemented at least partially in a data center;

a system for performing hardware testing using simulation;

a system for performing one or more generative content operations using a language model;

a system for synthetic data generation;

a collaborative content creation platform for 3D assets; or

a system implemented at least partially using cloud computing resources.

15. A system, comprising:

one or more processing circuits to read a portion of data from an unprocessed memory segment to a registry, change a first status identifier for the unprocessed memory segment, and to write the portion of data to a destination memory segment from the registry upon determining a second status identifier for the destination memory segment meets a predetermined value.

16. The system of claim **15**, wherein the predetermined value is indicative that a second portion of data associated with the destination memory segment is stored within a registry.

17. The system of claim **15**, wherein a first unit of processing reads the portion of data from the unprocessed memory segment and a second unit of processing sets the second status identifier.

18. The system of claim **15**, wherein a first unit of processing reads the portion of data from the unprocessed memory segment and the first unit of processing sets the second status identifier.

19. The system of claim **15**, wherein the one or more processing circuits are further to select the destination memory segment to form a contiguous block of available memory segments.

20. The system of claim **15**, wherein the system is one of:

a system for performing simulation operations;

a system for performing simulation operations to test or validate autonomous machine applications;

a system for performing digital twin operations;

a system for performing light transport simulation;

a system for rendering graphical output;

a system for performing deep learning operations;

a system implemented using an edge device;

a system for generating or presenting virtual reality (VR) content;

a system for generating or presenting augmented reality (AR) content;

a system for generating or presenting mixed reality (MR) content;

a system incorporating one or more Virtual Machines (VMs);

a system for performing operations for a conversational AI application;

a system for performing operations for a generative AI application;

a system for performing operations using a language model;

a system for performing one or more generative content operations using a large language model (LLM);

a system implemented at least partially in a data center;

a system for performing hardware testing using simulation;

a system for performing one or more generative content operations using a language model;

a system for synthetic data generation;

a collaborative content creation platform for 3D assets; or

a system implemented at least partially using cloud computing resources.

* * * * *